كلية الحوسبة والمعلوماتية |
College of Computing and Informatics
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

College of Computing & Informatics (CCI)

SENIOR PROJECT-II REPORT

SeeStack: AI-Powered Error Detection and Monitoring System for Developers

Author(s):

S220017052	ABDULAZIZ ALOTAIBI (LEADER)
S220003861	FAISAL ALGHANIM
S220006504	OMAR ALALLAF
S210054825	MOHAMMED BIN EID
S190173769	SALEM OMRAN
S220011106	MOHAMMED TALEB

Project Supervisor:

Dr. Khaled Alwasel

كلية الحوسبة والمعلوماتية |
College of Computing and Informatics
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

SeeStack: AI-Powered Error Detection and Monitoring System for Developers

By: **ABDULAZIZ ALOTAIBI, FAISAL ALGHANIM, OMAR ALALLAF, MOHAMMED BIN EID, SALEM OMRAN, MOHAMMED TALEB**

Thesis/Project submitted to:

College of Computing & Informatics, Saudi Electronic University, Riyadh, Saudi Arabia.

In partial fulfillment of the requirements for the degree of:

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

Project Supervisor	Project Committee Chair

ABSTRACT

In today's digital landscape, organizations rely on seamless software performance and robust security. But developers are too often challenged by fragmented monitoring tools, cryptic error logs, and manual analysis that prolong outages and degrade user experience. Modern distributed systems intensify this challenge, as a single failure can cascade through multiple services, making it increasingly difficult to diagnose and resolve incidents efficiently. Development teams waste precious time piecing together disparate performance metrics, stress testing results, and vulnerability reports, leading to costly downtime and a persistent cycle of reactive firefighting. These challenges are further exacerbated by the demand for faster delivery cycles and growing system complexity, pushing teams to the limit of what can be managed manually.

SeeStack addresses these issues through a unified, AI-powered platform for error detection, performance monitoring, stress testing, and proactive security scanning. By integrating advanced analytics, natural language processing, and predictive modeling, the platform interprets raw telemetry, automates error classification and grouping, and delivers clear recommendations in real time. Developers benefit from actionable insights through a single dashboard. Reducing the need to juggle multiple tools while empowering efficient root cause analysis, continuous performance optimization, and early vulnerability mitigation. The streamlined user experience enables faster onboarding of new team members and ensures best practices are embedded into daily operations. This approach helps standardize incident response and enhances cross-team communication by providing everyone with consistent, contextual information.

Key contributions of SeeStack include dramatic reductions in mean time to resolution (MTTR) and false positive alerts through intelligent event correlation, as well as improved system reliability and user experience. Automated load simulations and AI-driven log analysis help teams identify problems before end-users are impacted, while the platform's actionable guidance supports rapid remediation and long-term infrastructure resilience. The system's scalability accommodates organizations of various sizes, supporting growth without sacrificing performance. As a validated, production-ready solution, SeeStack enables organizations to shift from a culture of incident response to one of innovation and stability. Ultimately, the platform empowers software teams to deliver higher quality services, reduces operational overhead, and strengthens both technical and business outcomes.

DEDICATION

We would like to express our sincere gratitude to **Saudi Electronic University** for providing the academic environment and institutional support that made this project possible. We also extend our appreciation to our doctors, supervisors, and all those in charge of the programme for their guidance, valuable feedback, and continuous encouragement throughout the development of this work. Finally, we would like to thank our families for their patience, understanding, and unwavering support during the completion of this report.

PREFACE

Saudi Electronic University (SEU) is a top public university in the Kingdom of Saudi Arabia that prepares students for challenges in their areas by fusing higher education with modern technology. The SEU places a strong emphasis on applied learning, research, and innovation, giving students the chance to create useful solutions that meet business demands.

The university encourages students to use their academic knowledge in the creation of cutting-edge technical projects in this area. Ensuring system security, performance stability, and application dependability in increasingly complex digital infrastructures is one of the software industry's most urgent concerns today. The project team created SeeStack, an AI-powered monitoring platform, to solve this problem.

Developers and businesses can use this solution to track performance metrics, automate stress testing, monitor applications in real-time, and improve infrastructure security. The project intends to decrease downtime, enhance system resilience, and speed problem resolution by utilizing artificial intelligence and modern software technologies. The result of this study serves SEU's vision to promote technical innovation in the Kingdom.

REVISION HISTORY

Name	Date	Reason For Changes	Version
Faisal Alghanim	2025/9/24	Added Background, Problem, Scope, Objectives	V0.1
Faisal Alghanim	2025/9/24	Added Abstract & Preface	V0.2
Abdualziz Alotaibi	2025/9/25	Add Literature Review.	V1.0
Abdualziz Alotaibi	2025/9/25	Add Project Structure	V1.1
Faisal Alghanim	2025/9/26	Update Abstract	V1.2
Mohammed bin eid	2025/9/28	Add Project Introduction	V1.3
Omar Alallaf	2025/9/28	Update Problem Description (1.2)	V1.4
Mohammed Taleb	2025/9/29	Update Scope And Objectives	V1.5
Salem Omran	2025/9/30	Update Background/Overview	V1.6
Omar Alallaf	2025/11/16	Expand Background/Overview and add figures	V1.7
Faisal Alghanim	2025/11/19	Update Chapter 1,2	V1.8
Abdualziz Alotaibi	2025/11/20	Add Diagram for Chapter 2	V1.9
Mohhamed Bin Eid	2025/11/20	Add Chapter 4,4.1	V2.0
Omar Alallaf	2025/11/27	Add Chapter 4.2	V2.1
Abdualziz Alotiabi	2025/11/27	Add Diagrams For Chapter 5	V2.2
Mohhamed Taleb	2025/11/27	Add References	V2.3
Faisal Alghanim	2025/11/27	Add non-functional requirement & glossary	V2.4
Abdualziz Alotaibi	2025/11/28	Add Caption For Diagrams	V2.5

Mohammed bin Eid	2025/11/29	Add chapter 6.1	V2.6
Mohhamed Taleb	2025/11/29	Add chapter 6.2 (conclusion)	V2.7
Faisal Alghanim	2025/11/29	Add use case diagram in analysis model	V2.8
Salem Omran	2025/12/2	Add Table of figure and include caption for every figure	V2.9
Omar Alallaf	2025/12/2	Add Table of tables and include caption for every table	V3.0
Abdualziz Alotaibi	2025/12/2	Fix Layout and numbering for chapter to be appear in Table of content	V3.1
Faisal Alghanin	2025/12/2	Update Methodologies	V3.2
Mohhamed Bin Eid	2025/12/2	Reduce Glossary size	V3.3
Mohhamed Taleb	2025/12/2	Update References	V3.4
Faisal Alghanem	2025/12/2	Update Figures in Chapter 1	V3.5
Faisal Alghanem	2025/4/20	Add chapter 6	V4.0
Faisal Alghanem	2025/4/21	Add Interface Images in Chapter 6	V4.1
Faisal Alghanem	2025/4/22	Update chapter 6	V4.2
Abdualziz Alotaibi	2025/4/22	Add 4 Figures in chapter 6	V4.3
Omar Alallaf	2025/4/23	Add Chapter 7	V5
Salem Omran	2025/4/23	Add Chapter 8	V5.1

Mohhamed Taleb	2025/4/24	Add Chapter 9	V5.2
Mohammed bin Eid	2025/4/24	Update chapter 9 (Future work)	V5.3
Faisal Alghanem	2025/4/24	Update chapter 6	V5.4
Faisal Alghanem	2025/4/25	Add more interface images in Chapter 6	V5.5
Abdualziz Alotaibi	2025/4/25	Add 3 figures in chapter 6	V5.6
Mohammed bin Eid	2025/4/25	add 7 function test cases more c.7	V5.7
Salem Omran	2025/4/25	update chapter 8	V5.8
Mohhamed Taleb	2025/4/25	update chapter 9	V5.9
Faisal Alghanem	2025/5/10	Fix layout + change figures font in chapter 6 + Add dedication + Improvements overall	V6.0
Abdualziz Alotaibi	2025/5/10	Update figures in chapter 6	V6.1

TABLE OF CONTENTS

TABLE OF TABLES

TABLE OF FIGURES

CHAPTER 1: INTRODUCTION

In the evolving landscape of software engineering, ensuring system reliability, high performance, and strong security has become a daily challenge for developers and technology teams. As digital applications underpin critical business operations and user expectations continue to rise, even small disruptions can lead to cascading problems, lost productivity, or reputational damage. This project, SeeStack, addresses the unique needs of modern development teams by equipping them with an integrated solution for proactive application monitoring and intelligent error management. The primary audience includes software developers, DevOps engineers, and IT teams seeking to streamline the way they detect and resolve incidents across distributed architectures. Ultimately, the goal is to empower these professionals with meaningful insights and automation so they can deliver more stable, secure, and optimized digital experiences.

In a landscape where software powers commerce, communication, and critical infrastructure, uninterrupted application performance is paramount for organizations striving to meet stakeholder demands. Yet developers often face a fractured ecosystem of logs, dashboards, and manual tests that obscure the true root causes of failures, making troubleshooting both complex and frustrating. SeeStack emerges to bridge this persistent gap by unifying error detection, performance monitoring, stress testing, and security scanning into a single, AI-driven platform. Its goal is to transform raw telemetry into clear, contextual insights that guide developers through every stage of incident resolution and system optimization. This not only simplifies complex workflows but also empowers teams to act decisively when performance or security issues arise. Furthermore, SeeStack's emphasis on user-friendly interfaces ensures that both seasoned professionals and junior engineers can benefit from its capabilities. Such unified solutions are increasingly essential for organizations aiming to maintain a competitive edge in dynamic digital environments.

Traditional monitoring tools generate streams of unfiltered data such as error codes, metric graphs, and vulnerability reports that require deep technical expertise to interpret and correlate. Developers juggling multiple interfaces must manually track and analyze disparate events, a process that drains valuable time and increases the risk of oversight or critical mistakes. SeeStack resolves these workflow bottlenecks by automatically classifying errors, predicting performance bottlenecks, and translating security findings into plain language recommendations that can be quickly understood and acted upon. The result is faster diagnosis, guided remediation, and reduced downtime, all contributing to better user experiences and more efficient resource allocation. Increased visibility across the system's architecture enables teams to recognize trends and respond proactively before issues grow severe. This capability also supports long-term planning and helps organizations identify where additional investment or redesign may be needed. By integrating intelligent analysis and actionable insights, SeeStack becomes a catalyst for operational excellence rather than just another dashboard.

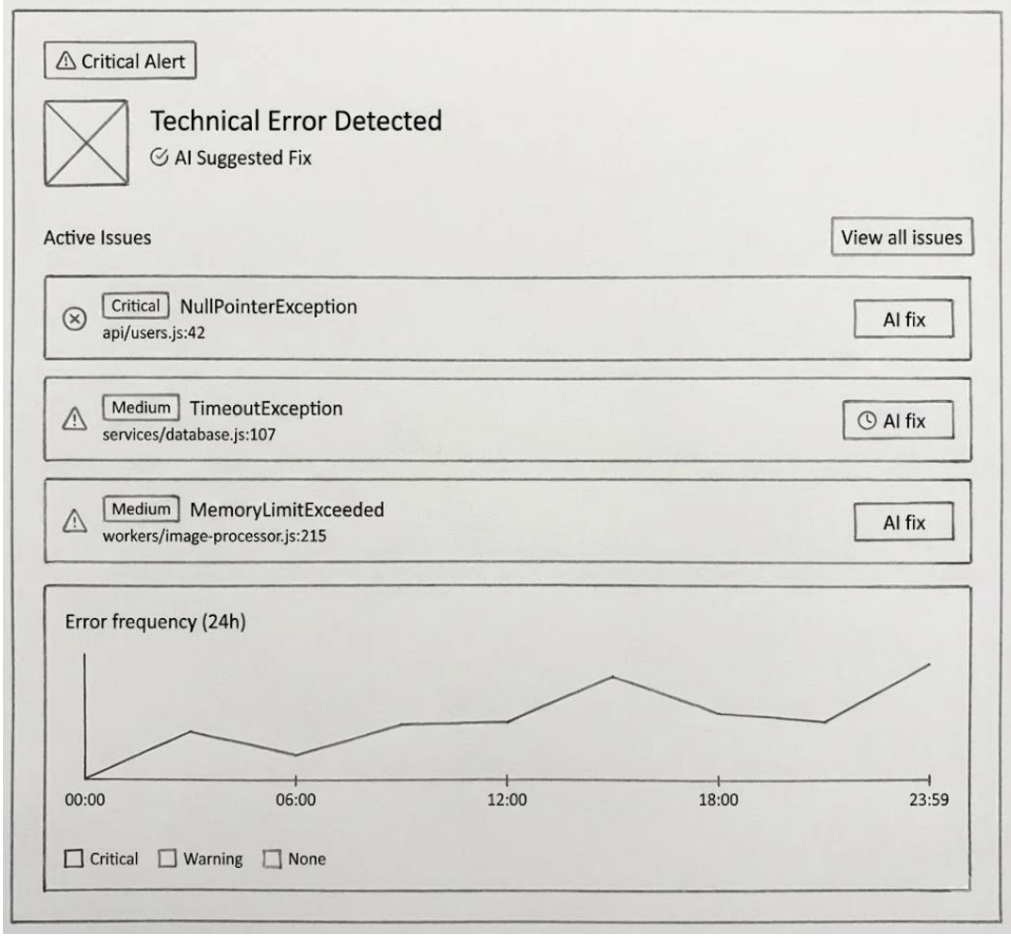


Figure 1:illustrates how SeeStack surfaces active errors such as a critical NullPointerException and offers AI suggested fixes.

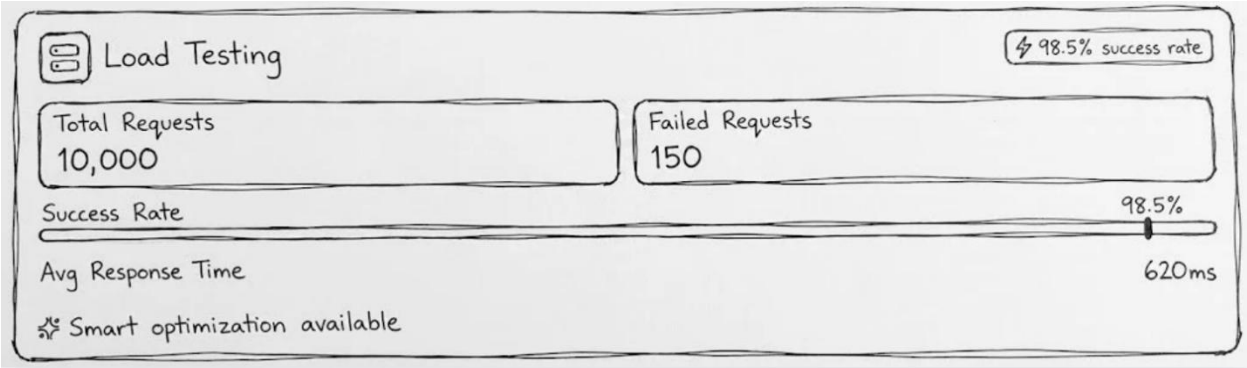


Figure 2: shows automated load testing results including total requests, success rate, and average response time under simulated user load.

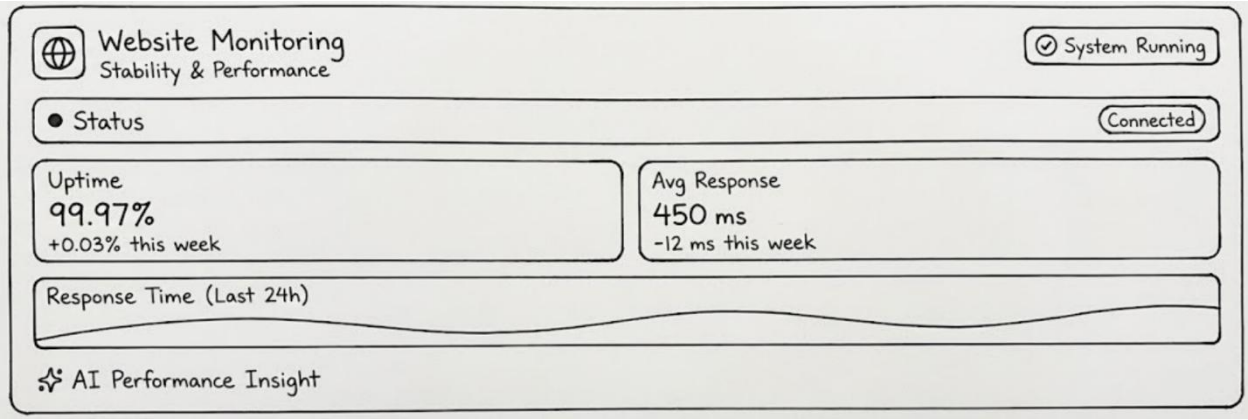


Figure 3: demonstrates continuous website monitoring with uptime and latency metrics enriched by AI powered performance insights.

1.1 Project Background/Overview:

Modern software systems operate across distributed environments that include microservices, APIs, and cloud infrastructure. Ensuring high performance, reliable error detection, and strong security remains increasingly complex. Developers often depend on a mix of disconnected monitoring tools that produce massive amounts of data without context. When failures occur, teams must sift through multiple dashboards and logs to find root causes, which wastes time and increases downtime.

SeeStack will be designed to solve these challenges through a unified, AI-powered platform that combines performance monitoring, stress testing, error detection, and security scanning in a single dashboard. This section presents the background of the project and discusses all key aspects involved, including system performance, error detection and monitoring, security, comparisons with old solutions, and the role of artificial intelligence.

1.1.1 Performance

In modern web applications, even a few seconds of delay can lead to lost customers and degraded user experience. SeeStack will focus on delivering continuous performance visibility by tracking uptime, response times, and overall system health.

The platform's performance module will collect real-time metrics from application servers, APIs, and endpoints. These metrics will reveal not only how the system behaves under normal conditions but also how it responds to sudden load spikes.

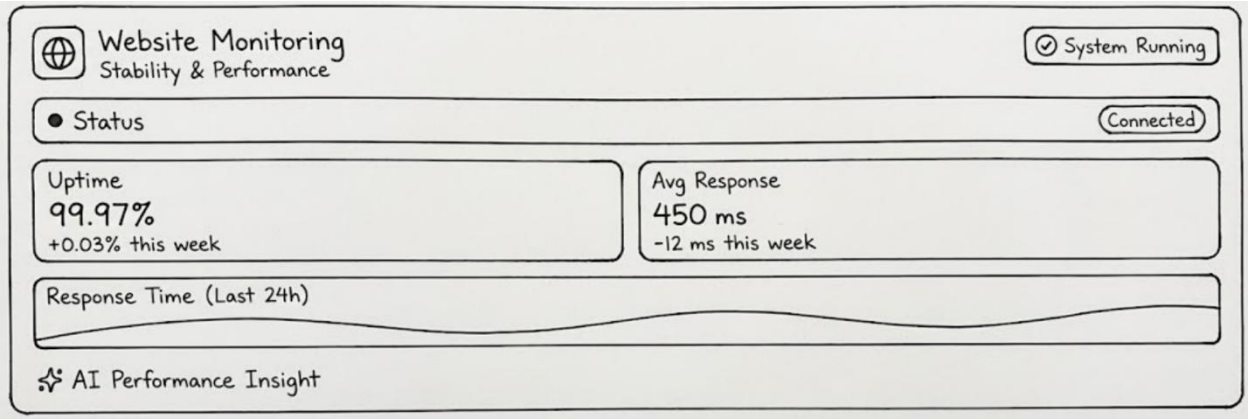


Figure 4: SeeStack performance dashboard showing overall uptime (99.97%) and average response time (≈ 450 ms).

Through this unified dashboard, developers will be able to view current and historical performance trends, identify slow endpoints, and compare results across environments. The goal will be to maintain consistent uptime and minimal latency for end users.

To ensure reliability at scale, SeeStack will also provide automated stress testing. Stress tests will simulate thousands of concurrent users or transactions, helping teams identify thresholds where performance begins to degrade. Instead of writing manual load scripts, developers will be able to trigger these tests directly from the dashboard and analyze the resulting metrics.

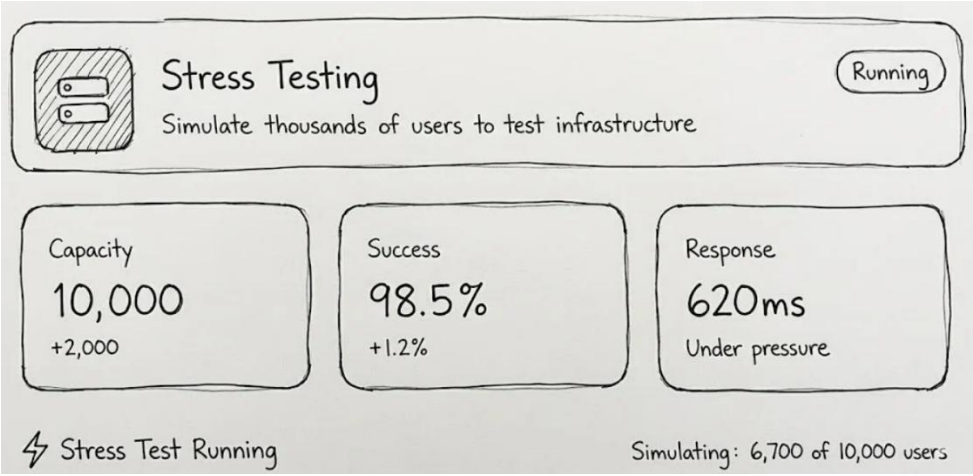


Figure 5: Stress-testing results showing simulated user capacity, success rate, and average response time under heavy load.

These results will allow early detection of performance bottlenecks, ensuring that applications remain stable before deployment to production.

1.1.2 Error detection

One of the most time-consuming parts of software maintenance is diagnosing unexpected exceptions. SeeStack's error monitoring module will continuously scan runtime logs, group similar errors, and identify their root causes. Instead of presenting cryptic stack traces, it will classify errors by type and severity and attach human-readable explanations.

For example, a developer may see a NullPointerException automatically grouped with related events and accompanied by a short AI-generated summary describing why it occurred and how to fix it.

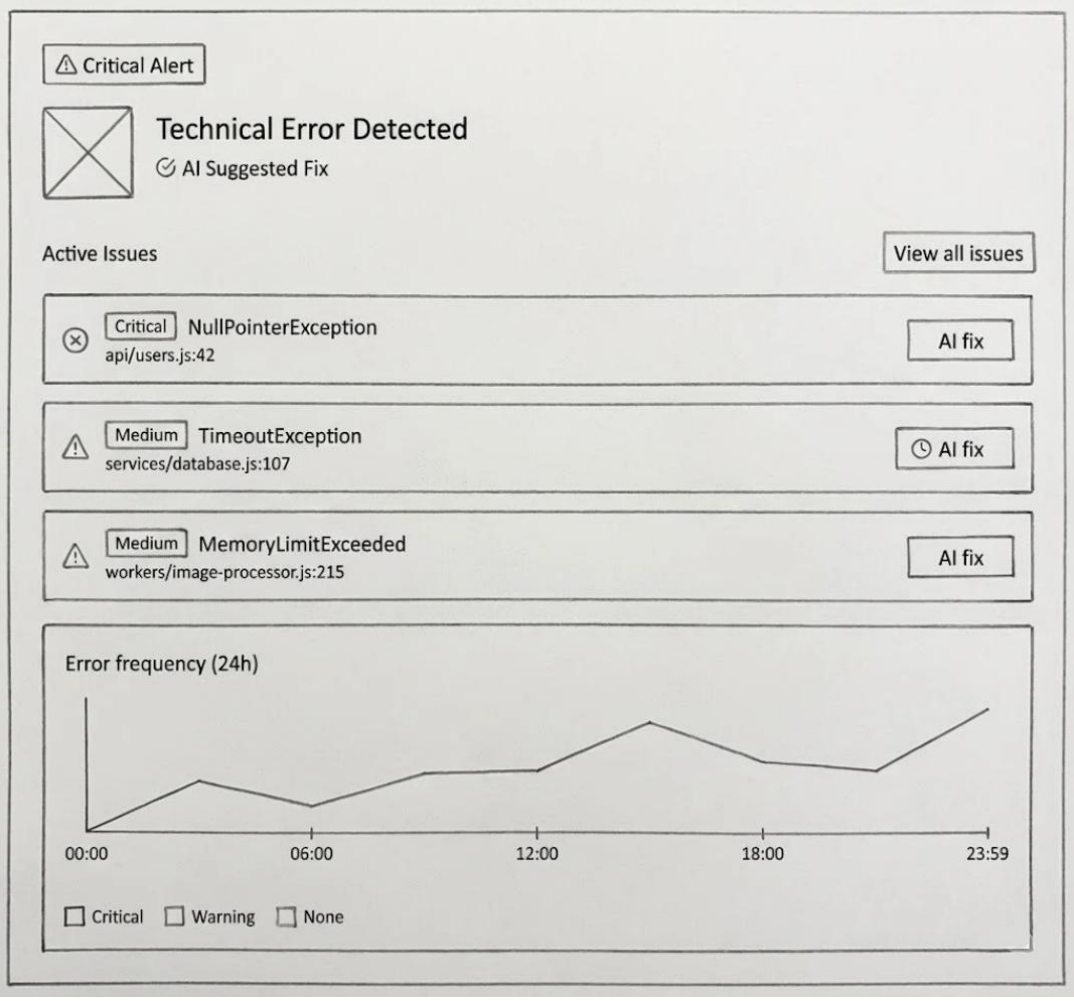


Figure 6: SeeStack dashboard automatically detecting errors and providing AI-suggested fixes for each issue.

This automation will dramatically reduce mean time to resolution (MTTR). In addition, the error frequency graph will allow teams to track recurring issues and correlate them with deployment events or traffic spikes. As a result, SeeStack will shift the debugging process from reactive problem solving to proactive maintenance.

1.1.3 Security

Security is a fundamental requirement of reliable software systems. SeeStack will integrate continuous vulnerability scanning and configuration auditing into its core monitoring pipeline. The platform will inspect code dependencies, network configurations, and runtime behaviors to detect potential weaknesses such as outdated libraries, misconfigured access permissions, or exposed endpoints.

Whenever a critical issue is discovered, the system will highlight it within the dashboard and provide AI-assisted remediation steps written in simple language.

By embedding security into daily monitoring workflows, SeeStack will help teams identify risks before exploitation, reducing the probability of data breaches and compliance violations.

1.1.4 Old Solutions

Prior to the development of integrated monitoring platforms like SeeStack, teams relied on separate tools for every domain: uptime monitoring from one vendor, error tracking from another, and manual vulnerability scanning performed occasionally. This fragmented approach created data silos and slowed down incident response. Developers had to constantly switch contexts between dashboards, correlate timestamps, and cross-check alerts manually.

SeeStack will replace this patchwork with a single cohesive environment that collects, analyzes, and visualizes all operational signals in real time. The unified architecture will reduce costs, simplify workflows, and ensure that performance, reliability, and security data share a common context.

1.1.5 Artificial intelligence (AI)

The defining feature of SeeStack will be its use of AI across all modules. Traditional monitoring systems only display raw data, leaving humans to interpret it. SeeStack will employ machine-learning models and natural-language processing to transform telemetry into actionable recommendations.

Its AI pipeline will perform three main functions:

- Pattern recognition: will identify anomalies in performance metrics or logs.
- Root-cause analysis: will group related events and infer possible causes.
- Recommendation generation: will produce plain-language suggestions for remediation or optimization.

By interpreting raw data automatically, SeeStack will reduce cognitive load on developers and enable proactive responses. Teams can focus on innovation instead of manual monitoring.

1.2 Problem Description:

Ensuring application optimal performance requires developers to navigate a maze of error logs, performance graphs, manual tests, and security scans. Without a unified view or contextual guidance, diagnosing issues becomes a slow, error-prone process that drains resources and delays feature delivery.

The following challenges illustrate the gaps in existing workflows:

- **Uncontextualized Error Messages:** Runtime exceptions such as a TimeoutException (when a request takes too long) or a NullPointerException (when code tries to use data that does not exist) often arrive without clear guidance on where or why they occurred.
- **Fragmented Tooling:** Performance, error, and security insights live in separate systems, forcing manual correlation and delaying response.

- **Cumbersome Stress Testing:** Simulating realistic traffic spikes by manually scripting and running tests is time consuming, so it often gets skipped until after problems strike.
- **Hidden Security Misconfigurations:** Vulnerabilities may remain invisible until exploited, exposing applications to data breaches or service disruptions.

SeeStack will be specifically designed to address these persistent pain points by delivering real-time data enriched with AI-powered analysis and human-readable explanations. By automatically collecting, classifying, and correlating errors and performance metrics, the platform will reduce the cognitive burden on developers and allow for faster, more targeted responses. Integrated stress testing and vulnerability detection will ensure that systems can be validated against real-world load conditions and maintain strong security postures. AI-driven insights will transform raw telemetry into actionable recommendations, speeding up remediation and empowering teams to anticipate potential problems before users are impacted. This unified, streamlined approach will free up developer resources, accelerate incident resolution, and set the foundation for continuous improvement in system reliability and user satisfaction. In doing so, SeeStack will not only fill critical gaps in traditional workflows but also support organizations' wider goals for resilient and secure digital infrastructure. With these capabilities, teams can move beyond reactive firefighting and transition to a more proactive, strategic mode of operations.

1.3 Project Scope:

SeeStack delivers an integrated solution that brings advanced monitoring, testing, and security into a single, coherent platform for modern development teams. By combining automated data collection with AI-powered interpretation, it significantly reduces cognitive overhead and accelerates decision-making. The platform's functionality is organized into four key modules, each representing a distinct AI-powered solution designed to meet real-world challenges in application management:

- **AI-powered Error Tracking:** Automatic detection, classification, grouping, and AI-assisted explanation of runtime errors through seamless SDK integration. This module uses intelligent algorithms to interpret logs and surface root causes, enabling developers to respond swiftly and accurately.
- **AI-powered Performance Monitoring:** Continuous collection of server metrics such as CPU load, memory usage, and response time, presented in real-time dashboards. Predictive analytics powered by machine learning models alert users to emerging bottlenecks before they impact user experience.
- **AI-powered Stress Testing:** Automated load simulations that emulate thousands of concurrent users or transactions, measuring system throughput and response behavior under pressure. The AI layer analyzes results, providing developers with actionable insights for infrastructure scaling and risk mitigation.
- **AI-powered Security Monitoring:** Continuous vulnerability scanning aligned with industry standards and regulations, enhanced with step-by-step mitigation guidance. AI detects configuration issues and potential threats, making security accessible to both specialists and non-experts.

As a result of this integrated approach, teams adopting SeeStack are expected to see significant improvements in system reliability, uptime, and the overall efficiency of software operations. With the platform's real-time dashboards and intelligent recommendations, organizations can move away from reactive incident response and toward proactive, data-driven management of their infrastructure. The comprehensive automation of monitoring, testing, and security processes reduces manual effort and error, allowing developers to focus on core innovation. Over time, fewer outages and faster resolution of issues should translate to better end-user satisfaction and lower operational costs. Results from production deployment will provide measurable reductions in both mean time to detect (MTTD) and mean time to resolve (MTTR) incidents. By establishing a clear, actionable workflow for system monitoring and protection, SeeStack positions teams to scale confidently and adapt quickly to changing business demands. Ultimately, the platform aims to facilitate a culture of continuous improvement and technical excellence within software-driven organizations.

1.4 Project Objectives:

Achieving proactive, intelligent monitoring requires clear goals that span data collection, analysis, and user experience. The objectives outlined below guide the development of SeeStack's core capabilities, ensuring the platform delivers real-time insights, scalable performance testing, and actionable AI recommendations.

The main objective of the project are:

1. To design and implement a **real-time monitoring pipeline** (errors, metrics, uptime) using Spring Boot, Kafka, and Redis.
2. To provide **stress testing capabilities** for simulating high user loads.
3. To integrate **AI models (OPENAI)** for analyzing errors, logs, and providing suggested fixes.
4. To develop a **React-based dashboard** for developers to visualize data, track performance, and receive alerts.
5. To incorporate **comprehensive security monitoring** (vulnerability checks, encryption compliance).
6. To containerize the solution with **Docker**, ensuring scalability and reproducibility.
7. To evaluate and validate the platform through **load testing, AI accuracy analysis, and developer usability feedback**.

1.5 Project Structure/Plan:

The project will be executed in several well-defined phases to ensure proper planning, development, testing, and deployment of the SeeStack platform.

Table 1: Structured Timeline of SeeStack Development Phases, Outlining Each Activity's Duration, Resources, and Planned Execution Window

Activity	Start Date	End Date	Duration	Resources Required
Requirement Gathering & Research	25 Sept 2025	05 Oct 2025	10 days	Project team, Supervisor
Literature Review	25 Sept 2025	10 Oct 2025	15 days	Research papers, Online resources
System Analysis (Features, Use-Cases)	06 Oct 2025	15 Oct 2025	10 days	Team, Supervisor
System Design (Diagrams, Architecture)	16 Oct 2025	25 Oct 2025	10 days	UML tools, Team
Development Phase I (Backend: Spring Boot, Kafka, Redis)	26 Oct 2025	20 Nov 2025	25 days	Developers, IDE, Server
Development Phase II (Frontend: React Dashboard, SDK)	21 Nov 2025	10 Dec 2025	20 days	Developers, React, APIs
Integration & Containerization (Docker)	11 Dec 2025	20 Dec 2025	10 days	Docker, Server
Testing Phase (Stress Tests, AI Validation)	21 Dec 2025	05 Jan 2026	15 days	Test servers, Locust, Dataset
Final Review & Documentation	06 Jan 2026	15 Jan 2026	10 days	Team, Supervisor
Project Submission & Presentation	16 Jan 2026	20 Jan 2026	5 days	Project team

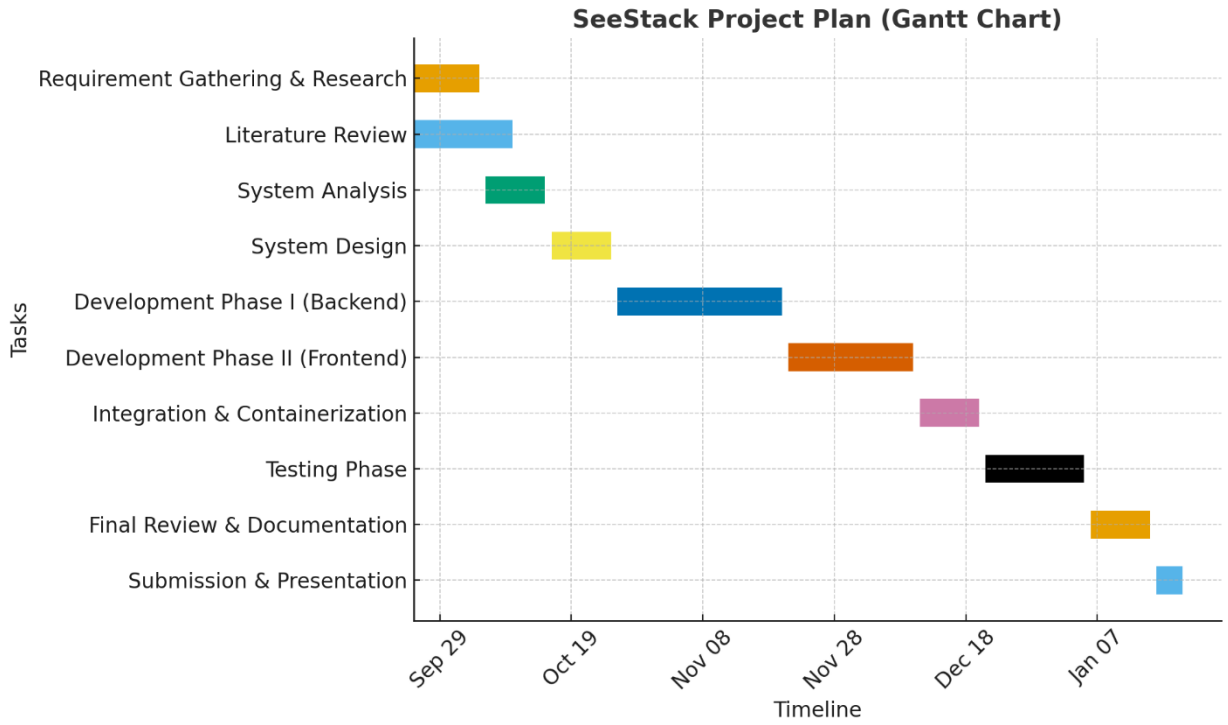


Figure 7: Gantt Chart

CHAPTER 2: LITERATURE REVIEW

2.1 Error Detection and Monitoring Platforms

The landscape of error tracking and application monitoring has evolved significantly over the past decade. Modern error detection platforms have become essential infrastructure for development teams managing increasingly complex distributed systems. **Sentry** established a foundational approach by offering real-time error tracking with intelligent error grouping, stack trace analysis, and performance tracing capabilities. However, Sentry's architecture remains primarily focused on application-layer error detection and lacks native support for infrastructure-level metrics, comprehensive log aggregation, and deep distributed tracing across microservice landscapes. This limitation means developers using Sentry must supplement it with additional tools such as Elasticsearch or Loki for log management and Jaeger or specialized APM solutions for end-to-end request tracing. While Sentry successfully reduces the burden of manual error triaging through fingerprinting and custom grouping rules, it provides minimal machine learning capabilities to generate actionable remediation steps beyond basic stack trace context.

Prometheus and Grafana have become ubiquitous in cloud-native environments, particularly for infrastructure and system-level monitoring. **Prometheus** excels as a time-series database for collecting metrics across servers, containers, and applications, while **Grafana** provides flexible visualization and dashboard creation. Despite their popularity, both tools operate under a pull-based architecture that requires careful configuration and scaling management. Prometheus lacks native support for long-term data retention and centralized log aggregation without additional components such as Thanos or Cortex. Additionally, Prometheus offers only rudimentary alerting through its AlertManager component, which requires extensive configuration and third-party integrations like PagerDuty to achieve sophisticated incident management. The visualization capabilities, while powerful, demand significant configuration effort and domain expertise to translate raw metrics into actionable insights. Teams adopting Prometheus and Grafana typically spend considerable time tuning alert thresholds and managing false positives, a workflow that does not adapt to changing application behaviors or emerging system patterns without manual intervention.

Datadog has positioned itself as a unified observability platform, integrating infrastructure monitoring, application performance monitoring, log management, distributed tracing, security monitoring, and user experience analytics into a single interface. Datadog's strength lies in its comprehensive data collection across the full technology stack and its machine learning-powered anomaly detection features such as Watchdog, which automatically identifies deviations from baseline behavior and correlates them across services. However, Datadog's event-based pricing model creates budget uncertainty for high-volume applications, as costs scale aggressively with data ingestion. Furthermore, while Datadog offers error tracking and grouping capabilities similar to Sentry, the platform does not specifically target natural language generation of debugging steps or AI-driven remediation recommendations tailored to individual error contexts. The breadth of Datadog's features, while comprehensive, also introduces a steep learning curve for new users and often requires specialized training to unlock the platform's full potential. Organizations implementing Datadog frequently report that onboarding junior developers or non-specialist teams onto the platform requires significant time investment.

2.2 Stress Testing and Performance Validation Tools

Performance validation through load and stress testing is critical for ensuring system reliability under real-world conditions. Traditional tools such as Apache JMeter, Locust, and OpenText LoadRunner enable teams to simulate concurrent users and measure system behavior under load. Locust, implemented in Python, allows developers to define user behavior scripts and has gained popularity for its ease of use in

simulating thousands of concurrent users. LoadRunner provides extensive support for over 50 protocols and technologies, making it suitable for complex, enterprise-grade applications. However, these tools remain primarily focused on metric collection and basic reporting; they do not offer AI-driven analysis of test results or automated identification of performance bottlenecks. Teams using traditional stress testing tools typically receive raw throughput, response time, and resource utilization data without contextual guidance on which bottlenecks are most critical or how architectural changes might alleviate them. Furthermore, traditional load testing tools require significant expertise to design realistic user behavior scenarios and often produce either inconclusive results or false positives if test parameters are not precisely calibrated. The absence of intelligent test analysis means that insights derived from stress testing often require manual interpretation and correlation with application architecture, delaying remediation and reducing the effectiveness of pre-deployment validation.

Modern cloud-native load testing platforms have begun incorporating AI-powered analysis capabilities, such as predictive analytics for identifying capacity issues before they materialize and automated recommendations for infrastructure scaling decisions. Despite these advances, most standalone load testing tools still operate in isolation from error tracking and security monitoring, requiring teams to manually correlate stress test results with error logs or security scan findings. This fragmented approach leads to incomplete root cause analysis and misses opportunities to identify cascading failures across service boundaries. Additionally, many stress testing solutions lack tight integration with continuous integration and continuous deployment pipelines, meaning test results are often reviewed after deployment rather than used to gate release decisions. The disconnect between stress testing outcomes and real-time monitoring of production systems further limits the ability to validate that load test scenarios accurately reflect production traffic patterns.

2.3 Security Monitoring and Vulnerability Scanning

Security scanning and vulnerability detection have traditionally been separate concerns from error monitoring and performance analysis. Tools such as Rapid7 InsightVM, Qualys, and Nessus perform periodic vulnerability scanning to identify known software vulnerabilities and configuration misconfigurations. Modern continuous security monitoring platforms like Snyk, Jit, and Syxsense have shifted the paradigm toward automated, real-time detection integrated within development workflows. These platforms typically employ credentialed and non-credentialed scanning techniques, compare system configurations against compliance benchmarks such as CIS and OWASP standards, and generate detailed remediation reports prioritized by severity. However, current vulnerability scanning tools primarily focus on static code analysis, dependency scanning, and configuration audits without correlating security findings with runtime error patterns or performance degradation that might indicate active exploitation attempts. Furthermore, most security scanning platforms operate independently from application monitoring, meaning security teams and development teams often lack visibility into whether reported vulnerabilities are actively exploited in production environments or remain latent in rarely-used code paths. The gap between vulnerability discovery and real-time security event monitoring creates a window where threats may be undetected until they manifest as errors or performance anomalies observed through application monitoring systems.

2.4 AI-Powered Insights and Natural Language Processing for Logs

Recent advances in natural language processing and machine learning have enabled new approaches to log analysis and debugging support. Research by Zhou et al. and industry implementations demonstrate that NLP-based techniques can identify root causes of system failures by analyzing unstructured log data and generating human-readable summaries. OpenAI API and similar large language models have been applied

to interpret stack traces, logs, and error contexts to produce natural language explanations of failure modes and suggest remediation steps. Unsupervised learning techniques such as autoencoders and DBSCAN clustering enable automatic anomaly detection without predefined rules, adapting to evolving system behaviors. The lack of integrated AI analysis means that developers receive fragmented suggestions from different tools. Stack trace explanations from one platform, performance insights from another, and security alerts from a third. Without cross-platform context to prioritize which issues pose the greatest risk or require the most urgent attention.

2.5 Comparison of Prior Work Against SeeStack

Table 2: Comparison of Existing Monitoring Tools with SeeStack

Aspect	Sentry	Prometheus	Datadog	ELK Stack	SeeStack
Error Detection & Grouping	✓ (Basic)	✗	✓ (Limited)	✗	✓ (AI-Semantic)
Developer-Centric Explanations	✗	✗	✗	✗	✓
Real-Time Performance Metrics	✗	✓	✓	✗	✓
Stress Testing	✗	✗	✗	✗	✓
Security Monitoring	✗	✗	✓ (Limited)	✗	✓
AI-Powered Remediation Guidance	✗	✗	✗	✗	✓
Privacy-Aware Processing	✗	✓	✗	✓	✓
Unified Dashboard	✓ (Error-only)	✗	✓	✗	✓ (Multi-domain)
Cross-Service Correlation	✗	✗ (Requires manual config)	✓ (Limited)	✗	✓
Operational Complexity	Low	High	High	Very High	Medium
Cost Model	Open-source	Open-source	Enterprise SaaS	Open-source	Cloud/On-premise

2.6 Positioning: Why SeeStack Fills Critical Gaps in Error Detection and Monitoring

Although prior work has made significant contributions to error detection and performance monitoring, these solutions exhibit critical gaps that fragment developer workflows and delay incident resolution. Sentry excels at error grouping but lacks native performance correlation, continuous security assessment, and AI-powered remediation generation. Forcing developers to maintain separate tools for each concern. Prometheus and Grafana provide robust infrastructure metrics but require sophisticated configuration expertise and offer minimal automated intelligence, demanding that teams manually interpret data and define alerting thresholds. Datadog integrates multiple observability domains but sacrifices transparency through unpredictable event-based pricing, introduces steep learning curves for teams unfamiliar with its breadth of features, and does not specifically address automated stress testing or AI-driven debugging guidance. None of these platforms seamlessly combine error tracking, performance monitoring, automated

stress testing, and proactive security scanning into a single system with unified AI-powered analysis. The fragmentation across tools forces developers to context-switch between interfaces, correlate findings manually across platforms, and lose critical contextual information that could accelerate root cause analysis and remediation.

SeeStack addresses these gaps by architecting a purpose-built, unified platform that integrates error detection, performance monitoring, automated stress testing, and security monitoring under one cohesive dashboard powered by AI. Unlike Sentry and Prometheus, SeeStack combines error tracking with continuous performance metrics, real-time stress testing, and security scanning while eliminating the need for manual configuration of alerting thresholds or complex YAML definitions by using machine learning to adapt to application behavior and prioritize meaningful alerts. Unlike Datadog, SeeStack emphasizes developer-centric usability, transparent and predictable cost scaling, and autonomous stress testing capabilities that validate system behavior before errors reach production. SeeStack further distinguishes itself by embedding advanced natural language processing and machine learning directly into its core pipeline to generate plain-language debugging recommendations, detect causal relationships between errors and performance degradation, and suggest architectural improvements informed by stress test results and security findings. By consolidating these capabilities with real-time AI-powered intelligence, SeeStack empowers teams to shift from reactive firefighting to proactive optimization, accelerates mean time to resolution through intelligently correlated insights, reduces operational overhead by removing multiple disconnected monitoring tools, and transforms monitoring from a defensive necessity into a strategic asset for continuous improvement and technical excellence.

CHAPTER 3: METHODOLOGY

3.1 Development Methodology

SeeStack will be developed using the **Waterfall Software Development Model**, a sequential and structured approach that progresses through distinct phases. Each phase must be substantially completed and validated before proceeding to the next stage. This methodology was selected due to the well-defined project scope, stable requirements established through comprehensive literature review, and the need for detailed system design before implementation commences. The Waterfall approach ensures systematic progression from requirements specification through design, implementation, testing, and deployment, providing clear milestones and comprehensive documentation at each phase.

3.2 Technology Stack Overview

SeeStack will be implemented using a modern, microservices-based architecture that combines multiple industry-standard technologies and tools. The technology stack spans backend services, frontend user interfaces, distributed data storage systems, message queuing infrastructure, and external AI services. This section describes the tools, frameworks, and platforms employed for implementing each component of the SeeStack platform.

3.3 Backend Infrastructure and Microservices

3.3.1 Spring Boot Framework

The SeeStack backend will be constructed using **Spring Boot**, a comprehensive Java framework designed for rapid development of production-grade microservices. Spring Boot provides sophisticated dependency injection capabilities through the Spring Framework, automatic configuration mechanisms that substantially reduce boilerplate code, and built-in support for embedded application servers. The framework enables developers to focus on business logic rather than infrastructure concerns.

Spring Boot is utilized throughout the backend for implementing core services including the SDK ingestion service, error analysis service, performance query service, and alert management service. The framework provides multiple key components: Spring Web for implementing RESTful HTTP endpoints that receive telemetry from client SDKs; Spring Data for database abstraction and repository patterns enabling efficient data access; Spring Kafka for integrating Apache Kafka message brokers; Spring Boot Actuator for exposing health checks and metrics endpoints compatible with Prometheus monitoring systems; and Spring Security for implementing authentication mechanisms and authorization controls.

The backend architecture exposes multiple REST API endpoints providing standardized interfaces for telemetry ingestion, error querying, metrics retrieval, stress test management, alert configuration, and security report generation. These endpoints utilize JSON for data serialization, supporting both simple and complex data structures required by the platform.

3.3.2 Apache Kafka for Event Streaming

Apache Kafka serves as the distributed message queue backbone enabling decoupling of telemetry ingestion from processing and analysis. Kafka's architecture is specifically designed for high-throughput, event-driven systems capable of handling millions of messages per second across multiple topics and consumer groups.

Within SeeStack, Kafka manages distinct topics for different event types: a primary events topic receiving all telemetry data including HTTP requests, errors, and logs; a dedicated errors topic routing error events to the AI analysis service; a logs topic aggregating application logs for search and analysis; and optional metrics topics for performance data. Each topic is configured with a substantial number of partitions enabling parallel processing across multiple consumer instances, supporting horizontal scalability of processing services.

The Kafka producer configuration within SeeStack implements at-least-once delivery semantics, configuring acknowledgement requirements to ensure replication across all brokers before confirming message acceptance. This approach guarantees no data loss in the event of broker failures. Producer batching settings accumulate multiple messages before transmission, optimizing network throughput and reducing latency overhead.

Consumer services subscribe to Kafka topics using consumer groups, enabling automatic workload distribution across multiple service instances. The AI Analysis Service consumes error events from the errors topic within its dedicated consumer group, with Kafka automatically assigning topic partitions to individual service instances. This partitioning strategy enables parallel processing of errors, where each service instance processes a distinct subset of partitions independently.

Retention policies for Kafka topics are configured to maintain events for seven days, enabling event replay for debugging purposes, re-analysis of historical data, or recovery scenarios. Log compaction is disabled for telemetry topics to preserve complete event history without applying deduplication logic.

3.3.3 Redis Caching Layer

Redis provides a high-performance in-memory data store serving as the caching layer for frequently accessed data. Redis substantially reduces database load on ClickHouse by caching computed results, enabling the dashboard to achieve sub-second response times for common queries.

Within SeeStack, Redis caches multiple categories of data: recently detected errors for quick dashboard retrieval; aggregated error statistics including occurrence counts and temporal metadata; performance metrics including uptime percentages and response time percentiles; user preferences and dashboard configurations; and results of expensive analytical queries. Redis implements multiple data structures optimized for different use cases: string data structures store serialized JSON objects; sorted sets maintain time-ordered event data enabling efficient temporal queries; hash structures store multi-field records such as error statistics; and list structures manage queues for asynchronous tasks such as stress test jobs and AI analysis requests.

Cache expiration policies implement time-to-live (TTL) mechanisms automatically removing stale data: recently detected errors expire after five minutes allowing updates as new incidents occur; performance metrics expire after five minutes reflecting current system state; user configurations expire after one hour; and results of expensive queries expire after ten minutes. Invalidation strategies automatically clear cached entries when new data arrives that would invalidate cached results. Specifically, when new errors are detected, the cached recent errors list is evicted, forcing refresh on the next dashboard access.

For production deployments, Redis is configured in cluster mode distributing cache data across multiple nodes, providing redundancy, automatic failover if individual nodes fail, and enabling cache size growth across multiple machines. This clustering approach eliminates single points of failure and improves overall system reliability.

3.3.4 OpenAI Integration for Intelligent Analysis

The **OpenAI API** with GPT-5.1 language models provides natural language processing capabilities enabling the platform to analyze error stack traces and generate human-readable root cause explanations and suggested remediation steps. This integration represents the core AI functionality differentiating SeeStack from traditional monitoring platforms.

The AI Analysis Service consumes error events from Kafka, extracting relevant context including error type, stack trace frames, affected service name, and historical similar errors. This contextual information is synthesized into structured prompts submitted to the OpenAI API. OpenAI processes these prompts and returns natural language responses structured as JSON containing root cause analysis, suggested fixes, relevant code examples, and severity assessments.

API integration implements robust error handling and reliability mechanisms: asynchronous API calls utilize timeout settings of thirty seconds to prevent service hangs; retry logic implements exponential backoff with attempts after two seconds, five seconds, and ten seconds if transient failures occur; circuit breaker patterns detect repeated failures and prevent cascading failures. Rate limiting mechanisms ensure the platform remains within OpenAI API quotas, queuing analysis requests during peak periods and implementing batch processing to analyze multiple related errors together, reducing the number of API calls required.

API key management stores OpenAI credentials as environment variables with access controlled through Spring configuration properties, preventing credential exposure in source code repositories. Responses from OpenAI are parsed into structured data structures enabling programmatic access to root cause information, suggested fixes, and confidence scores.

3.4 Frontend User Interface

3.4.1 React Framework and TypeScript

The SeeStack frontend dashboard will be developed using **React**, a JavaScript library for building interactive, component-based user interfaces. **TypeScript** provides static type checking, substantially reducing runtime errors and improving code maintainability compared to untyped JavaScript. React's component-based architecture enables code reuse and modularity, allowing development of complex dashboards through composition of simple, well-defined components.

The React application utilizes component hierarchies representing different aspects of the monitoring platform: an error list component displaying errors in tabular format with filtering and sorting capabilities; an error detail component showing comprehensive information including stack traces, affected users, temporal trends, and AI-generated insights; a performance dashboard component visualizing real-time metrics through charts and gauges; a stress testing component providing configuration interfaces and real-time progress visualization; and alert configuration components enabling users to create and manage alert rules.

State management within the React application employs Redux or Context API to maintain global application state including user authentication status, selected project context, active time range filters, and user preferences. This centralized state management enables consistent data flow between components and reduces prop drilling through component hierarchies.

Client-side routing via React Router enables seamless navigation between different dashboard pages representing monitoring domains: errors page for error tracking and analysis; performance page for real-time metrics visualization; stress testing page for load simulation configuration and results; security page for vulnerability scanning and compliance reporting. Routing is entirely client-side, eliminating full-page reloads and providing smooth user experience.

3.4.2 UI Component Libraries and Visualization

The frontend leverages **Material-UI**, a comprehensive component library implementing Material Design specifications, providing pre-built, professionally styled UI components including tables, cards, dialogs, buttons, text fields, and navigation drawers. Material Design guidelines ensure visual consistency and usability across the dashboard.

Recharts provides charting and visualization capabilities for representing time-series performance metrics and trend analysis. Charts implemented include line charts for temporal trends in response time and error rates, stacked area charts for aggregated metrics showing request success and failure rates, bar charts for comparing metrics across dimensions such as endpoints or services, and geographic heatmaps displaying latency distributions by region.

Axios serves as the HTTP client library enabling the React frontend to make REST API calls to backend services. Axios provides interceptor capabilities allowing centralized handling of authentication tokens, automatic inclusion of authorization headers, and unified error handling across all API calls.

React-Query abstracts data fetching logic, handling API call caching, automatic refetching strategies, loading and error states, and request deduplication. This library substantially reduces boilerplate code for managing asynchronous data flows.

3.4.3 Real-Time Communication via WebSocket

WebSocket connections establish persistent, bidirectional communication channels between frontend and backend, enabling real-time delivery of dashboard updates. When alerts trigger, new errors occur, or metrics change, the backend publishes event messages to connected WebSocket clients, enabling instantaneous dashboard updates without polling.

WebSocket implementations establish connections upon dashboard load using authentication tokens to ensure secure connections to authorized users. The client-side implements automatic reconnection logic with exponential backoff strategies, attempting reconnection after two seconds, then five seconds, then ten seconds if initial connections fail. This resilience ensures the dashboard continues functioning even during temporary network disruptions.

Message types communicated over WebSocket include error detection events triggering immediate error list updates, metric update events providing fresh performance data, alert trigger events notifying users of incidents, and stress test progress updates showing real-time test execution status. Message format utilizes JSON serialization enabling flexible payload structures supporting different message types.

3.4.4 Dashboard Functionality

The **Error Monitoring Interface** presents errors detected across the monitored application estate in tabular format, grouping errors by fingerprint (cryptographic hash of normalized stack trace) to correlate related errors. Displayed information includes error type or exception class, severity classification, occurrence count indicating frequency, number of affected users, timestamp of most recent occurrence, and AI-generated suggestion summarizing root cause and recommended fix. Filtering capabilities enable users to restrict displayed errors by severity level, error type classification, or time range selection. Clicking individual errors opens detailed views presenting complete stack traces with syntax highlighting, affected user information, temporal trend analysis showing error frequency over time, and the AI-generated root cause analysis with suggested remediation steps.

The **Performance Dashboard** provides real-time visibility into application health metrics including uptime percentage (target 99.9%), average response time across all requests, error rate as percentage of total requests, and request throughput measured in requests per second. Time-series visualizations display trends over configurable time windows including last hour, last day, or last seven days. Geographic heatmaps show response time distribution across geographic regions, enabling identification of regional performance issues. A table of slowest endpoints identifies performance bottlenecks requiring optimization.

The **Stress Testing Interface** enables users to configure and execute load tests simulating realistic application usage patterns. Configuration parameters include target URL, number of virtual users to simulate, test duration, ramp-up time for gradually increasing load, and optional custom request payloads. Upon test initiation, real-time progress visualization displays current number of concurrent users, current throughput in requests per second, success rate percentage, and current response time percentiles. Upon test completion, a results summary provides recommendations for infrastructure scaling, identifies error patterns under load, and suggests optimization opportunities.

The **Alert Configuration Interface** enables creation of alert rules triggering notifications when specified conditions occur. Rule configuration specifies condition type (error threshold, performance threshold, security finding), condition parameters (threshold value, time window), severity level, and notification channels including email, Slack, webhooks, or other configured services. A test notification mechanism verifies channel configuration. Alert management displays active alert rules, historical alert trigger counts, and most recent trigger timestamps, enabling users to track alert effectiveness.

3.5 Data Storage and Persistence

3.5.1 ClickHouse Columnar Database

ClickHouse is an open-source columnar database system optimized for analytical queries on massive datasets. Unlike traditional row-oriented databases storing data as complete records, ClickHouse organizes data by column, enabling compression ratios of ten to forty times, substantially reducing storage costs while maintaining query performance.

ClickHouse's columnar architecture and vectorized query execution leverage Single Instruction Multiple Data (SIMD) CPU capabilities to process data in batches rather than row-by-row, providing massive performance improvements for analytical queries. These architectural characteristics enable SeeStack to

ingest millions of telemetry events per second while supporting complex analytical queries completing in sub-second timeframes.

ClickHouse's MergeTree table engine provides time-series specific optimizations ideal for telemetry storage. Tables are organized with time-based partitioning by date, enabling efficient time-range queries and automated data lifecycle management. Daily partitions can be dropped when Time-To-Live (TTL) policies expire without scanning entire tables.

.

3.5.2 PostgreSQL Relational Database

PostgreSQL stores relational data requiring ACID (Atomicity, Consistency, Isolation, Durability) compliance and complex queries. Within SeeStack, PostgreSQL manages user accounts with email, authentication method, and session metadata; project definitions including project name and metadata; API key storage using hashed values ensuring plaintext keys are never persisted; alert rule definitions including condition specifications and notification channel configurations; and alert trigger history for auditing and trending analysis.

The database schema implements normalized design reducing data redundancy while maintaining referential integrity through foreign key constraints. User sessions table stores JWT tokens, token expiration timestamps, client IP addresses, and user agent strings enabling security auditing and session management.

Authentication credentials are stored using cryptographic hashing rather than plaintext. API keys are hashed using SHA256, with only the hash value persisted; incoming API keys are hashed and compared to stored values without ever comparing plaintext keys. User passwords, in cases where password authentication is supported, are hashed using bcrypt with salt, providing security against password compromise if the database is breached.

3.6 Software Development Kit (SDK) Architecture

3.6.1 Multi-Language SDK Support

SeeStack provides Software Development Kits (SDKs) for multiple programming languages and platforms enabling telemetry collection from diverse application environments. Supported language ecosystems include JavaScript/Node.js for backend services and browser applications, Java for Spring Boot applications and enterprise systems, Python for data science and backend applications, and mobile platforms including iOS and Android applications.

Each SDK implements consistent functionality across language implementations: automatic exception capture integrating with language-specific exception handling mechanisms; HTTP request and response interception capturing timing and payload metadata; integration with logging frameworks for capturing application logs; automatic context attachment including user identifiers, request identifiers, and environment information; local buffering reducing transmission overhead; retry logic with exponential backoff implementing resilience against network failures; and privacy controls preventing capture of sensitive information such as passwords and API keys.

3.6.2 SDK Initialization and Configuration

SDKs are initialized with configuration parameters including the API key enabling identification of the sending application, project identifier specifying the target project within SeeStack, environment specification (development, staging, production) enabling environment-based filtering, and release version information for correlating errors with specific software versions.

Upon initialization, SDKs establish event collection mechanisms by registering handlers with language-specific exception handling systems, deploying HTTP middleware or interceptors, and configuring logging integration. The SDK maintains an in-memory event buffer storing events awaiting transmission, implementing a fixed-size circular buffer that prevents memory exhaustion by automatically dropping oldest events if the buffer fills.

3.6.3 Error Fingerprinting and Event Batching

The SDK implementation includes error fingerprinting logic normalizing exception stack traces into reproducible hashes enabling grouping of related errors. Fingerprinting normalizes stack trace data by extracting key frames, removing variable values and memory addresses, and creating hash values from normalized data. This process enables SeeStack to recognize that multiple distinct error instances represent the same underlying issue despite occurring in different execution contexts.

Event transmission occurs through batching: the SDK accumulates events in local buffers until either one thousand events accumulate or a configurable time interval (typically ten seconds) elapses, whichever occurs first. Batching reduces transmission overhead by amortizing HTTP request overhead across multiple events. Upon transmission, the SDK constructs HTTP POST requests containing JSON payloads with arrays of events, includes the API key in the X-API-Key header for authentication, and implements retry logic with exponential backoff if transmission fails due to network issues.

3.6.4 Privacy and Data Sensitivity Controls

SDKs implement mechanisms preventing capture of sensitive information that should never be transmitted: automatic redaction of common sensitive fields such as password, credit_card, and authorization values; configurable redaction patterns enabling applications to specify additional fields requiring redaction; automatic HTTPS enforcement preventing transmission over unencrypted connections; and local-only processing ensuring no data is transmitted to external servers unless explicitly configured.

The SDK validates captured data before transmission, sanitizing field values to remove or obscure sensitive patterns, implementing content security policies, and respecting user privacy preferences such as Do Not Track signals.

3.7 Authentication and Authorization

3.7.1 Passwordless Authentication System

SeeStack implements a passwordless authentication approach eliminating reliance on passwords prone to reuse, weak security practices, and phishing attacks. The authentication flow begins with users entering their email address in the login interface. The authentication service generates either a six-digit one-time

password (OTP) or a time-limited magic link valid for fifteen minutes. The OTP or magic link is transmitted to the user's registered email address through SMTP.

Users retrieve the OTP from their email inbox and enter it into the authentication interface, or alternatively click the magic link in the email redirecting to the authentication interface. The authentication service validates the OTP against the generated value or validates the magic link against stored tokens, ensuring the request originated from someone with access to the registered email address. Upon successful validation, the authentication service generates a JWT token encoding user identity, email address, authorized project identifiers, and permission scopes.

JSON Web Tokens (JWT) encode user identity and authorization information in digitally signed tokens. Tokens include standard claims including subject identifier (user_id), issue time (iat), and expiration time (exp) set to ten hours after token generation. Custom claims include email address, list of authorized project identifiers, and permission scopes such as read:errors, write:alerts, and read:security_reports. The token is digitally signed using HMAC-SHA256 with a server-side secret key, preventing tampering without detection.

Subsequent API requests include the JWT token in the Authorization header using Bearer scheme: "Authorization: Bearer eyJhbGc...". The Spring Security authentication filter intercepts requests, extracts the JWT token, validates the signature using the server-side secret key, verifies the token has not expired, and extracts the user identity and permissions. If validation fails, the request is rejected with HTTP 401 Unauthorized status.

3.7.2 API Key Authentication for Programmatic Access

While passwordless authentication serves human users accessing the dashboard, SDKs and automated systems require different authentication mechanisms. API Key authentication enables SDKs to authenticate with the SeeStack platform without human user credentials.

Users generate API keys through the dashboard, receiving long random strings (256-bit cryptographic random values) representing the API key. The platform stores only cryptographic hashes of API keys (SHA256), never storing or displaying plaintext keys after initial generation. This approach prevents total compromise if the database is breached; attackers cannot use the compromised hashes to directly authenticate, since only the plaintext key can be hashed to match the stored value.

Programmatic clients include the API key in requests using the X-API-Key HTTP header. The platform hashes the received key and compares it to stored hashes, enabling authentication without ever transmitting or storing plaintext keys. API keys are scoped to specific projects; a single API key used across all SDKs integrating with a project enables identification of the sending application but provides no access to other projects.

API key management enables revocation without password reset processes. Users immediately revoke compromised API keys through the dashboard, invalidating all subsequent authentication attempts using the revoked key. Audit logs record API key generation and revocation events enabling security investigations.

CHAPTER 4: SYSTEM ANALYSIS

4.1 Product Features:

Table 3: Core Features and System Characteristics

Purpose & Value Proposition	SeeStack offers monitoring, stress testing, technical error detection, and security monitoring for websites/applications. Real-time data, alerts, AI-assisted suggestions.
Target Users	Developers, DevOps engineers, site reliability teams, owners of web apps or services who care about uptime, performance, error detection, and resilience. Also likely small-to-medium businesses and startups (given free & paid plans).
System Components	• SDKs for error/log/capture/traces integration into apps (• Monitoring agents/ping-checks for websites/services (uptime, availability) • Stress testing / load testing tools • Dashboard / analytics components • Alerting / notification subsystem • AI module / suggestions / root cause analysis • Possibly security / port scanning, etc.
Data Flows	Apps emit error/log/trace → sent to backend → stored, grouped, classified → analyzed (severity, frequency etc) → surfaced in dashboard + alerts. Monitoring agents ping/measure sites → metrics collected. Stress tests generate traffic → results collected. AI module ingests logs/errors/traces to suggest root causes.
Non-Functional Requirements (inferred)	Reliability / high availability, scalability (to handle large amounts of error/trace/log data, traffic simulations), performance (dashboard responsiveness, minimal latency), security (data in transit & at rest, access control), accuracy (error detection, grouping similar errors), real-time or near real-time response for alerts. Also usability, ease of integration (SDK plug & play).
Constraints	Probably need to support many different platforms / languages (SDKs for web, mobile, backend). Handling high data volumes can be expensive. For stress testing, may be limitations on how much traffic can be generated without cost or infrastructure issues. Regulatory / compliance / security obligations. Pricing tiers limit usage.

Table 4: Key Features, Their Functions, and the Value They Deliver

feature	Description / What It Should Do	Value / Why It's Important
Multi-SDK & Platform Support	Provide SDKs for many languages/platforms (Node.js, Python, Java, mobile platforms iOS/Android, etc.) with consistent capabilities (errors,	Users have varied tech stacks; broad support reduces barriers to adoption. Ensures more apps can integrate quickly.

	logs, traces). Also support serverless environments, microservices.	
Real-Time Error Detection & Root-Cause Analysis	Automatically capture exceptions, logs, traces; group similar errors; prioritize by severity or impact; provide root cause insights (e.g. stack-trace, related events). Possibly use AI/ML for anomaly detection.	Helps teams find and fix critical issues before they affect users. Reduces debugging time.
Performance Monitoring & Uptime Tracking	Monitor website/app availability (uptime), response times, resource usage (CPU, memory), latency, etc. Provide dashboards, historical trends.	Essential for reliability; helps detect degradation before outage. Users care about performance and user experience.
Stress / Load Testing Tools	Ability to simulate many concurrent users or requests over configurable durations. Visualizations of results (success/fail rate, response times, bottlenecks). Ability to test from different geographic locations.	Lets dev/ops teams test capacity and avoid being surprised by traffic spikes. Geographic testing helps uncover latency issues.
Alerts / Notifications	Configurable alerts: email, SMS, Slack, webhook, etc. Thresholds configurable (response time, error count, uptime drop). Escalations (e.g. if issue persists).	Users need to know when things go wrong; not all errors are equal. Fast notification reduces MTTR.
Dashboard & Analytics / Reporting	A central dashboard that shows key metrics; historical trends; comparison over time; segmentation (by endpoint, module, geography); exporting reports; custom dashboards.	Helps users understand patterns, monitor SLAs, present metrics to stakeholders.
AI / Smart Recommendations	Suggestions on how to fix errors; anomalies detection; predictive warnings (e.g. errors trending upward); optimization tips.	Adds differentiation; reduces workload for users; increases value beyond simple monitoring.
Integration	Integrate with other tools: e.g. version control (GitHub, GitLab), CI/CD pipelines, alerting tools (Slack, PagerDuty), issue trackers (Jira, Trello), chat ops.	Users often have existing toolchains; integrations help embed SeeStack into their workflow.
Security Monitoring	Monitor for vulnerabilities, open ports, possible intrusions; scan code or dependencies for known security issues; check SSL/TLS configs; perhaps check for data leaks; integrate with security logs.	Security is increasingly a priority. Users trust platform more if security is robust. Adds value by bundling performance and security.
Team & Access Management	Allow multiple users / roles / permissions per project; audit logs;	Most real apps are worked on by teams; access control

	share dashboards; possibly organisation/team-level billing.	and collaboration are necessary.
Scalability & Multi-Tenancy	Ability to handle many customers, large volumes of data; data partitioning; efficient storage; handle burst loads (stress tests); possibly geographical distribution.	To ensure system performs well under load; keep costs manageable; deliver service globally.
Customizable Monitoring / Thresholds / Rules	Let users define custom metrics or events; set custom thresholds; define rules (e.g. trigger alert when metric A > X and B < Y); ability to ignore certain noise (filter).	Not all use cases are the same; flexibility is key. Helps reduce false positives.
Data Retention & Privacy / Compliance	Define how long logs/errors are stored; allow GDPR/HIPAA/other regulatory compliance; encryption; user data privacy; ability to purge data; compliance certifications.	Many customers care about legal/regulatory risk. Must ensure data is handled securely.
Localization / Regionalization	Support for multiple time zones; local language; compliance with local laws; possibly provide monitoring nodes in various geographies.	Improves usability globally; reduces latency; legal compliance.
Cost / Pricing Flexibility / Usage Limits	Clearly defined tiers; over-usage/trial limits; ability for enterprise/custom plans; usage-based billing perhaps; discounts; free tier.	Allows accommodating users from small to large. Helps adoption.
Reliability / High Availability	System must itself have monitoring, redundancy, backup; minimal downtime; graceful degradation; for example for stress-test service.	If monitoring service is down, customers cannot trust data. Critical.
User Experience / Onboarding	Easy SDK setup; clear docs; guided tour; sample dashboards; templates; onboarding flow; quick wins.	Reduces friction; faster time-to-value; improves conversion from trial to paid.

4.2 Functional Requirements:

4.2.1 Use-Case 1: Monitor Application Errors

Table 5: End-to-End Flow for Error Monitoring and AI-Assisted Analysis

Identifier	UC-1

Purpose	Enable developers to monitor and track runtime errors in their applications through automatic error detection and AI-powered analysis.
Priority	High
Pre-conditions	Developer has integrated SeeStack SDK into their application - Application is running and generating telemetry data - Developer has valid API key and authentication credentials
Post-conditions	Error events are captured and stored in the system - Developer receives notifications for critical errors - AI analysis generates root cause and fix suggestions

Typical Course of Action

S#	Actor Action	System Response
1	Developer's application throws the a NullPointerException during user checkout process.	SDK automatically captures the exception with full stack trace , timestamp, and context data.
2		SDK batches the error event and transmits it to SeeStack API via HTTPS .
3		System validates API key and project_uuid for authentication .
4		System enriches error with geographic data (IP location , country, city, VPN detection).
5		System generates error fingerprint using MD5 hash of normalized stack trace for grouping similar errors .
6		System stores error in ClickHouse observability_error_events table.
7		System publishes error event to Kafka for asynchronous processing.
8		AI Analysis Service consumes event and sends stack trace to OpenAI GPT-4 for analysis.
9		System updates error record with AI-generated root cause, suggested fix, and related error links.

10	Developer opens SeeStack dashboard and navigates to Errors tab.	System queries ClickHouse and returns error list with AI suggestions, grouped by fingerprint .
11	Developer clicks on error group to view details.	System displays detailed error view with stack trace, occurrence frequency graph, affected users , and AI recommendations.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	SDK Network Failure	If network request fails, SDK queues error locally and retries with exponential backoff (5s, 10s, 30s).
2	Invalid API Key	System returns 401 Unauthorized response. SDK logs authentication failure and stops sending telemetry until valid credentials are provided.
3	Duplicate Error	If error fingerprint already exists in last 5 minutes, system increments occurrence counter instead of creating new record.

4.2.2 Use-Case 2: Perform Stress Test

Table 6: Workflow for Executing High-Load Stress Tests and Generating Performance Insights

Identifier	UC-2	
Purpose	Allow DevOps engineers to simulate high user load on their applications to identify performance bottlenecks and capacity limits before production deployment.	
Priority	High	
Pre-conditions	DevOps engineer has SeeStack account with stress testing feature enabled - Target application endpoint is accessible via HTTP/HTTPS - Engineer has defined test scenarios (users, duration, request patterns)	
Post-conditions	Stress test results are stored in the system - Performance metrics are visualized in dashboard - System generates recommendations for scaling	
Typical Course of Action		
S#	**Actor Action**	**System Response**

1	DevOps engineer navigates to Stress Testing module in dashboard.	System displays stress test configuration form.
2	Engineer configures test parameters: - Target URL: https://api.example.com/payment - Virtual users: 5000 - Duration: 10 minutes - Ramp-up time: 2 minutes - Request type: POST with payload	
3	Engineer clicks "Start Test" button.	System validates target URL accessibility and parameters.
4		System spawns distributed test workers (Docker containers) to generate concurrent requests.
5		System starts ramping up virtual users from 0 to 5000 over 2 minutes.
6		System sends HTTP POST requests to target URL and measures: - Response time (P50, P95, P99) - Success rate - Error rate - Throughput (requests/second)
7		System captures failed requests with error codes and response bodies.
8		System monitors target server resource usage (CPU, memory) if monitoring agent is installed.
9		System stores metrics in time-series format in ClickHouse.
10	Engineer monitors test progress via live dashboard.	System updates dashboard in real-time with current metrics and graphs.
11		System detects performance degradation: - P95 latency > 500ms at 3500 users - Error rate increases to 5%
12		Test completes after 10 minutes.
13		AI Analysis Service analyzes results and generates recommendations: - "Scale to 3 application servers"

		- "Optimize database query at /payment" - "Enable connection pooling"
14	Engineer reviews test summary report.	System displays comprehensive report with graphs, bottleneck analysis, and AI recommendations.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Target URL Unreachable	- System returns error: "Target URL is not accessible. Please verify the endpoint and network connectivity." - Test does not start..
2	Insufficient Resources	If system cannot allocate required workers (high cluster load), system queues test and notifies engineer: "Test queued. Expected start time: 5 minutes."
3	Test Aborted by User	- Engineer clicks "Stop Test" button. - System gracefully shuts down workers and saves partial results. - System marks test status as "Aborted" with completion percentage.

4.2.3 Use-Case 3: Configure Alert Notifications

Table 7: Configuration and Execution Flow for Automated Alert Notifications

Identifier	UC-3
Purpose	Allow administrators to configure automated alerts for critical errors, performance degradation, and security issues with customizable notification channels.
Priority	Medium
Pre-conditions	- Administrator has logged into SeeStack dashboard - Administrator has appropriate permissions for alert configuration - At least one notification channel is available (email, Slack, webhook)
Post-conditions	- Alert rule is saved and activated in the system - System monitors conditions and triggers notifications - Alert history is logged for audit purposes

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Administrator navigates to Settings → Alerts page.	System displays list of existing alert rules and "Create Alert" button.
2	Administrator clicks "Create Alert" button.	System displays alert configuration form with fields: - Alert name - Alert type (Error, Performance, Security) - Condition/Trigger - Severity (Critical, High, Medium, Low) - Notification channels - Throttling rules
3	Administrator configures alert: - Name: "Critical Payment Errors" - Type: Error - Condition: error_type = "PaymentException" AND severity = "fatal" AND count > 10 in 5 minutes - Severity: Critical - Channels: Email (team@company.com), Slack (#incidents channel) - Throttle: Max 1 notification per 10 minutes	
4	Administrator clicks "Test Alert" to verify configuration.	System simulates alert trigger and sends test notification to configured channels.
5		System displays: "Test notification sent successfully to 2 channels."
6	Administrator clicks "Save and Activate" button.	System validates alert configuration and stores rule in database.
7		System starts monitoring alert condition in real-time using Kafka event stream.
8		System displays success message: "Alert rule activated successfully."
--- Time passes: Alert condition is triggered ---		

9		System detects 12 PaymentException errors occurred within 5 minutes.
10		System evaluates throttling rules (last notification was 15 minutes ago, OK to send).
11		System generates alert notification with details: - Alert name - Trigger condition - Affected users - Error frequency graph - Link to dashboard
12		System sends notification via: - Email to team@company.com - Slack message to #incidents
13		System logs alert in alert_history table with timestamp and recipients.
14	Administrator receives notification and opens email.	Administrator clicks dashboard link in email.
15		System displays pre-filtered error dashboard showing only PaymentException errors from alert trigger window.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Invalid Alert Condition	- If condition syntax is invalid, system returns error: "Invalid condition syntax. Expected format: field operator value." - Alert is not saved.
2	Notification Channel Failure	- If email service is unavailable, system logs failure and retries after 5 minutes. - If Slack webhook fails, system falls back to email only. - System sends internal notification to administrators about channel failure.

3	Alert Throttled	- If last notification was sent within throttle window (10 min), system skips notification but increments suppressed_count counter. - Next notification includes: "12 additional occurrences suppressed due to throttling."

4.2.4 Use-Case 4: Analyze Error with AI

Table 8: AI-Driven Use-Case for Error Interpretation, Root-Cause Identification, and Automated Fix Suggestions

Identifier		UC-4
Purpose		Provide developers with AI-powered root cause analysis and suggested fixes for application errors using natural language processing and machine learning models.
Priority		High
Pre-conditions		- Error event exists in the system with stack trace - OpenAI API integration is configured and operational - Developer has access to error details
Post-conditions		- AI analysis is generated and stored with error record - Developer receives human-readable explanation - Suggested fix is displayed with code examples
Typical Course of Action		
S#	**Actor Action**	**System Response**
--- Background: Error captured and stored ---		
1		System publishes error event to Kafka "ai-analysis" topic.
2		AI Analysis Service consumes event from Kafka.
3		System extracts key information: - Error type: NullPointerException - Error message: "Cannot invoke method getName() on null object" - Stack trace (top 10 frames) - Context: User checkout process - Affected code: UserService.java:145

4		System constructs prompt for OpenAI: "Analyze the following Java error and provide root cause analysis and fix: [stack trace + context]"
5		System sends request to OpenAI GPT-4 API with error details.

6		OpenAI analyzes error and returns structured response:
		{
		"root_cause": "User object is null when accessing getName(). Likely database query returned null for non-existent user ID.", "suggested_fix": "Add null check before calling getName()", "code_example":
		"if (user != null) { String name = user.getName(); }",
		"severity_assessment": "High – affects checkout flow", "related_patterns": ["NullPointer after DB query"]
		}

7		System stores AI analysis in observability_error_events table: - ai_suggestion field - ai_root_cause field - ai_confidence_score
8		System groups similar errors by pattern matching and links them: - Finds 3 other NullPointerException errors in UserService - Updates ai_related_errors array
9	Developer opens error details in dashboard.	System retrieves error with AI analysis and displays:
10		Dashboard shows structured sections: ┌─ AI Analysis ──────────┐ │ Root Cause: │ │ User object is null when... │

		┌──────────────────────────┐ │ Suggested Fix: │ Add null check before... │ │ Code Example: │ if (user != null) { ... } │ │ Related Errors: 3 │ Confidence: 95% └──────────────────────────┘
11	Developer clicks "View Related Errors" link.	System displays list of 3 related NullPointerException errors with similar patterns in UserService.
12	Developer clicks "Mark as Resolved" after implementing fix	System updates error status to "resolved" and records: - Resolution timestamp - Resolver user ID - Time to resolution (MTTR)
13		System creates knowledge base entry linking error pattern to solution for future reference.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	OpenAI API Failure	- If OpenAI API returns error (rate limit, timeout, service outage), system retries 3 times with exponential backoff. - If all retries fail, system logs error and sets ai_suggestion to "AI analysis temporarily unavailable. Please try again later." - System queues error for batch analysis when API recovers.
2	Low Confidence Analysis	- If OpenAI confidence score < 70%, system adds disclaimer: "AI analysis has low confidence. Manual review recommended." - System surfaces error to human expert for validation.

3	Insufficient Error Context	- If stack trace is missing or truncated, system uses basic pattern matching to suggest generic fixes. - System displays: "Limited analysis due to insufficient error data."

4.2.5 Use-Case 5: Generate Security Report

Table 9: Use-Case for Creating Comprehensive Security Reports and Compliance Assessments

Identifier	UC-5	
Purpose	Enable security officers to generate comprehensive security reports showing vulnerabilities, compliance status, and security trends across monitored applications.	
Priority	Medium	
Pre-conditions	- Security officer has authenticated with appropriate permissions - Security monitoring is enabled on target applications - Vulnerability scanning has been performed	
Post-conditions	- Security report is generated and stored - Report is available for download in multiple formats - Compliance metrics are calculated and displayed	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Security officer navigates to Security → Reports page.	System displays security reporting interface with report options.
2	Officer configures report: - Report type: Comprehensive Security Audit - Time period: Last 30 days - Applications: All (3 applications) - Include: Vulnerabilities, Compliance, Security events, Trends - Format: PDF + JSON	
3	Officer clicks "Generate Report" button.	System validates report parameters and initiates generation process.

4		System queries multiple data sources: - Vulnerability scan results - Security events from logs - Access control violations - Encryption compliance status
5		System calculates security metrics: - Total vulnerabilities: 47 - Critical: 3, High: 12, Medium: 22, Low: 10 - OWASP Top 10 coverage: 9/10 protected - Compliance score: 87% - Security events: 234 blocked attempts
6		System generates trend analysis: - Vulnerabilities decreased 15% vs last month - Average time to patch: 4.2 days - SQL injection attempts: 45 (blocked)
7		System invokes AI to generate executive summary: "Overall security posture is good with 87% compliance. Immediate attention required for 3 critical vulnerabilities in payment service. Recommend updating Spring Boot to version 3.2.1."
8		System compiles report with sections: - Executive Summary - Vulnerability Details - Compliance Matrix - Security Events Timeline - Recommendations - Appendix (CVE details)

9		System generates PDF with charts, tables, and visualizations.
10		System stores report in document storage with metadata.
11		System displays: "Report generated successfully. Report ID: SEC-2025-1127"
12	Officer clicks "Download PDF"	System serves PDF file for download.
13	Officer clicks "Email Report"	System prompts for email recipients.
14	Officer enters email addresses and clicks "Send".	System sends email with: - PDF attachment - Report summary in email body - Secure download link
15		System logs report generation in audit trail with timestamp and user.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Insufficient Data	- If no vulnerability scans exist for selected period, system displays warning: "No vulnerability scan data found. Report will be incomplete." - Officer can choose to continue with partial report or cancel.
2	Report Generation Timeout	- If report generation takes > 60 seconds (large dataset), system processes asynchronously. - System displays: "Report is being generated. You will receive email notification when ready." - System sends email with download link upon completion.
3	Compliance Framework Selection	- Officer can select specific framework (GDPR, HIPAA, SOC2, PCI-DSS) for targeted compliance report.

		- System adjusts compliance checks and scoring based on selected framework requirements.

4.3 Nonfunctional Requirements

Nonfunctional requirements define the quality attributes, constraints, and system characteristics that the SeeStack platform must satisfy. These requirements are critical for ensuring the system delivers reliable performance, maintains security, scales effectively, and provides an intuitive user experience. Unlike functional requirements that specify what the system should do, nonfunctional requirements specify how well the system should perform its functions and operate under various conditions. This section outlines the key nonfunctional requirements organized into categories including performance, reliability, security, usability, scalability, maintainability, and compliance. Each requirement has been carefully derived from the project objectives, technical architecture, and the need to serve diverse development teams managing modern distributed systems.

4.3.1 Performance Requirements

Response Time and Latency

The SeeStack dashboard and API must respond to user requests with minimal latency to ensure a responsive, productive user experience. All dashboard views including error lists, performance metrics, stress test results, and security alerts shall load completely within 2 seconds under normal operating conditions with up to 500 concurrent users connected simultaneously. API endpoints for fetching error events, metrics, and analysis results must respond within 500 milliseconds at the 95th percentile (P95), with no more than 5% of requests exceeding 1 second of response time. Real-time updates via WebSocket connections must be delivered to connected dashboards within 100 milliseconds of event ingestion to ensure developers see live data with minimal delay. Under peak load conditions with 1000 concurrent users, response times may increase to 3-5 seconds for non-critical operations, but critical alerts and error notifications must continue to meet the 500-millisecond requirement.

Throughput and Event Processing

The SeeStack platform must handle high-volume telemetry streams and process events at a rate sufficient to support organizations with large-scale distributed applications generating millions of events daily. The backend event ingestion pipeline, built on Apache Kafka and Spring Boot microservices, must process at least 50,000 error events per second during peak traffic periods without data loss or significant queue buildup. The stress testing module must be capable of generating and processing results from simulated load tests with up to 100,000 concurrent virtual users, measuring response times and throughput at the 50th, 95th, and 99th percentiles without performance degradation. Real-time metric collection from monitored applications must support capturing and aggregating metrics including CPU, memory, and response times from at least 10,000 distinct endpoints or services simultaneously. Database systems including ClickHouse for time-series data and Redis for caching must efficiently handle concurrent read and write operations to prevent bottlenecks when thousands of developers simultaneously query dashboards and generate stress tests.

Resource Utilization

The SeeStack platform must operate efficiently and minimize resource consumption to reduce infrastructure costs and environmental impact while maintaining high performance. The backend services and microservices including Spring Boot applications, Kafka brokers, and Redis cache must consume no more than 4GB of RAM under normal operating load with 500 concurrent dashboard users and 50,000 events per second ingestion rate. CPU utilization across the backend cluster should remain below 70% during normal operations and below 85% during peak load, ensuring headroom for handling traffic spikes without performance degradation. Each Docker container deployed for backend services must have memory limits set between 512MB and 2GB depending on its role, with health checks ensuring containers restart if memory consumption exceeds defined thresholds. The React frontend dashboard must be optimized to load in under 100KB of JavaScript after gzip compression, ensuring fast load times for users on slower network connections and reducing bandwidth costs.

4.3.2 Reliability and Availability Requirements

System Uptime and Availability

SeeStack must maintain high availability to ensure developers and DevOps teams can reliably access monitoring and alerting capabilities whenever needed, minimizing missed incidents during system downtime. The platform must achieve and maintain at least 99.9% uptime (approximately 43 seconds of downtime per month) during normal operational periods, excluding planned maintenance windows which must be scheduled and communicated in advance. Scheduled maintenance and updates must be performed during designated maintenance windows with at least 48 hours advance notice to all users, and must not exceed 1 hour of downtime per month on average. The monitoring and alerting infrastructure must operate independently from the dashboard and analytics components, ensuring that even if the dashboard becomes unavailable, error detection and alert delivery continue functioning normally. Health monitoring endpoints must be available on all backend services and exposed through load balancers, allowing automated monitoring and failover to route traffic away from failing instances within 30 seconds.

Fault Tolerance and Graceful Degradation

The SeeStack platform must be resilient to component failures and gracefully degrade functionality rather than failing completely when individual services or infrastructure components experience problems. If the AI analysis service becomes unavailable or experiences performance degradation, the system must continue accepting and storing error events, making them available in the dashboard, while queuing AI analysis requests for processing when the service recovers. In the event of Redis cache failures, the system must fall back to direct database queries with increased latency, typically doubling response times, rather than returning errors to users, allowing the system to continue functioning in a degraded mode. If Kafka brokers experience temporary unavailability, the SDK running within client applications must implement local buffering storing up to 1000 events in memory with automatic retry and exponential backoff strategies to prevent data loss during transient network issues. Dashboard WebSocket connections that are interrupted must automatically reconnect with exponential backoff, ensuring users seamlessly resume receiving real-time updates without manual intervention.

Data Resilience and Backup

All critical data stored in SeeStack must be protected against loss through redundancy, regular backups, and disaster recovery procedures to ensure data integrity and recoverability. Error events, performance metrics, and configuration data must be replicated across at least three independent storage nodes or availability zones, ensuring no single point of failure can result in data loss. Daily incremental backups and weekly full backups of all databases including ClickHouse, Redis, and PostgreSQL must be performed

automatically and stored in geographically distributed locations to protect against regional disasters. Backup integrity must be verified through automated testing, performing periodic restore operations to confirm backups can be successfully recovered and data consistency is maintained. The recovery point objective (RPO) must be no more than 1 hour, meaning no more than 1 hour of data loss is acceptable in the worst-case scenario, and the recovery time objective (RTO) must be no more than 4 hours to restore full service.

4.3.3 Security Requirements

Authentication and Authorization

SeeStack must implement a passwordless authentication mechanism ensuring only authorized users and applications can access the dashboard and sensitive data. Users must authenticate using email-based passwordless login methods, with a one-time password (OTP) or magic link sent to their registered email address enabling secure access without password complexity requirements. When users click the magic link or enter the OTP sent to their email, the system must validate the token within 15 minutes of generation, ensuring security against interception or replay attacks. Upon successful authentication, the system must generate a JWT access token with a 10-hour expiration time for API and dashboard access, along with a JWT refresh token also expiring after 10 hours enabling token refresh without requiring re-authentication. API authentication for programmatic access must require valid API keys issued to projects and associated with verified user accounts, with automatic revocation if credentials are compromised or if suspicious activity is detected.

Data Protection and Encryption

All sensitive data transmitted between clients and SeeStack servers must be encrypted to protect against eavesdropping and man-in-the-middle attacks. All data in transit must use HTTPS with TLS 1.2 or later, with strong cipher suites including ECDHE and AES-256 enforced for all connections to the SeeStack platform, dashboards, and API endpoints, with certificate pinning available for high-security deployments. WebSocket connections for real-time dashboard updates must also be encrypted using WSS (WebSocket Secure) with the same TLS encryption standards as HTTPS connections. Data at rest in databases, caches, and file storage must be encrypted using AES-256 encryption with encryption keys managed through a secure key management service (KMS) separate from the data storage systems to prevent attackers from decrypting data even if they gain access to storage systems. Encryption keys must be rotated automatically every 90 days, with procedures ensuring data remains readable after key rotation and old keys are securely destroyed according to security standards.

Vulnerability Management and Security Scanning

SeeStack must implement comprehensive security vulnerability management to identify and remediate security issues promptly, maintaining a secure platform for users to trust with their application data. All dependencies including libraries, frameworks, and packages must be regularly scanned using automated software composition analysis (SCA) tools to identify known vulnerabilities (CVEs), with critical vulnerabilities patched within 48 hours. Source code must be scanned for security vulnerabilities using static application security testing (SAST) tools integrated into the continuous integration pipeline, blocking commits that introduce high-severity security issues. Container images and Docker registries must be scanned for vulnerabilities at build time and continuously monitored for newly discovered vulnerabilities in production deployments, with automated alerts and remediation workflows for critical issues. Infrastructure and configuration must be audited against industry-standard security benchmarks including CIS and NIST to identify misconfigurations, open ports, unnecessary services, and insecure defaults that could expose systems to attack.

Secure SDKs and Integrations

SDKs provided to client applications must be built with security as a primary concern, ensuring SDKs do not introduce vulnerabilities into customer applications. SDKs must not execute arbitrary code, transmit sensitive data including passwords and API keys unnecessarily, or modify application behavior beyond collecting and transmitting telemetry data to SeeStack servers. All SDKs must validate and sanitize event data before transmission to prevent injection attacks, ensuring error messages, stack traces, and context data do not contain malicious payloads that could affect the SeeStack platform. SDKs must implement secure communication protocols including HTTPS and TLS with certificate validation to prevent man-in-the-middle attacks, with support for custom certificate authorities for on-premises deployments with internal PKI. Integration webhooks and external API integrations must verify webhook signatures using cryptographic algorithms (HMAC-SHA256) to ensure callbacks originate from the SeeStack platform and have not been tampered with.

4.3.4 Scalability Requirements

Horizontal Scalability

The SeeStack platform must scale horizontally to accommodate growing numbers of users, applications, and events without requiring changes to application code or database schemas. Stateless backend services including Spring Boot microservices must be designed to scale horizontally by adding additional instances behind a load balancer, with new instances automatically joining the service mesh and receiving traffic without manual configuration. Kafka topic partitioning must enable parallel event processing across multiple consumer instances, with at least 100 partitions for high-throughput topics to support distributed processing and scaling to handle growth in event volume. Redis caching layers must be configured in cluster mode with multiple nodes, allowing automatic distribution of cache data across cluster members and enabling cache scaling as memory requirements grow. Container orchestration platforms including Kubernetes must automatically scale pod counts based on CPU and memory utilization metrics, adding new instances during high load and removing them during low-traffic periods to optimize resource utilization and cost.

Data Volume and Storage Scalability

The SeeStack platform must efficiently handle ever-increasing volumes of event data including errors, metrics, and traces generated by customer applications, storing and querying data efficiently even as data volumes grow into terabytes. ClickHouse time-series database must be configured with distributed table engines and data partitioning by date and time to enable efficient query execution on multi-terabyte datasets, with automatic data compression using ZSTD reducing storage requirements. Data retention policies must be implemented to manage storage costs, automatically archiving older data to cold storage such as S3 or Glacier after a configurable retention period. Query optimization must include materialized views, pre-aggregated metrics, and sampling strategies to enable fast queries on large datasets without scanning all raw data, reducing CPU and memory requirements for high-volume queries. Storage must be provisioned on scalable cloud infrastructure allowing on-demand expansion without downtime, with monitoring alerting administrators when storage reaches 80% capacity to plan additional storage provisioning.

Network and Concurrent Connection Scalability

The SeeStack platform must support thousands of concurrent connections from SDKs, dashboard users, and integrations without experiencing performance degradation or connection limits. SDK event ingestion endpoints must support at least 100,000 concurrent TCP connections from distributed client applications, with connection pooling and keep-alive mechanisms reducing overhead and connection count. WebSocket connections for real-time dashboard updates must scale to at least 10,000 concurrent connected users

simultaneously receiving live data, with pub/sub messaging patterns enabling efficient broadcast of updates to multiple subscribers. API rate limiting must be configured to prevent abuse while accommodating legitimate high-volume integrations, with tiered limits based on subscription tier enabling appropriate resource allocation. CDN integration for static assets including JavaScript, CSS, and images must distribute content geographically, reducing latency for users worldwide and offloading traffic from origin servers.

4.3.5 Usability and User Experience Requirements

Intuitive User Interface and Navigation

The SeeStack dashboard and user interfaces must be designed to be intuitive and user-friendly, enabling developers and DevOps engineers of varying experience levels to quickly become productive without extensive training. Navigation must follow established user interface patterns and conventions including left sidebar navigation, breadcrumbs, and search functionality enabling users to easily find features and information. Visual design must follow established design systems and accessibility guidelines including WCAG 2.1 Level AA, with consistent color schemes, typography, spacing, and interactive elements enabling users to focus on content. Information hierarchy must clearly distinguish between different types of alerts including critical, high, medium, and low using color coding, icons, and typography, ensuring users quickly identify urgent issues. Dashboard customization must allow users to create personalized views with preferred metrics, charts, and filters, saving configurations and enabling quick access to frequently viewed information.

Onboarding and Getting Started Experience

New users must be able to quickly get value from SeeStack with minimal setup and learning curve, reducing time-to-productivity and increasing adoption rates. Guided onboarding workflows must step new users through initial setup including project creation, SDK integration, and viewing first errors and metrics, with example code snippets for popular frameworks. Interactive tutorials and in-app guidance must explain key features such as error grouping, stress testing, and alert configuration through contextual help during first-time feature access. Sample dashboards and pre-built alert templates must accelerate time-to-value, allowing users to get immediate insights before customizing configurations for their specific use cases. Documentation must include quick-start guides for popular programming languages and frameworks, with copy-paste examples enabling integration within minutes rather than hours.

Search and Filtering Capabilities

The SeeStack platform must provide powerful search and filtering capabilities enabling users to quickly locate specific errors, metrics, and insights within potentially millions of events. Full-text search must enable searching error messages, stack traces, and metadata using keyword queries, with search results ranked by relevance and filtered by time range and severity. Tag-based filtering must allow users to organize and search events by application, environment including dev and production, service, or custom tags, enabling users to focus on relevant data. Time-range selection must provide flexible options including preset ranges such as last 1 hour and last 24 hours as well as custom date/time ranges enabling users to analyze events from specific periods. Multi-faceted search must combine multiple filters simultaneously enabling complex queries without requiring specialized query languages for advanced users.

4.3.6 Maintainability and Extensibility Requirements

Code Quality and Documentation

The SeeStack codebase must maintain high code quality through consistent style, comprehensive documentation, and automated quality checks ensuring long-term maintainability and ease of onboarding new developers. Code must adhere to established style guides and coding standards for each programming

language, enforced through automated linters including ESLint for JavaScript and Checkstyle for Java integrated into the continuous integration pipeline. Comprehensive inline documentation must explain complex algorithms, business logic, and architectural decisions, with code comments clarifying non-obvious implementations and rationale for specific design choices. Unit test coverage must maintain at least 80% code coverage for critical business logic including error processing, AI analysis, and authentication, with continuous integration blocking commits that reduce coverage below the target. API documentation must be comprehensive and current, generated from code annotations including Swagger and OpenAPI with examples for each endpoint enabling integration developers to understand required parameters.

Plugin Architecture and Extensibility

SeeStack must provide extension points enabling customers and partners to build integrations and plugins extending functionality without modifying core platform code. Plugin interfaces must define standard APIs for error processing, metric collection, alert delivery, and dashboard components, enabling third-party developers to create plugins implementing these interfaces. Webhook endpoints must allow external systems to subscribe to events including errors detected, alerts triggered, and tests completed, receiving HTTP callbacks with event data enabling custom automation. Custom alert channels must be implementable as plugins, allowing organizations to integrate with internal communication systems beyond pre-built integrations. Integration SDKs and sample code must be available for popular platforms enabling developers to build integrations quickly using documented APIs and patterns.

Configuration Management and Customization

SeeStack must enable extensive customization through configuration files, environment variables, and administrative interfaces rather than requiring code modifications or recompilation. Configuration as code must enable storing alert rules, notification channels, and data retention policies in version-controlled configuration files enabling infrastructure-as-code practices and configuration auditing. Environment-based configuration must use environment variables for deployment-specific settings including database URLs and API keys enabling the same container image deployment across development, staging, and production environments. Admin interfaces must enable non-technical administrators to configure alert thresholds, retention policies, user permissions, and integrations without requiring database access or code changes. Feature flags must enable gradual rollout of new features, A/B testing of UI changes, and disabling features causing issues in production without requiring code deployment or restarting services.

4.3.7 Compliance and Regulatory Requirements

Data Privacy and Protection Regulations

SeeStack must comply with international data protection regulations to protect user privacy and enable customers to meet their own regulatory obligations. GDPR compliance must be implemented for users and data in the European Union, including data subject access requests, right to be forgotten, data portability, and privacy impact assessments. CCPA and CPRA compliance must be implemented for California residents, including disclosing data collection practices, enabling opt-out rights, and preventing discriminatory pricing for exercising privacy rights. HIPAA compliance must be available for healthcare customers, including business associate agreements (BAAs), encryption, access controls, and audit logging required by healthcare regulations. Data processing agreements must be available with customers specifying data handling practices, retention policies, and responsibilities for data protection enabling customers to meet their own regulatory obligations.

Industry Standards and Certifications

SeeStack should achieve recognized security and quality certifications to provide assurance to enterprise customers and demonstrate commitment to security and reliability best practices. SOC 2 Type II certification must be obtained and maintained, demonstrating that security, availability, processing integrity, confidentiality, and privacy controls are operating effectively through independent audits. ISO 27001 certification must be pursued to demonstrate implementation of comprehensive information security management systems aligned with international best practices. Regular security audits and penetration testing must be conducted by independent third-party security firms, with findings remediated and evidence of remediation provided to customers upon request. Industry-specific compliance requirements must be researched and implemented for targeted verticals, including healthcare, finance, education, and government sectors.

Regulatory Reporting and Audit Capabilities

SeeStack must provide comprehensive audit logs and reporting capabilities enabling customers to meet regulatory reporting obligations and demonstrate compliance to auditors and regulators. Comprehensive audit logs must capture all user actions including login, data access, and configuration changes with timestamps, user identities, and IP addresses enabling forensic analysis during security investigations. Audit log immutability must be ensured through digital signatures or write-once storage, preventing tampering with logs to hide unauthorized access or malicious activities. Retention of audit logs must comply with regulatory requirements, with logs retained for extended periods as required by specific regulations, with secure archival after initial retention periods. Audit log export must enable customers to retrieve logs in standard formats including CSV and JSON for analysis, reporting, and import into security information and event management (SIEM) systems.

4.3.8 Deployment and Operations Requirements

Container and Infrastructure Automation

SeeStack must be designed for deployment on containerized infrastructure and modern cloud platforms, enabling automated provisioning and management of infrastructure resources. Docker containerization must be mandatory for all components including backend services, databases, and caching layers, with official Docker images published and regularly updated with security patches. Kubernetes deployment manifests must be provided and maintained, enabling users to deploy SeeStack on any Kubernetes cluster including public cloud, on-premises, and hybrid deployments with standardized deployment patterns. Infrastructure as Code must enable defining and provisioning the entire SeeStack infrastructure through version-controlled configuration files enabling reproducible deployments and infrastructure auditing. Helm charts must be created and maintained for Kubernetes deployments, providing parameterized deployment templates enabling users to customize deployments for their specific requirements.

Monitoring and Observability of SeeStack Itself

SeeStack must implement comprehensive monitoring and observability of its own infrastructure and services, enabling rapid detection and resolution of platform issues affecting customers. Distributed tracing must be implemented using standard protocols including OpenTelemetry to track requests across microservices, enabling identification of performance bottlenecks and service failures. Structured logging must capture application events including service startup, configuration loading, and errors in JSON format with correlation IDs enabling tracing of requests through the system. System metrics including CPU, memory, disk, and network must be collected from all infrastructure components and stored in time-series databases, enabling analysis of infrastructure utilization and capacity planning. Alert rules must be

configured for SeeStack infrastructure itself, triggering notifications when services become unhealthy or performance degrades, enabling rapid response to platform issues.

Disaster Recovery and Business Continuity

SeeStack must implement comprehensive disaster recovery and business continuity procedures ensuring the platform can continue operating during regional failures, natural disasters, or major infrastructure incidents. Multi-region deployment must be supported, with infrastructure, databases, and services distributed across geographically separated regions enabling failover if an entire region becomes unavailable. Automated failover mechanisms must detect failures in primary regions and automatically redirect traffic to secondary regions, with health checks verifying failover is complete and monitoring confirms secondary region health. Regular disaster recovery drills must be conducted, simulating regional failures and testing the ability to recover services in secondary regions within the defined recovery objectives. Cross-region replication must ensure critical data including configuration and user accounts are synchronized to secondary regions within acceptable latency, enabling recovery from primary region failures.

4.4 Analysis Models

Use Case Diagram

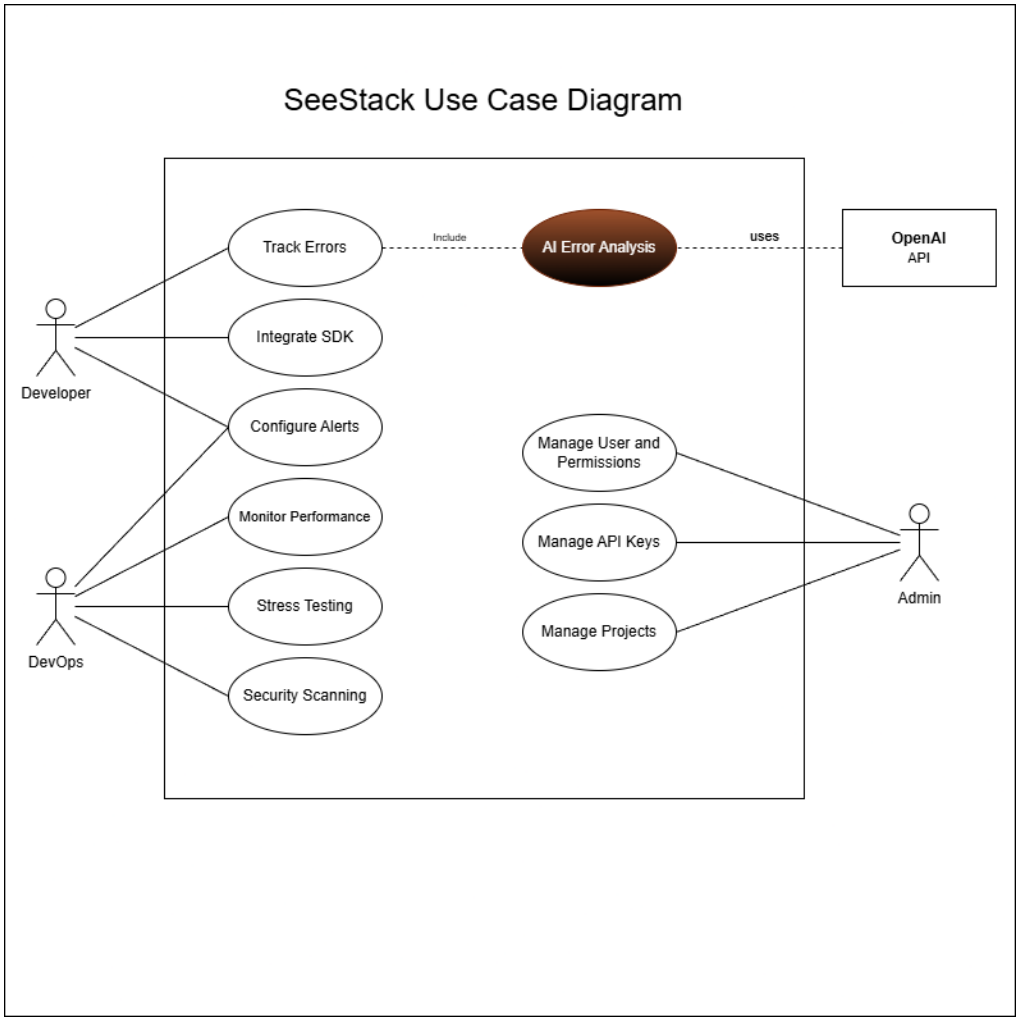


Figure 8: SeeStack Use Case Diagram

The Use Case Diagram visually outlines the functional interactions between the system's actors and the SeeStack platform, defining the scope of operations for different user roles. The system identifies three primary actors: Developer, DevOps, and Admin, along with an interaction with the external OpenAI API.

- **Developer**: This actor is primarily responsible for application integration and maintenance. Key use cases include "Integrate SDK," "Configure Alerts," and "Track Errors." The "Track Errors" use case explicitly includes an "AI Error Analysis" process, which leverages the OpenAI API to generate insights.

- **DevOps**: This role focuses on infrastructure reliability and monitoring. The DevOps actor interacts with the system to "Configure Alerts," "Monitor Performance," "Stress Testing," and perform "Security Scanning."
- **Admin**: The administrator oversees the platform's management and access control. Their functions include "Manage User and Permissions," "Manage API Keys," and "Manage Projects."

CHAPTER 5: SYSTEM DESIGN

5.1 Database Schema Overview

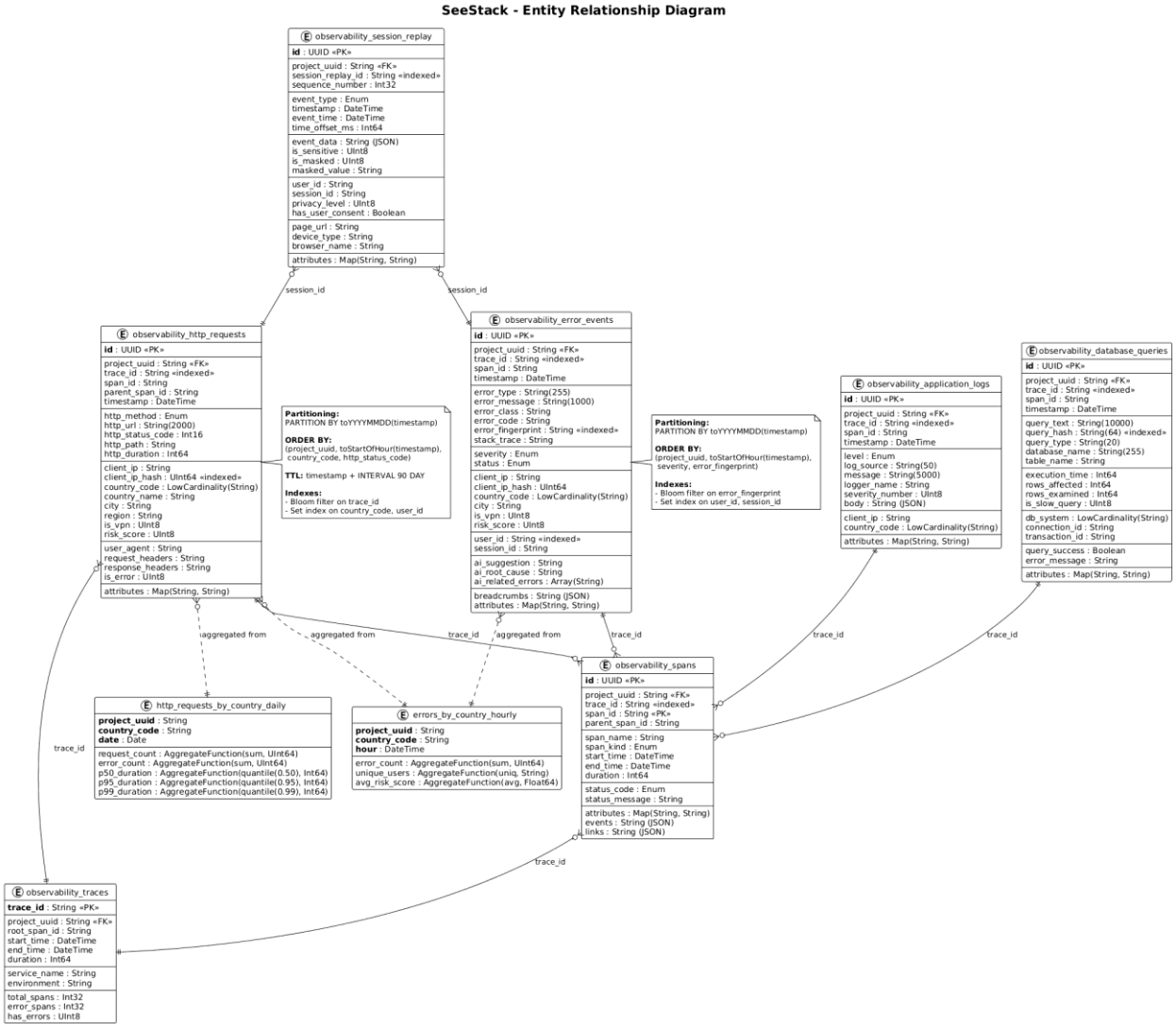


Figure 9: Database Schema Overview

The SeeStack Entity Relationship Diagram (ERD) illustrates how observability data is stored, organized, and interconnected across the platform.

Core Entities

1. **Session Replays**

This entity captures complete user interaction sessions, incorporating privacy controls, device fingerprinting, and ordered event sequences. Each session is associated with multiple HTTP requests and error events through the session_id key, allowing comprehensive contextual correlation.

2. **HTTP Requests**

   This table maintains detailed transactional metadata, including request URLs, status codes, headers, performance indicators, and geographic attributes. It uses trace_id to support distributed tracing and employs time-based partitioning (PARTITION BY toYYYYMMDD) to optimize query efficiency at scale.

3. **Error Events**

   Error events record runtime exceptions along with stack traces, severity classifications, and AI-generated remediation suggestions. These events reference both sessions and distributed traces (session_id and trace_id), enabling effective cross-service error correlation and root-cause identification.

4. **Distributed Traces**

   This entity adheres to OpenTelemetry tracing standards, modeling request propagation across microservices. It contains span hierarchies, timing metrics, and aggregated error indicators, supporting comprehensive end-to-end latency and performance analysis.

5. **Security Scans**

   Security scan records document the lifecycle of vulnerability assessments, including scan status, execution metadata, and JSON-formatted findings. Each scan is linked to relevant application activity for contextual security insight.

6. **Database Queries**

   This table monitors database query behavior, capturing execution durations, success indicators, and slow-query metrics to support performance optimization and diagnostics.

7. **Application Logs**

   Application logs are aggregated by severity level and support full-text search for rapid retrieval. Log entries also correlate with trace identifiers to maintain continuity across the observability pipeline.

Relational Structure

All major entities are interconnected through shared foreign keys, primarily trace_id, session_id, and project_uuid, which provide unified navigation across the full observability stack and support comprehensive multi-dimensional analysis.

5.2 Component Diagram Overview

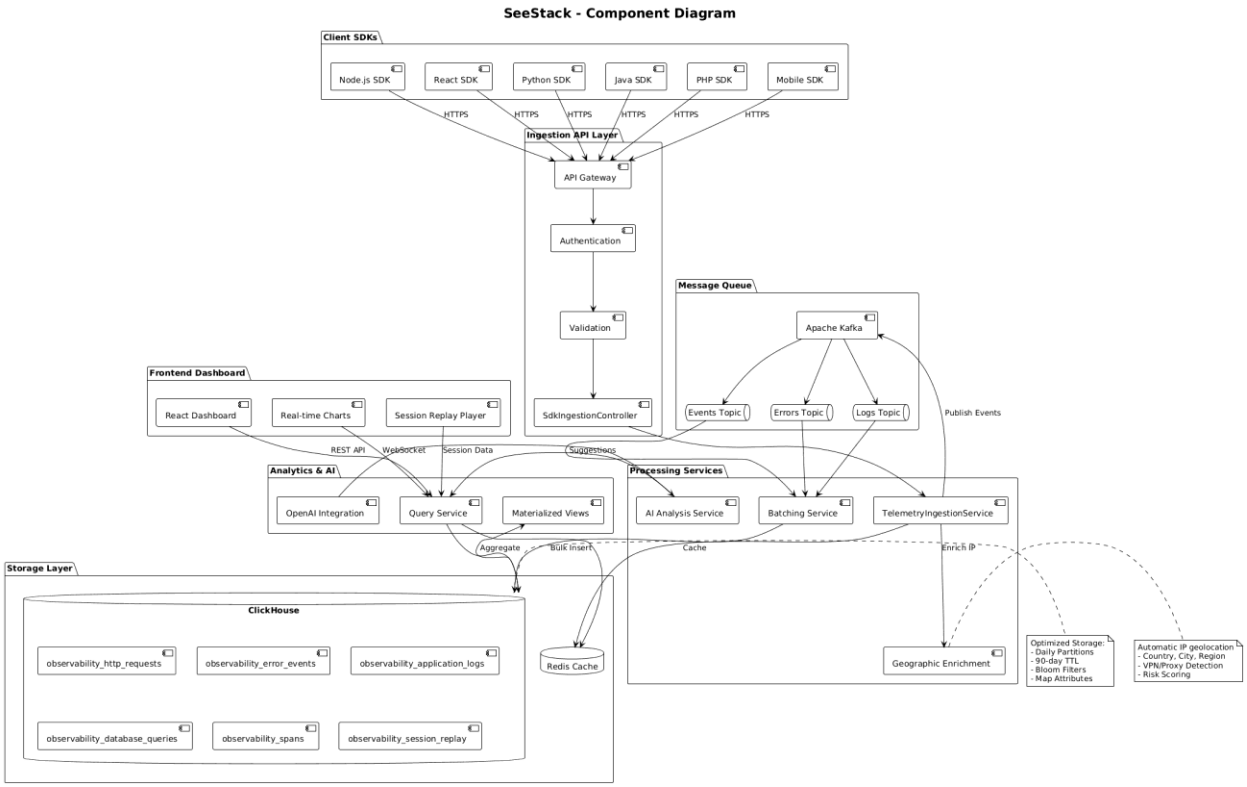


Figure 10: Component Diagram Overview

The SeeStack Component Diagram illustrates the distributed architecture of the platform and explains how each system layer contributes to telemetry ingestion, processing, analysis, and presentation.

System Components

1. **Client SDKs**

   The platform provides multiple language-specific SDKs, including Node.js, React, Python, Java, PHP, and mobile variants. These SDKs collect telemetry data and transmit it securely over HTTPS to the ingestion layer, enabling seamless integration across diverse application environments.

2. **Ingestion API Layer**

   This layer manages the intake of telemetry submissions. The API Gateway performs authentication, validation, and request routing, while the SdkIngestionController receives batched telemetry and forwards it to downstream processing services.

3. **Message Queue**

Apache Kafka acts as the event streaming backbone and includes dedicated topics for events, errors, and logs. This queueing architecture separates ingestion from processing, supports fault tolerance, and enables the platform to scale under high event volumes.

4. **Frontend Dashboard**

   The dashboard is developed using React and provides real-time charts, session replay features, and interactive data visualizations. It retrieves processed data through REST APIs and WebSocket streams to maintain continuous monitoring capabilities.

5. **Processing Services**

   The platform incorporates three primary processing services, each responsible for a distinct operational function.

   The AI Analysis Service generates structured error explanations through OpenAI integration.

   The Batching Service aggregates large volumes of telemetry events to improve storage efficiency.

   The TelemetryIngestionService enriches incoming data with geographic and contextual metadata.

6. **Analytics and AI**

   The Query Service performs complex aggregations across telemetry datasets, supporting advanced analytical operations. Materialized views are maintained to improve dashboard performance, and OpenAI integration provides intelligent suggestions that assist with diagnostics and interpretation.

7. **Storage Layer**

   The storage architecture is built on ClickHouse, a high-performance columnar database that stores six primary telemetry tables. The schema uses 90-day TTL policies, Bloom filters, and the MergeTree engine to optimize performance. Redis caching is applied to accelerate frequently accessed queries, and the geographic enrichment subsystem adds contextual attributes such as country, city, region, VPN or proxy status, and risk scoring.

5.3 Class Diagram Overview

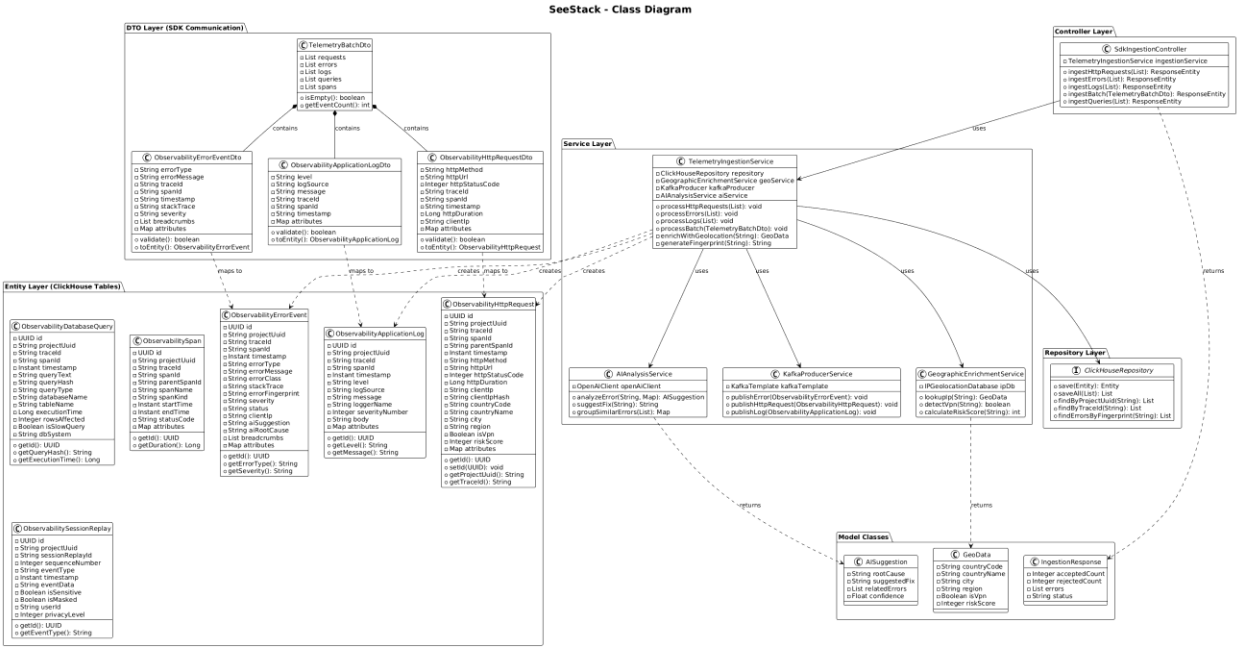


Figure 11:Class Diagram Overview

The SeeStack Class Diagram presents the layered architecture of the platform, organized into five tiers that define the flow of data, control, and business logic across the system.

Architecture Layers

1. **DTO Layer (SDK Communication)**

   This layer contains the TelemetryBatchDto, which functions as the primary data transfer object for all SDK interactions. It aggregates collections of HTTP requests, error events, logs, database queries, and security scan results submitted by client applications prior to ingestion.

2. **Controller Layer**

   The SdkIngestionController provides REST endpoints for receiving telemetry data from multiple SDKs. Methods such as ingestHttpRequest, ingestError, and ingestBatch accept incoming observability events and forward them to downstream processing services.

3. **Service Layer**

   The TelemetryIngestionService manages the core business logic of the ingestion pipeline. It processes incoming telemetry lists, enriches each record with geographic metadata, publishes events to Kafka for real-time streaming, and coordinates with AI analysis services to generate automated error interpretations.

4. **Repository Layer**

   This layer contains several specialized repositories, each designed to interact with a specific subsystem.

   The ClickHouseRepository performs high-performance writes to the columnar storage engine.

   The KafkaProducerService manages event streaming pipelines and ensures reliable publishing to Kafka topics.

   The GeographicEnrichmentService appends location metadata to incoming requests, including country, region, and risk-related attributes.

5. **Entity Layer**

   This layer maps directly to the ClickHouse database schema. It includes entities such as ObservabilityErrorEvent, ObservabilityHttpRequest, ObservabilitySpan, ObservabilityApplicationLog, and ObservabilityDatabaseQuery. Each entity defines field mappings, validation rules, and constraints required for consistent storage.

Model Classes

Supporting model classes include AISuggestion, which stores AI-generated recommendations, GeoData, which represents geographic information derived during enrichment, and IngestBatchResponse, which reports the status of telemetry ingestion operations

5.4 Sequence Diagram Overview

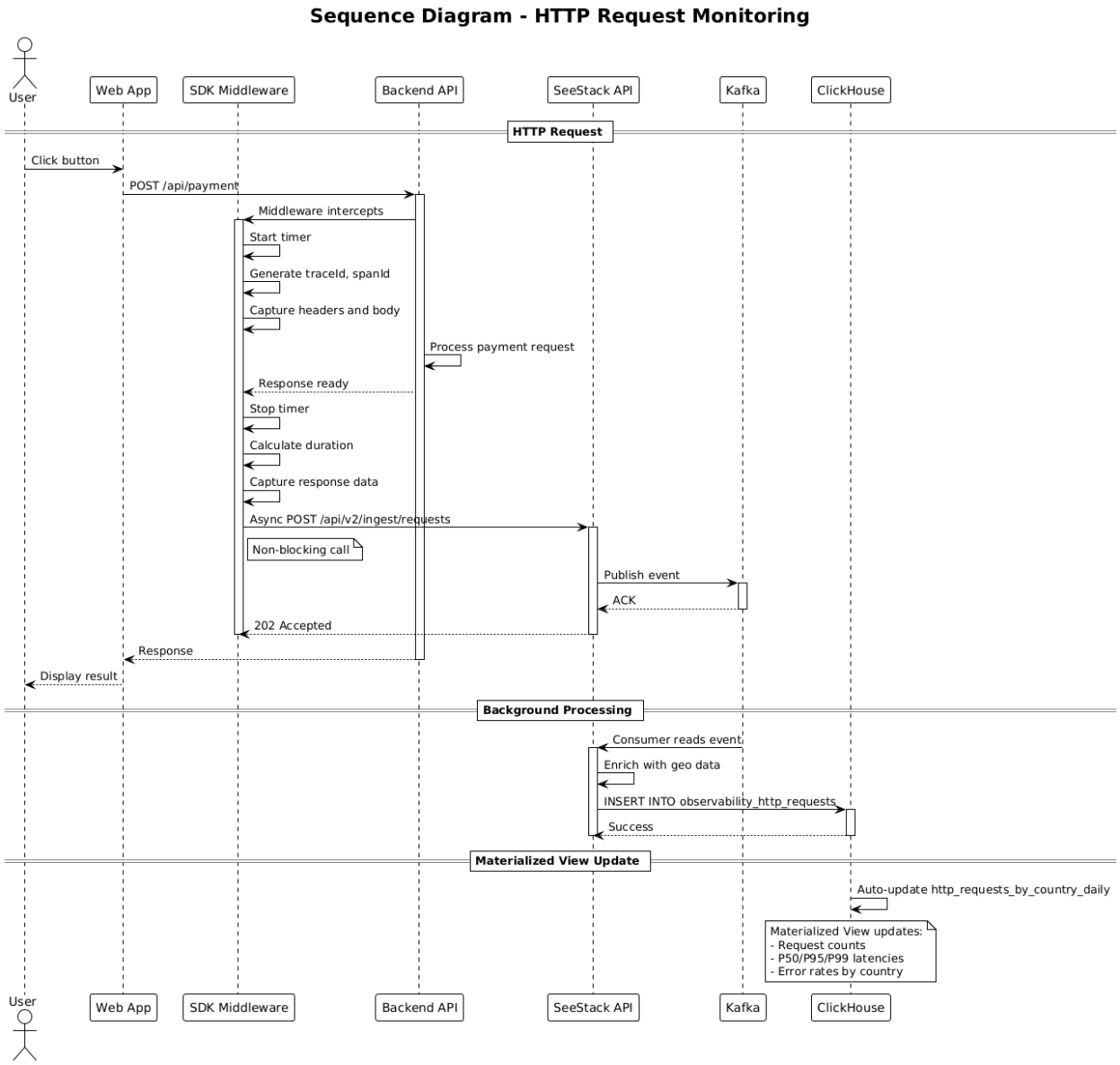


Figure 12: Sequence Diagram Overview

This sequence diagram describes the end-to-end workflow of HTTP request monitoring in the SeeStack system.

When a user interacts with the web application (for example, by clicking a button), the Software Development Kit (SDK) middleware intercepts the /api/payment request, starts a timer, generates trace and span identifiers (traceId and spanId), and captures the request headers and body. After the backend completes processing and returns the HTTP response, the middleware records the response, stops the timer, and computes the request duration. The middleware then sends the captured telemetry

asynchronously to the SeeStack API, which responds with an HTTP 202 (Accepted) status code so that the user-facing response is not delayed.

In the background, SeeStack publishes the request event to a Kafka topic, where a consumer service enriches it with geolocation metadata. The enriched event is then inserted into the **observability_http_requests** table in ClickHouse. Materialized views are automatically updated to maintain aggregated metrics such as request counts, P50/P95/P99 latencies, and error rates by country, which support low-latency analytics and dashboard visualizations.

5.5 Sequence Diagram Overview

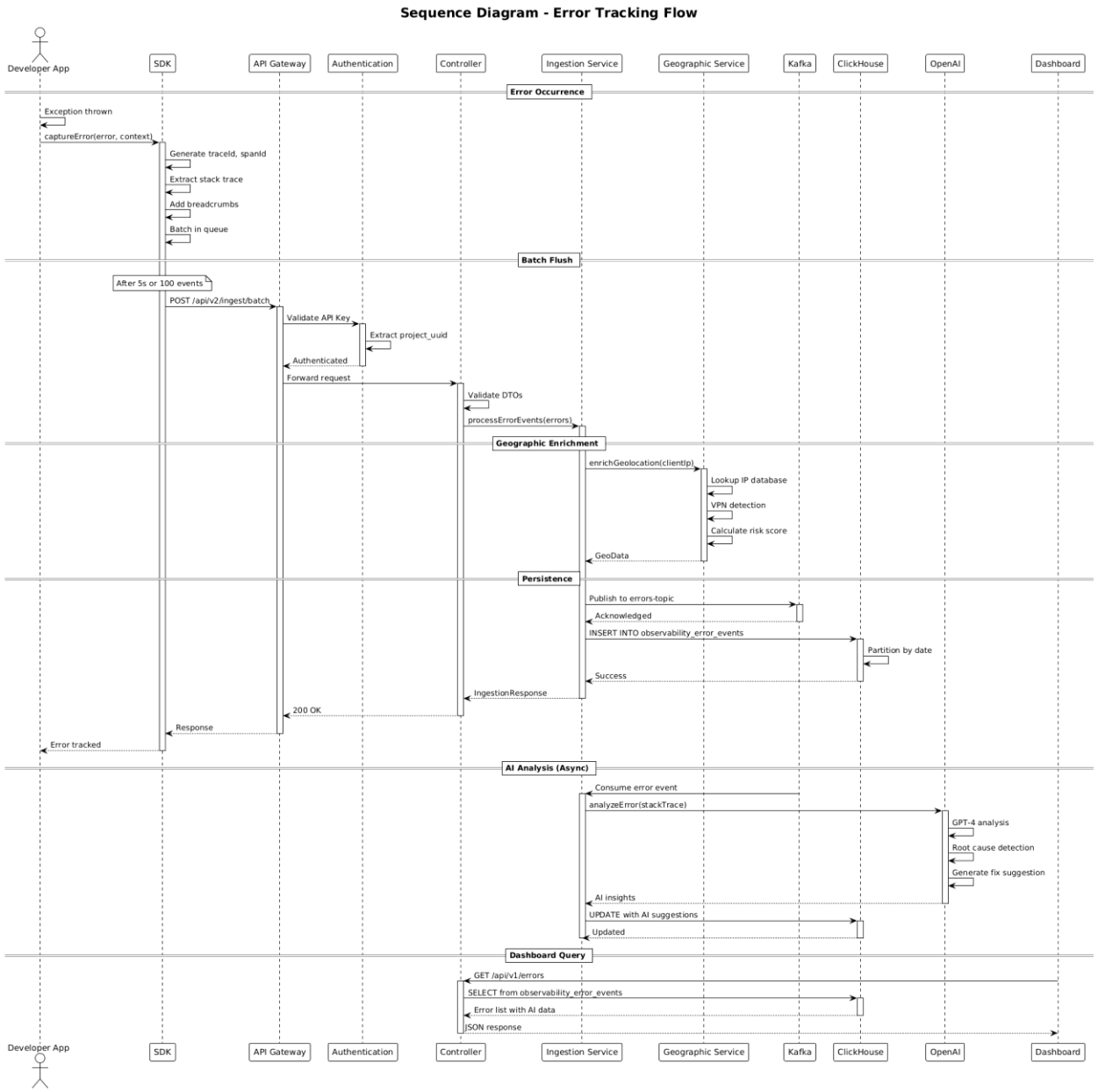


Figure 13: Sequence Diagram Overview

This sequence diagram explains the end-to-end error tracking flow in SeeStack, from the moment an exception occurs in the application to the point where the developer views AI-generated insights on the dashboard.

When an exception is thrown in the developer's application, the SDK captures the error details, including the stack trace, contextual metadata, and breadcrumbs, then buffers these events and periodically sends them in batches to the ingestion API. The API gateway validates the API key, authenticates the request, and forwards it to the controller, which validates the data transfer objects (DTOs) and passes the error events to the ingestion service for further processing.

The ingestion service enriches each error with geolocation attributes such as country, city, VPN or proxy usage, and a risk score, then publishes the resulting events to Kafka and persists them into ClickHouse partitioned by date. In parallel, the AI analysis service asynchronously consumes error events, invokes OpenAI to analyze the stack trace and context, and updates the stored records with AI-generated suggestions and root-cause insights.

When the dashboard queries recent errors, the system reads from ClickHouse, includes the associated AI insights, and returns a JSON response to the frontend. As a result, the developer can view each tracked error in the user interface together with its analysis, probable root cause, and recommended remediation steps.

5.6 Deployment Diagram Overview

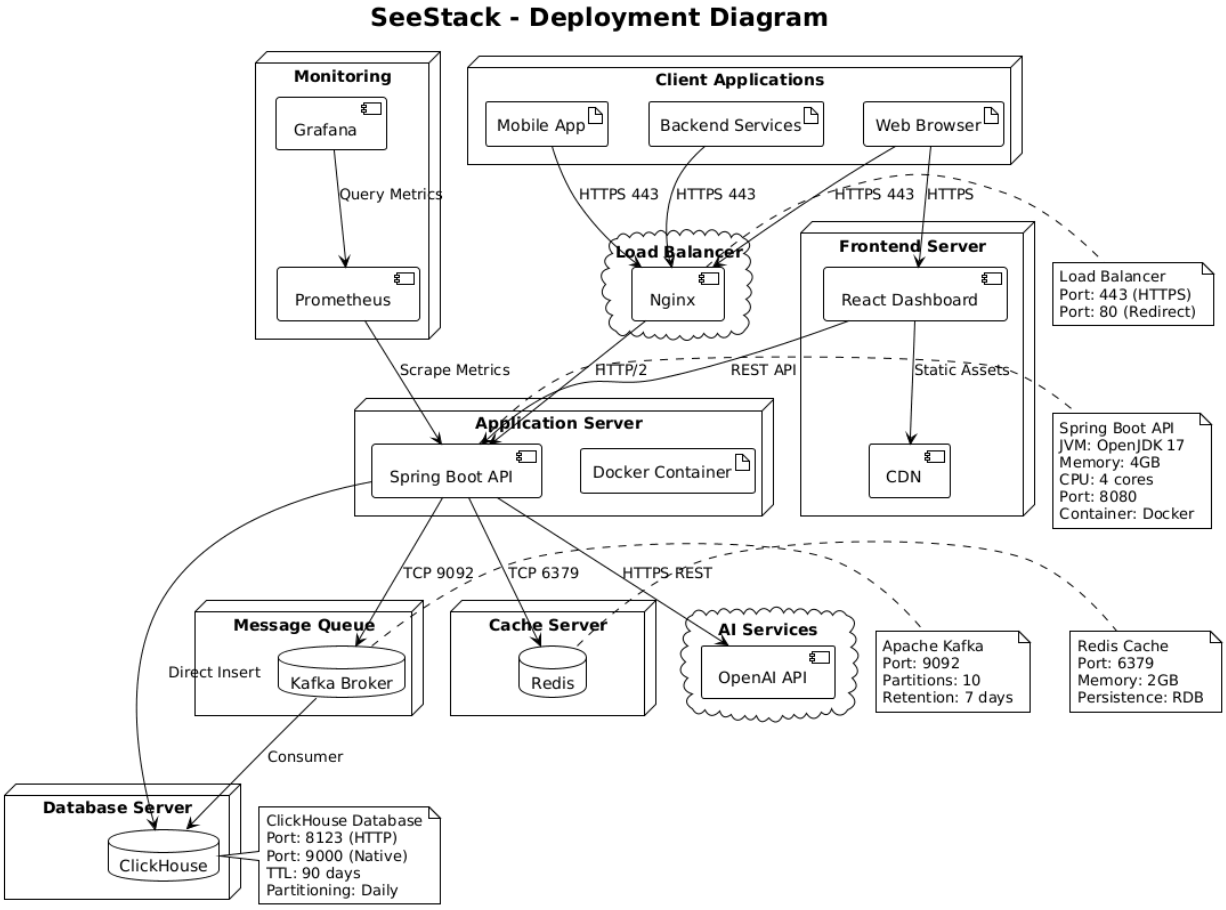


Figure 14: Deployment Diagram Overview

This deployment diagram illustrates how SeeStack is deployed across the underlying infrastructure components.

Client applications, including mobile applications, backend services, and web browsers, send HTTPS traffic to an Nginx load balancer, which forwards REST API requests to a Spring Boot API running inside a Docker container on the application server. The React-based monitoring dashboard is hosted on a dedicated frontend server and delivered via a Content Delivery Network (CDN), while Prometheus scrapes metrics from the Spring Boot API and Grafana visualizes these metrics on observability dashboards.

The application server communicates with Kafka for message queuing, Redis for caching, ClickHouse as the primary analytics database, and the external OpenAI API for AI-assisted analysis. ClickHouse is configured with daily table partitions, both HTTP and native ports, and a 90-day time-to-live (TTL) policy, whereas Kafka and Redis are configured with dedicated ports, partitions, retention policies, and persistence options to support scalability, fault tolerance, and overall system reliability.

CHAPTER 6: SYSTEM IMPLEMENTATION

The SeeStack platform translates the architectural design into an operational software system composed of seven principal elements: the Spring Boot backend, the React-based dashboard, three first-party software development kits (SDKs), a dual-database persistence layer, the core algorithms for error grouping, monitor classification, AI sanitisation, latency analysis, and security risk scoring, and a containerised deployment environment. The delivered system reflects the realised project scope and documents the engineering decisions that shaped its final form.

6.0 Implementation Overview and Key Improvements

The implemented release of SeeStack delivers five end-user capabilities: AI-assisted error analysis, runtime error monitoring, website availability monitoring, automated load testing, and educational security scanning. AI-assisted error analysis is the platform's primary differentiating capability, while the remaining four capabilities provide the supporting observability and diagnostic surface needed for practical use. Continuous host-side performance monitoring with system metric collection formed part of the original vision, but it remains outside the realised scope of the delivered release and is therefore treated as future work.

Beyond feature delivery, the final system also incorporates a number of deliberate engineering decisions intended to improve separation of concerns, security posture, integration simplicity, operational reliability, and deployment efficiency. These decisions affect how the platform stores data, accepts and processes events, manages credentials, exposes documentation, and constrains potentially risky capabilities such as load generation and security inspection. Accordingly, the engineering value of the system lies not only in the functions it provides, but also in the way those functions have been implemented to remain maintainable and deployable under realistic operating conditions.

The delivered capabilities are supported by unit testing, deployment readiness probes, functional acceptance testing, and traceable implementation artefacts. In parallel, the documentation layer has been prepared for future expansion: although the current backend implements the five capabilities described above, the documentation site already includes a scaffold for later modules such as logs, HTTP telemetry, session replay, cron monitoring, and feature flags. Table 6.1 summarises the principal engineering refinements introduced in the delivered system and explains their significance to the final design.

Table 6.1. Principal implementation refinements in the delivered system

No.	Improvement	Impact
1	Dual-database persistence using PostgreSQL and ClickHouse	Separates relational transactional state from high-volume telemetry so that each workload is stored in an engine suited to its write and query pattern.
2	Zero-dependency SDK design for JavaScript, Java, and Python	Reduces client integration friction and limits dependency-chain risk in consuming applications.

3	Deterministic SHA-256 error fingerprinting	Provides stable, language-independent identifiers for recurring failures, improving deduplication and cross-SDK consistency.
4	Apache Kafka 3.8 in KRaft mode	Simplifies message-broker operations by removing the ZooKeeper dependency and reducing infrastructure complexity.
5	One-time ingest-key reveal pattern	Improves secret-handling practice by exposing the raw credential only once and storing only a hashed representation thereafter.
6	Hand-written HS256 JWT issuer	Keeps token issuance logic small and directly auditable for a constrained use case, while reducing external dependency surface.
7	Multi-stage Docker build for the backend	Produces a smaller runtime image with fewer packages, lowering attack surface and improving deployment efficiency.
8	Internal REST and Kafka contract	Decouples synchronous request acceptance from asynchronous persistence, improving burst tolerance and scalability.
9	Fumadocs-based documentation scaffold with OpenAPI integration	Improves developer onboarding and maintainability by aligning API documentation with the implemented service contract while preparing for future module growth.
10	Zero-dependency OpenAI integration with pre-dispatch PII sanitisation	Enables AI-assisted analysis while reducing third-party data exposure and avoiding unnecessary SDK dependencies.
11	In-process load runner with hard caps and per-target cooldown	Adds first-party load-generation capability while constraining misuse through bounded limits and enforced cooldown behaviour.
12	Custom port and security-header analyser with educational scope	Provides lightweight visibility into exposed services and missing protective headers without positioning the feature as a full penetration-testing tool.

6.1 Backend Implementation

The backend functions as the coordinating core of the SeeStack platform. It is implemented as a self-contained Spring Boot 3.4.4 application in Java 17, packaged as an executable JAR and exposed over HTTP on port 8080. Its responsibilities span user authentication, project and API-key administration, error-event ingestion, scheduled website availability checks, AI-assisted error analysis, bounded HTTP load testing, and security scanning. The source code is organised under `backend/src/main/java/com/seestack` using a conventional Java project structure.

6.1.1 Module layout

The backend is organised into seven feature-specific modules supported by a cross-cutting shared security package. Each module encapsulates its own controllers, DTOs, services, repositories, and JPA entities, thereby reducing inter-module coupling and improving separation of concerns.

Table 6.2 presents the backend module structure and summarises the principal responsibility assigned to each module. As shown in Table 6.2, the implementation separates authentication, project and key management, error ingestion and retrieval, monitoring, AI analysis, load testing, and security scanning into distinct backend areas, while the shared package centralises the security mechanisms used across the system.

Table 6.2. Backend module structure and responsibilities

Module	Path under com/seestack/modules	Main responsibility
auth	auth/	User registration, login, credential validation, JWT issuance, and token parsing
teams	teams/	User ownership structures, project management, and ingest-key lifecycle management
errors	errors/	API-key-protected error ingestion and authenticated retrieval of grouped issues
monitors	monitors/	CRUD operations for monitored URLs and scheduled availability checks
ai	ai/	AI-assisted error analysis with OpenAI integration, structured response parsing, PII pre-sanitisation, and per-fingerprint caching
loadtest	loadtest/	Bounded HTTP load testing against registered monitors, including thread-pool execution, percentile latency capture, and per-target cooldown enforcement
security	security/	TCP port probing across a fixed service set, HTTP security-header inspection, and additive risk scoring against an educational scope
shared	shared/security/	Security configuration, JWT filtering, API-key filtering, and current-user resolution

The backend exposes a compact set of REST endpoints corresponding to these modules. The interface is intentionally organised around capability-oriented routes so that each major backend function is accessible through a focused and traceable API surface.

Table 6.3 summarises the principal backend REST endpoints and identifies their authentication requirements and operational purpose. See Table 6.3 for the main service routes through which the delivered backend exposes account management, project administration, error ingestion and retrieval, monitoring, AI analysis, load testing, and security scanning.

Table 6.3. Principal backend REST endpoints

Method and path	Authentication	Purpose
POST /api/auth/register	None	Creates a new user account, provisions a default project, and returns an ingest key
POST /api/auth/login	None	Validates credentials and issues a JWT
GET /api/v1/projects	JWT	Returns the projects owned by the authenticated user
POST /api/v1/projects	JWT	Creates a new project and reveals its ingest key once
DELETE /api/v1/projects/{id}	JWT	Deletes a project and its dependent records
POST /ingest/v1/errors	API key	Accepts runtime error events asynchronously
GET /api/v1/errors	JWT	Returns grouped issues with filtering support
GET /api/v1/errors/{fingerprint}	JWT	Returns detailed information for a specific grouped issue
POST /api/v1/monitors	JWT	Registers a monitored URL
GET /api/v1/monitors	JWT	Returns monitor status, configuration, and uptime information
POST /api/v1/errors/{fingerprint}/ai-analysis	JWT	Triggers AI-assisted analysis for a grouped issue and supports forced regeneration
GET /api/v1/errors/{fingerprint}/ai-analysis	JWT	Returns the cached AI analysis for a grouped issue, when one exists
GET /api/v1/errors/ai-analysis/status	JWT	Reports whether the AI analysis feature is currently configured and available
POST /api/v1/load-tests	JWT	Executes a bounded HTTP load test against a registered monitor
GET /api/v1/load-tests	JWT	Returns previous load tests for the project together with cooldown metadata
GET /api/v1/load-tests/{id}	JWT	Returns the metric record for a single completed load test
POST /api/v1/security-scans	JWT	Executes a port and security-header scan against a target host
GET /api/v1/security-scans	JWT	Returns previous security scans for the project with risk classifications
GET /api/v1/security-scans/{id}	JWT	Returns the detailed findings for a single completed scan

6.1.2 Authentication and JSON Web Tokens

Authentication is implemented internally rather than delegated to an external identity provider. This approach reduces operational dependencies while keeping the trust boundary within a

smaller and more auditable code base. User passwords are hashed with Spring Security's `BCryptPasswordEncoder`, and session tokens are issued according to the JSON Web Token standard using the HS256 signing algorithm.

The JWT issuer has been written by hand and depends on no third-party JWT library; only `javax.crypto.Mac` and the Jackson JSON serialiser are required. The relevant excerpt is reproduced below (`backend/src/main/java/com/seestack/modules/auth/service/JwtService.java`):

```
public String issue(UUID userId, String email) {
    long exp = Instant.now().getEpochSecond() + TTL  SECONDS;
    Map<String, Object> header  = Map.of("alg", "HS256", "typ", "JWT");
    Map<String, Object> payload = new HashMap<>();
    payload.put("sub",   userId.toString());
    payload.put("email", email);
    payload.put("exp",   exp);
    String h = b64url(writeJson(header));
    String p = b64url(writeJson(payload));
    String s = b64url(hmacSha256(h + "." + p));
    return h + "." + p + "." + s;
}
```

The `register` operation (`backend/src/main/java/com/seestack/modules/auth/service/AuthService.java`) normalises the submitted email address, asserts its uniqueness, applies the BCrypt hashing function, persists a `UserEntity`, creates a default project name with a single ingest key, and returns both the issued JWT and the raw ingest key. The raw key is returned to the caller exactly once; thereafter, only its prefix and SHA-256 hash are retained.

6.1.3 Project and API-key management

Project records are managed through `/api/v1/projects` and protected by the JWT authentication filter. Ingest keys are generated in the format `ask_live_<random-suffix>`, hashed using SHA-256, and stored only as a hash together with their public prefix. The raw key is disclosed only at creation time, which aligns the implementation with accepted secret-management practice by ensuring that database compromise does not expose recoverable plaintext credentials.

6.1.4 Error ingestion pipeline

The error-ingestion path is designed as the highest-throughput workflow in the system and is therefore implemented asynchronously. The synchronous HTTP endpoint performs only the minimum required work: authenticating the project key, computing an error fingerprint, generating an event identifier, and publishing the event to Kafka. Persistent storage and aggregation are delegated to downstream consumers.

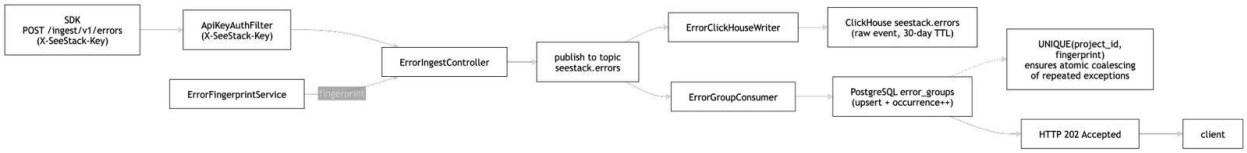


Figure 6.1. End-to-end error-ingestion workflow.

The synchronous controller (`backend/src/main/java/com/seestack/modules/errors/controller/ErrorIngestController.java`) returns the HTTP status code 202 *Accepted* (RFC 9110, section 15.3.3), signalling that the request has been accepted for processing but that processing has not yet completed:

```
@PostMapping
public ResponseEntity<ApiResponse<Map<String, String>>> ingest(
      @Valid @RequestBody ErrorIngestRequest request,
      HttpServletRequest httpRequest) throws JsonProcessingException {

   UUID projectId   = (UUID) httpRequest.getAttribute(ApiKeyAuthFilter.PROJECT_ID_ATTRIBUTE);
   String fingerprint = fingerprintService.generate(
         request.exceptionClass(), request.stackTrace());
   UUID eventId = UUID.randomUUID();

   ErrorKafkaEvent event = new ErrorKafkaEvent(/* ... 14 fields ... */);
   kafkaTemplate.send(TOPIC, fingerprint, objectMapper.writeValueAsString(event));

   return ResponseEntity.status(HttpStatus.ACCEPTED)
         .body(ApiResponse.ok(Map.of("id", eventId.toString())));
}
```

The downstream consumer reads each event from the Kafka topic and performs two parallel writes: a raw insertion into the ClickHouse `seestack.errors` table for time-series analytics, and an *upsert* into the PostgreSQL `error_groups` table for grouped summary display. The `UNIQUE(project_id, fingerprint)` constraint on `error_groups` makes the upsert atomic and is the mechanism by which recurrent exceptions are coalesced into a single dashboard row.

6.1.5 Monitor scheduler

Website availability monitoring is implemented through a scheduled Spring component that executes once per minute. On each cycle, the scheduler retrieves the active monitor configurations,

determines which checks are due, performs real HTTP GET requests, classifies each result, and publishes the resulting verdict to the Kafka topic `seestack.monitor-checks`.

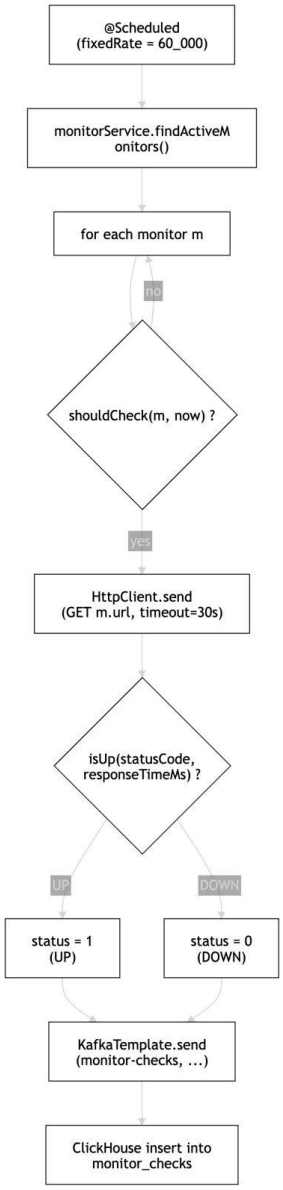


Figure 6.2. Monitor scheduling and status-classification workflow.

This design keeps the scheduling logic inside the backend while preserving asynchronous handling of monitoring data. The classification rule applied by the scheduler is formalised later in the core algorithms section.

The relevant excerpt is reproduced below:

```
@Scheduled(fixedRate = 60_000)
public void runChecks() {
    List<MonitorConfigEntity> monitors = monitorService.findActiveMonitors();
    Instant now = Instant.now();
    for (MonitorConfigEntity m : monitors) {
        if (shouldCheck(m, now)) {
            lastCheckTimes.put(m.getId(), now);
            performCheck(m);
        }
    }
}
```

The classification rule itself is presented in the core-algorithms section later in this chapter as one of the two core algorithms of the system.

6.1.6 AI-assisted error analysis

AI-assisted error analysis is the primary novel capability of the SeeStack platform. It accepts a grouped issue, dispatches a sanitised representation of that issue to a remote large language model, and returns a structured remediation report consisting of a plain-language explanation, a single best-hypothesis root cause, an ordered list of fix steps, a list of prevention bullets, and severity and confidence ratings. The capability is gated by the `OPENAI_API_KEY` environment variable; when no key is configured, a status endpoint reports the feature as unavailable rather than blocking application startup.

The integration relies on `gpt-4o-mini` accessed through a hand-written client over `java.net.http.HttpClient`. No third-party OpenAI SDK is required, which keeps the supply-chain footprint of the AI module identical to the SDK design philosophy described later in section 6.3. The integration source is located at `backend/src/main/java/com/seestack/modules/ai/service/`.

The system prompt requests a strict JSON shape and forbids markdown, code fences, or text outside the JSON object:

```
You are a senior software engineer triaging a production error in a
monitoring system.
Reply with ONLY a JSON object with these exact fields:
  "explanation"  — 1-2 sentences in plain English describing what happened.
  "rootCause"    — your single best hypothesis for the underlying cause.
  "fixSteps"     — an ordered array of 3-6 short, concrete remediation steps.
  "prevention"   — an array of 2-4 short bullet points to prevent recurrence.
  "severity"     — one of "low", "medium", "high".
  "confidence"   — your confidence in this analysis: "low", "medium", or
"high".
```

Each generated analysis is persisted in the PostgreSQL table `ai_error_analyses` and indexed by the unique pair `(project_id, fingerprint)`. A second request for the same grouped issue retrieves the cached analysis without contacting the language model, while an explicit force-refresh path regenerates the analysis and updates the persisted record. Before any payload leaves the backend, the `ErrorDataSanitizer` component pre-scrubs values that resemble bearer tokens, OpenAI-style keys, JSON Web Tokens, and long hexadecimal strings, and redacts metadata keys whose names match a fixed sensitive-key list. The full sanitisation rule is formalised in section 6.6.3.

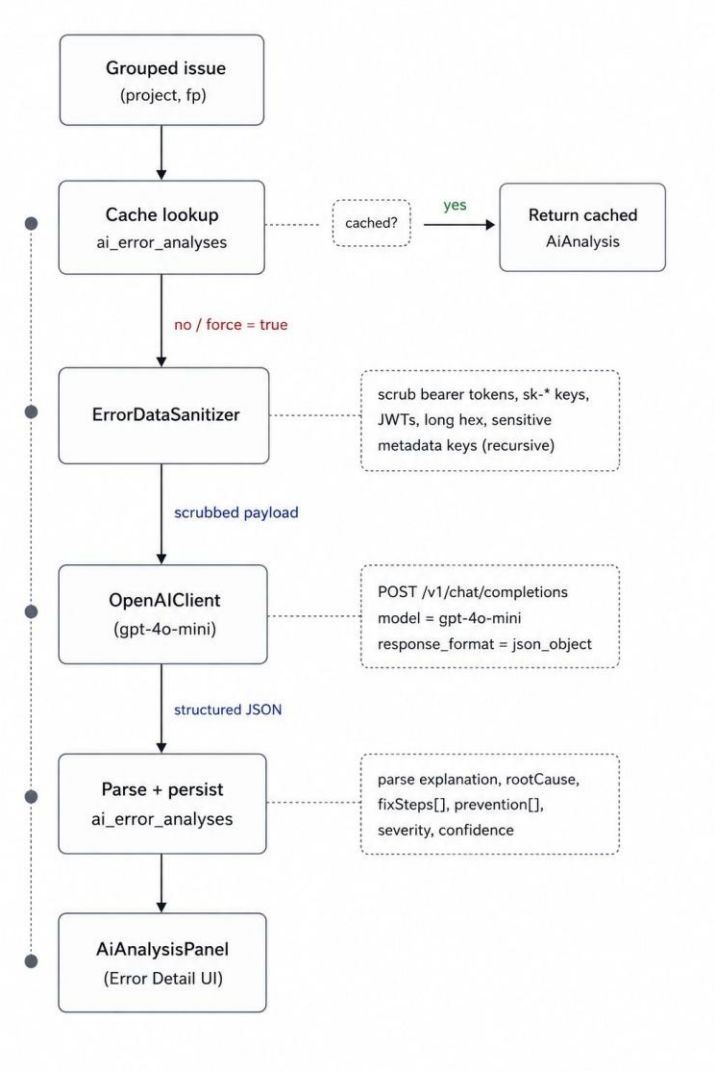


Figure 6.3. AI-assisted error analysis workflow from grouped issue lookup to dashboard rendering.

6.1.7 Automated load testing

Automated load testing exposes a first-party capability for issuing bounded synchronous HTTP requests against a registered monitor target. The runner is implemented in pure Java using `java.net.http.HttpClient` together with a `ThreadPoolExecutor`. It does not depend on Locust, k6, or any other external load tool. The source is located at `backend/src/main/java/com/seestack/modules/loadtest/service/`.

Three hard caps protect the runtime: a single test issues at most 100 requests, runs at most 10 concurrent workers, and applies a 5-second per-request timeout. A 60-second cooldown is enforced per `(project_id, target_url)` pair; any submission inside this window is rejected with HTTP 429 together with the remaining cooldown seconds. Successful execution records totals, success and failure counts, average, minimum, maximum, and 95th-percentile latency, and a status-code distribution map. The percentile calculation is formalised in section 6.6.4. Persistence uses the PostgreSQL table `load_tests`.

The 95th percentile is preferred over a mean because tail latency, rather than central tendency, is the metric most relevant to availability monitoring. Even within the bounded 100-request window, the percentile observation provides a more faithful indicator of the slow-request behaviour that would surface as user-visible latency in production.

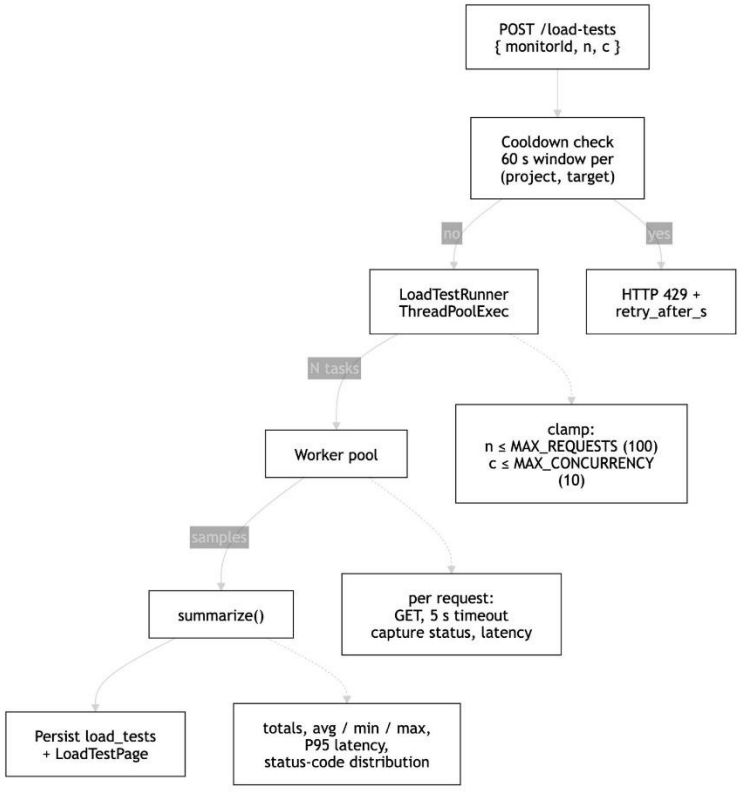


Figure 6.4. Automated load-test workflow from request acceptance through bounded execution to dashboard rendering.

6.1.8 Security scanning

Security scanning combines a TCP port probe with a lightweight HTTP analyser to surface exposed services and missing protective headers. The module is positioned as educational in scope and is not a production penetration-testing or CVE-matching tool; this scope is reflected in the source code, the user-facing dashboard, and the deferred-work statement in chapter 9. The implementation lives at `backend/src/main/java/com/seestack/modules/security/service/`.

The port probe attempts a TCP connection against a fixed service set comprising ports 22 (SSH), 80, 8080 (HTTP), 443, 8443 (HTTPS), 3306 (MySQL), 5432 (PostgreSQL), and 6379 (Redis), each with a 1.5-second connect timeout. Once a web port is detected, the analyser issues a single HTTP or HTTPS GET request and inspects the response for four security headers: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, and `X-Content-Type-Options`. Risk is calculated through an additive model formalised in section 6.6.5 and translated into the discrete levels HIGH (≥60), MEDIUM (30–59), and LOW (<30). All findings are persisted in the PostgreSQL table `security_scans` together with the resolved host, the detected services map, the HTTP probe summary, the security-header presence map, the numeric risk score, and a generated human-readable summary string.

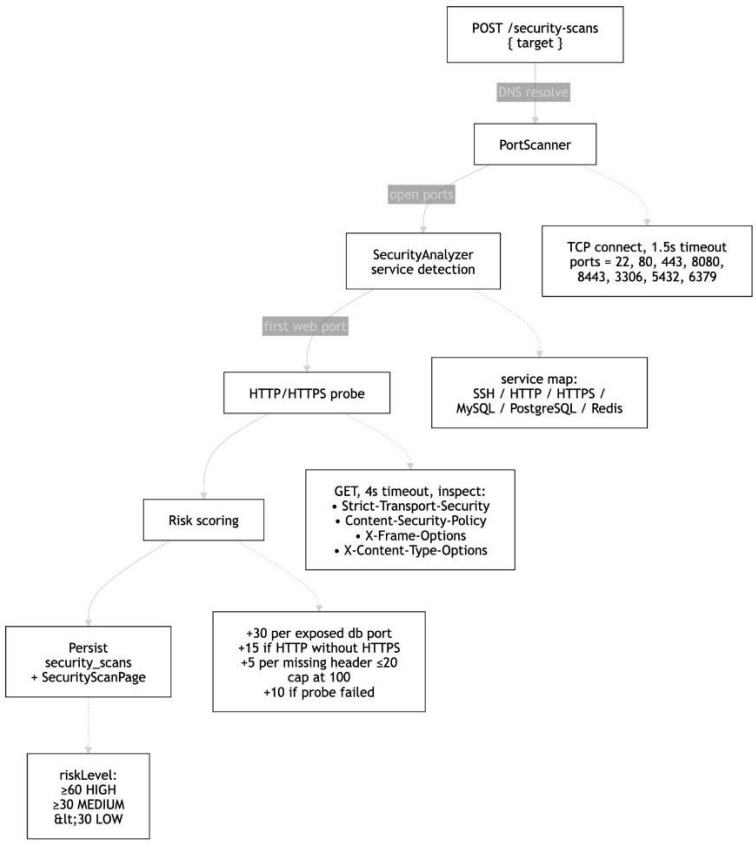


Figure 6.5. Security scan workflow from target submission through port probing and header inspection to additive risk classification.

6.2 Frontend Implementation

The frontend provides the primary user-facing interface to the SeeStack platform. It is implemented as a single-page application using React 19, Vite 5, and TypeScript 5, and is served during development on port 3000. Requests prefixed with /api/ and /ingest/ are proxied to the backend on port 8080, allowing the browser to interact with the system as if it were served from a single origin. The frontend source root is `packages/web/`.

6.2.1 Feature folder layout

The frontend is structured by functional area, with each feature folder containing its own pages, components, hooks, and route definitions. This arrangement localises change and reduces the risk that modifications in one feature will affect unrelated interface components.

```
packages/web/src/features/
├── auth/         LoginPage, RegisterPage, useLogin
├── overview/       OverviewPage, useOverviewData, LatestSecurityScanCard
├── projects/       ProjectsPage, useProjects
├── errors/         ErrorsPage, ErrorDetailPage, useErrors,
│                   AiAnalysisPanel, ErrorInsightsCard
├── monitors/       MonitorsPage, MonitorDetailPage, useMonitors
├── load-test/      LoadTestPage
├── security-scan/ SecurityScanPage
└── sdk-setup/      SdkSetupPage, CodeBlock
```

The router mounts these features into a shared `AppShell`, which provides persistent navigation across the principal sections: Overview, Projects, Errors, Monitors, Load Test, Security Scan, and SDK Setup. Sidebar entries for Load Test and Security Scan are presented with the `Gauge` and ShieldCheck icons respectively, signalling their active rather than navigational role within the dashboard.

6.2.2 Authentication flow

Authentication in the frontend is encapsulated within the `useLogin` hook, which wraps both the login and registration endpoints. On successful authentication, the issued JWT is stored in browser `localStorage` under the key `seestack_token`, and the Zustand authentication store is populated with the user's identity. In the registration flow, the auto-provisioned default project is immediately selected so that the SDK Setup page can display the newly created ingest key.

6.2.3 One-time API-key reveal behaviour

The `ProjectsPage` component maintains an in-memory structure named `revealedKeys`, indexed by project identifier. Immediately after a project is created, the raw ingest key returned by the backend is inserted into this structure and rendered on the page with a copy-to-clipboard control and a warning that the value will not be shown again. After the user leaves the page, only the safe `ask_live_` prefix remains visible because the backend does not retain the raw key.

6.3 SDK Implementation

The SDK layer provides the integration surface through which client applications submit error events to the platform. Three first-party SDKs are implemented for JavaScript, Java, and Python, and each accepts a project ingest key together with the backend endpoint and exposes a single operation for submitting an exception to `/ingest/v1/errors`. A runnable example application was used during runtime validation to generate the error events later recorded in PostgreSQL.

6.3.1 Design constraints

A primary design objective of the SDK layer is the complete removal of external runtime dependencies. The JavaScript SDK uses the native `fetch` API, the Python SDK uses `urllib.request` from the standard library, and the Java SDK uses `java.net.http.HttpClient`. This design reduces integration effort and eliminates dependency-chain exposure in host applications.

Table 6.4. Characteristics of the first-party SDKs

Property	JavaScript SDK	Java SDK	Python SDK
Source file	`sdks/javascript/seestack-sdk.js`	`sdks/java/SeeStack.java`	`sdks/python/seestack_sdk.py`
HTTP transport	Native `fetch()`	`java.net.http.HttpClient`	`urllib.request`
Minimum runtime	Node.js 18 or modern browser	JDK 11	Python 3.8
Main class	`SeeStack`	`SeeStack`	`SeeStack`
Capture method	`captureException(err, meta)`	`captureException(Throwable)`	`capture_exception(exc, ...)`
External runtime dependencies	None	None	None

6.3.2 JavaScript SDK

The JavaScript SDK exposes a `SeeStack` class (`sdks/javascript/seestack-sdk.js`) the constructor of which validates the ingest key, normalises the endpoint, and stores the environment identifier:

```
const { SeeStack } = require('./seestack-sdk')

const seestack = new SeeStack({
  apiKey: 'ask_live_...',
  endpoint: 'http://localhost:8080',
  environment: 'production',
})

await seestack.captureException(err, { user: { id: 'u_42' } })
```

Internally, the call splits the stack trace into trimmed frame strings, constructs the JSON payload, and submits it with the `X-SeeStack-Key` header. The method returns a result object of the form `{ ok, status, eventId, raw }` that may be used by the caller for diagnostic logging.

6.3.3 Java SDK

The Java SDK (`sdks/java/SeeStack.java`) is implemented as a single class. It constructs a long-lived `HttpClient` with a ten-second connection timeout, walks the `Throwable.getStackTrace()` array into a list of frame strings, and serialises the request body using a small hand-rolled JSON encoder so that no Jackson dependency is required:

```
SeeStack seestack = new SeeStack(apiKey, endpoint, "production");
try {
    riskyWork();
} catch (Exception e) {
    int status = seestack.captureException(e);
}
```

6.3.4 Python SDK

The Python SDK (`sdks/python/seestack_sdk.py`) exposes the same operation under the PEP 8-compliant name `capture_exception(exc, *, level, environment, user, metadata)`, returning a tuple `(http_status, event_id)`:

```
from seestack_sdk import SeeStack

seestack = SeeStack(api_key="ask_live_...",
                    endpoint="<http://localhost:8080>",
```

```
                    environment="production")

try:
    risky()
except Exception as e:
    status, event_id = seestack.capture_exception(e)
```

Because each SDK produces an identically structured JSON envelope, the `ErrorFingerprintService` on the backend computes the same fingerprint for the same logical exception irrespective of the source language. This property is what permits the upsert-by-fingerprint pattern in `error_groups` to coalesce events from heterogeneous client applications.

6.4 Documentation Site

The documentation site serves as the public developer-facing reference for integrating engineers. Its purpose is to enable adoption of the SDKs and ingest endpoints without requiring direct access to the source repository. The site is implemented as a standalone Next.js application located under `docs/`, with its own package manifest, lock file, and deployment pipeline. It is statically rendered and does not depend on runtime calls to the backend, which simplifies hosting and removes cross-origin concerns.

6.4.1 Technology stack

The site is built on Fumadocs, which combines MDX-based authoring with a production-oriented reading experience.

Table 6.5. Documentation site technology stack

Layer	Technology	Version
Application framework	Next.js (App Router, standalone output)	16.2.2
UI runtime	React / React DOM	19.2.4
Documentation framework	Fumadocs core / UI / MDX / OpenAPI	16.7.10 / 14.2.11 / 10.6.6
Styling	Tailwind CSS / PostCSS	4.2.2 / 8.5.8
Type system	TypeScript (strict mode)	6.0.2
Search	`@orama/orama`	3.1.18
Syntax highlighting	Shiki	4.0.2
Package manager	Bun	1.x
Deployment pipeline	Nixpacks	Node.js + Bun

6.4.2 Content organisation and routing

Content is authored in MDX and divided into two principal collections: a narrative SDK documentation set and an API reference generated from an OpenAPI 3.1 specification. The Fumadocs source loader provides a hierarchical page tree, slug-based page lookup, and supporting utilities such as Open Graph image generation and markdown export. Navigation is rendered through the `DocsLayout` component and a catch-all route structure that enables deep linking to any page at build time.

6.4.3 Developer-experience features

The documentation site includes several features intended to improve developer usability beyond plain markdown delivery.

Table 6.6. Developer-experience features of the documentation site

Feature	Implementation mechanism
Multi-language code tabs	Fumadocs `Tabs` component with persisted language selection
Copy-to-clipboard on code blocks	Built-in `MarkdownCopyButton`
Client-side full-text search	Orama static index generated at build time
Dark mode	Fumadocs `RootProvider` with Tailwind dark-mode utilities
AI-oriented documentation endpoints	`/llms.txt` and `/llms-full.txt` route handlers
Dynamic Open Graph image generation	`docs/app/og/docs/[...slug]/route.tsx`
Markdown export per page	`getPageMarkdownUrl()` helper

The `/llms.txt` and `/llms-full.txt` endpoints are especially significant because they expose the documentation corpus in a format suitable for ingestion by large language model assistants. This feature allows integrating developers to query or reuse the documentation through AI tooling more efficiently.

6.4.4 Build, deployment, and API regeneration

All pages are statically generated at build time using Next.js. The API reference is regenerated on demand from `docs/openapi-sdk.json` through the script `docs/scripts/generate-openapi.mts`, which writes the generated documentation into `./content/docs/api`. Deployment is handled by Nixpacks, which installs dependencies with Bun, builds the site, and launches the production server using the hosting platform's supplied port.

6.5 Database Implementation

The persistence layer uses two complementary database systems to match different workload characteristics. PostgreSQL 16 stores low-volume transactional and relational metadata such as users, projects, API keys, monitor configurations, and grouped issue summaries. ClickHouse 24 stores high-volume time-series data such as raw error events and monitor-check history. This design follows a polyglot persistence approach in which storage technology is selected according to workload requirements rather than standardised across dissimilar data types.

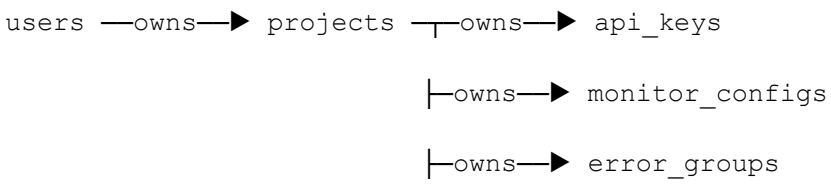
6.5.1 PostgreSQL schema

Schema evolution is managed by Flyway during backend startup through four sequenced migrations: `V1__schema.sql` defines the original six-table relational schema after numerous exploratory migrations were collapsed into a single canonical baseline; `V2__security_scans.sql` adds the `security_scans` table; `V3__load_tests.sql` adds the `load_tests` table; and `V4__security_scan_analysis.sql` extends `security_scans` with risk-analysis columns and introduces the `ai_error_analyses` table for cached AI responses. The four-migration arrangement preserves a reproducible base schema while keeping each subsequent capability addition auditable.

```
CREATE TABLE users (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  name          VARCHAR(255),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE projects (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name       VARCHAR(255) NOT NULL,
  slug       VARCHAR(100) NOT NULL,
  platform   VARCHAR(100),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, slug)
);
```

The relational design is intentionally narrow and centres on ownership by project.



```
                    ├─owns──▶ ai_error_analyses

                    ├─owns──▶ load_tests

                    └─owns──▶ security_scans
```

The `error_groups` table includes a unique constraint on `(project_id, fingerprint)`, which enables atomic insert-or-update behaviour during ingestion. Child entities cascade on deletion, so removing a project also removes its associated keys, monitors, grouped issues, AI analyses, load tests, and security scans while preserving referential integrity.

Capability-specific tables follow the same project-ownership pattern:

```
CREATE TABLE ai_error_analyses (
  id                          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id                  UUID NOT NULL REFERENCES projects(id) ON DELETE
CASCADE,
  fingerprint                 VARCHAR(64) NOT NULL,
  payload                     TEXT NOT NULL,
  model                       VARCHAR(100) NOT NULL,
  occurrences_at_generation   BIGINT NOT NULL,
  created_at                  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (project_id, fingerprint)
);

CREATE TABLE load_tests (
  id                        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id                UUID NOT NULL REFERENCES projects(id) ON DELETE
CASCADE,
  monitor_id                UUID REFERENCES monitor_configs(id) ON DELETE SET
NULL,
  target_url                TEXT NOT NULL,
  status                    VARCHAR(20) NOT NULL,
  total_requests            INTEGER, successful_requests INTEGER, failed_requests
INTEGER,
  avg_response_time_ms      DOUBLE PRECISION,
  min_response_time_ms      INTEGER, max_response_time_ms INTEGER,
p95_response_time_ms INTEGER,
  status_code_distribution TEXT,
  created_at                TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at              TIMESTAMPTZ
);
```

```
CREATE TABLE security_scans (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id          UUID REFERENCES projects(id) ON DELETE CASCADE,
  target              TEXT NOT NULL,
  resolved_host       TEXT,
  scanned_ports       TEXT, open_ports TEXT, closed_ports TEXT,
  detected_services   TEXT, http_info TEXT, security_headers TEXT,
  risk_score          INTEGER, risk_level VARCHAR(10),
  summary             TEXT,
  status              VARCHAR(20) NOT NULL,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at        TIMESTAMPTZ
);
```

The ai_error_analyses unique constraint on (project_id, fingerprint) mirrors the upsert pattern used in error_groups, so a regenerated analysis replaces rather than duplicates the cached record. The load_tests table records all metrics required by section 6.6.4, and the security_scans table accommodates both the original port-scan output and the analysis columns.

6.5.2 ClickHouse schema

The ClickHouse schema is created from `backend/src/main/resources/clickhouse/init.sql` during backend startup through the `ClickHouseSchemaInitializer` bean. This guarantees that the required tables exist before Kafka consumers begin processing events.

```
CREATE TABLE IF NOT EXISTS seestack.errors (
    project_id     UUID,        fingerprint String,
    exceptionClass String,      message     String,
    stack_trace    String,      level       LowCardinality(String),
    environment    LowCardinality(String),  release String,
    user_id String, user_email String, user_ip String,
    metadata String, timestamp DateTime64(3)
) ENGINE = MergeTree()
ORDER BY (project_id, timestamp)
TTL toDateTime(timestamp) + INTERVAL 30 DAY;

CREATE TABLE IF NOT EXISTS seestack.monitor_checks (
    monitor_id UUID, project_id UUID,
    status UInt8, response_time_ms UInt32,
    status_code UInt16, timestamp DateTime64(3)
) ENGINE = MergeTree()
```

```
ORDER BY (monitor_id, timestamp)
TTL toDateTime(timestamp) + INTERVAL 90 DAY;
```

The `seestack.errors` table applies a thirty-day TTL to constrain storage growth while preserving a useful investigation window for repeated incidents. The use of `LowCardinality(String)` for fields such as `level` and `environment` improves storage efficiency because those columns contain only a small set of repeated values.

6.6 Core Algorithms

Two algorithms are central to the behaviour of the delivered system: error fingerprint generation and monitor status classification. Both are implemented as static helpers and tested independently through JUnit 5, which allows verification without booting the full Spring application context.

6.6.1 Error fingerprinting algorithm

The error fingerprint is a deterministic 64-character hexadecimal digest derived from the SHA-256 hash of a canonical input string. That input consists of the exception class name concatenated with up to the first five non-framework stack frames. Frames associated with the Java standard library, Spring, Apache utilities, and Netty are filtered out before hashing to reduce framework noise in the grouping result.

```
input  ← exceptionClass
input  ← input + "|" + join("|", filter(stackFrames, ¬ internal).take(5))
output ← lowercase_hex( SHA-256( UTF-8(input) ) )
```

If no application frames are available, the fingerprint falls back to `SHA-256(exceptionClass)`. Because the algorithm is deterministic and language-agnostic, identical logical exceptions reported by different SDKs produce the same fingerprint and are therefore coalesced into the same grouped issue.

6.6.2 Monitor up/down classification algorithm

Each scheduled monitor probe is classified through a single static predicate.

```
public static boolean isUp(int statusCode, int responseTimeMs) {
    return statusCode >= 200 && statusCode < 400 && responseTimeMs < 30_000;
}
```

A monitor is classified as **up** only when the HTTP status code is within the 2xx or 3xx range and the response time is strictly less than 30,000 milliseconds. Status code 399 is therefore treated as up, whereas status 400, status 0, or a response time of exactly 30,000 milliseconds are classified as down.

6.6.3 PII sanitisation algorithm

PII sanitisation is applied to every value that the AI module forwards to the OpenAI endpoint. The sanitiser combines two mechanisms: a value-level regular expression filter that removes obvious credential-shaped strings, and a key-level redaction filter that removes values whose keys appear in a known sensitive-key list.

```
for each metadata entry (key, value):
    if lowercase(key) ∈ BLOCKED_KEYS:
        value ← "[REDACTED]"
    else if value is a string:
        for each pattern in {bearer, sk-key, jwt, longHex}:
            value ← regex_replace(value, pattern, "[REDACTED]")
    else if value is a map:
        recurse into value
```

The regular expressions cover bearer tokens (`Bearer\\s+[a-z0-9._\\-]+`), OpenAI-style API keys (`sk-[a-zA-Z0-9_\\-]{20,}`), JSON Web Tokens (`eyJ[…]\\.[…]\\.[…]`), and long hexadecimal strings (`[a-f0-9]{40,}`). The blocked-key list covers `password`, `passwd`, `pwd`, `token`, `access_token`, `refresh_token`, `api_key`, `apikey`, `secret`, `authorization`, `auth`, `cookie`, `session`, `session_id`, `x-api-key`, `x-auth-token`, `private_key`, and `client_secret`. The exception message and each of the top ten stack frames are passed through the same value-level filter before being added to the LLM payload.

The implementation is reproduced below from `ErrorDataSanitizer.java`:

```java
public String scrubString(String input) {
    if (input == null || input.isEmpty()) return input;
    String out = input;
    for (Pattern p : VALUE_PATTERNS) {
        out = p.matcher(out).replaceAll("[REDACTED]");
    }
    return out;
}

public Map<String, Object> scrubMetadata(Map<String, Object> metadata) {
    if (metadata == null) return Map.of();
    Map<String, Object> copy = new HashMap<>();
    for (Map.Entry<String, Object> e : metadata.entrySet()) {
        String key = e.getKey();
        if (key == null) continue;
        if (BLOCKED_KEYS.contains(key.toLowerCase())) {
            copy.put(key, "[REDACTED]");
            continue;
        }
        Object val = e.getValue();
```

```
        if (val instanceof String s) {
            copy.put(key, scrubString(s));
        } else if (val instanceof Map<?, ?> m) {
            copy.put(key, scrubMetadata((Map<String, Object>) m));
        } else {
            copy.put(key, val);
        }
    }
    return copy;
}
```

6.6.4 P95 latency calculation algorithm

The load runner reports the 95th-percentile response time alongside the conventional totals, success rate, and average latency. The algorithm is a closed-form rank computation over the collected sample.

```
sortedTimes ← sort_ascending(samples)
N           ← length(sortedTimes)
p95         ← sortedTimes[ min( N − 1, ceil(N × 0.95) − 1 ) ]
```

The `min(N − 1, …)` clamp prevents an out-of-bounds access for small samples, and the `ceil(N × 0.95) − 1` index produces the same value as the nearest-rank percentile method commonly used in load-testing literature. Sorting is O(N log N), which is acceptable under the 100-request hard cap of the runner. P95 is preferred over the mean because tail latency is the more relevant indicator of the slow-request behaviour that would surface as user-visible delay in production.

The corresponding extract from `LoadTestRunner.summarize` is reproduced below:

```
Collections.sort(sortedTimes);
int p95 = sortedTimes.isEmpty() ? 0
        : sortedTimes.get(Math.min(
              sortedTimes.size() - 1,
              (int) Math.ceil(sortedTimes.size() * 0.95) - 1));
double avg = total > 0 ? (sum * 1.0 / total) : 0;
return new Result(total, success, failed, avg, min, max, p95, dist);
```

6.6.5 Security risk-scoring algorithm

The security scanner combines its port-probe and HTTP-probe outputs into a single integer score using an additive model that is transparent to the user. The score is then translated into a discrete risk level.

```
score ← 0
```

```
for each open port p:
    if p ∈ {3306, 5432, 6379}: score ← score + 30
if hasHttp ∧ ¬hasHttps: score ← score + 15
if probe ≠ null:
    missing ← count(headers where present = false)
    if missing > 0: score ← score + min(20, missing × 5)
    if probe.statusCode = 0: score ← score + 10
score ← min(100, score)
riskLevel ← score ≥ 60 ? "HIGH"
          : score ≥ 30 ? "MEDIUM"
          :              "LOW"
```

The model rewards exposed database ports heavily (+30 each), penalises plaintext HTTP without an HTTPS counterpart (+15), accumulates +5 per missing security header up to a +20 cap, and adds +10 when the HTTP probe fails to reach the target. The 100-point cap and the 60/30 thresholds keep the level transitions interpretable: a single exposed database port is sufficient to register a MEDIUM finding, two are sufficient to register HIGH, and the absence of all four checked security headers under HTTP-only exposure produces HIGH on its own.

The corresponding extract from `SecurityAnalyzer.analyze` is reproduced below:

```java
int score = 0;
for (int p : openPorts) {
    if (DATABASE_PORTS.contains(p)) score += 30;
}
if (httpsPort == null && httpPort != null) score += 15;
if (probe != null) {
    long missing = headers.values().stream().filter(v -> !v).count();
    if (missing > 0) score += (int) Math.min(20, missing * 5);
    if (probe.statusCode == 0) score += 10;
}
score = Math.min(100, score);
String riskLevel = score >= 60 ? "HIGH"
                 : score >= 30 ? "MEDIUM"
                 :               "LOW";
```

Table 6.7. Unit-test coverage for core algorithms

Test suite	Location	Number of cases	Coverage focus
`ErrorFingerprintServiceTest`	`backend/src/test/../errors/service/`	9	Determinism, framework-frame filtering, five-frame

			cap, empty-frame fallback, class-level distinction
MonitorSchedulerTest	backend/src/test/../monitors/service/	10	Success cases, redirects, client and server errors, slow responses, and connection failures
ApiKeyGeneratorServiceTest	backend/src/test/../teams/service/	7	Prefix correctness, key length, uniqueness, deterministic SHA-256 hashing, and prefix extraction
ErrorInsightsServiceTest	backend/src/test/../errors/service/	8	Dashboard insights aggregation under fixed clock and time-window filtering
ErrorDataSanitizerTest	backend/src/test/../ai/service/	5	Bearer, sk-, JWT, long-hex regex redaction; key-name redaction; nested-map recursion
SecurityAnalyzerTest	backend/src/test/../security/service/	5	Service detection, database-port weighting, HTTP-without-HTTPS penalty, level thresholds, score capping
LoadTestRunnerLimitsTest	backend/src/test/../loadtest/service/	1	Hard-cap public constants alignment with controller, UI, and documentation

The unit tests demonstrate that both algorithms have been verified at their functional boundaries and typical operating conditions.

6.7 Deployment and Infrastructure

The runtime topology of SeeStack is defined primarily by the backend Dockerfile and the orchestration file infra/docker/docker-compose.yml. The deployment environment is designed to allow a developer to start the full stack quickly and verify a working registration flow through the frontend with minimal manual configuration.

6.7.1 Containerised infrastructure topology

The infrastructure layer consists of four main supporting services running as Docker containers on the shared bridge network seestack-net, together with the backend service itself. Each service

uses persistent volumes where needed and exposes a health check so that dependency ordering can be controlled through Docker Compose.

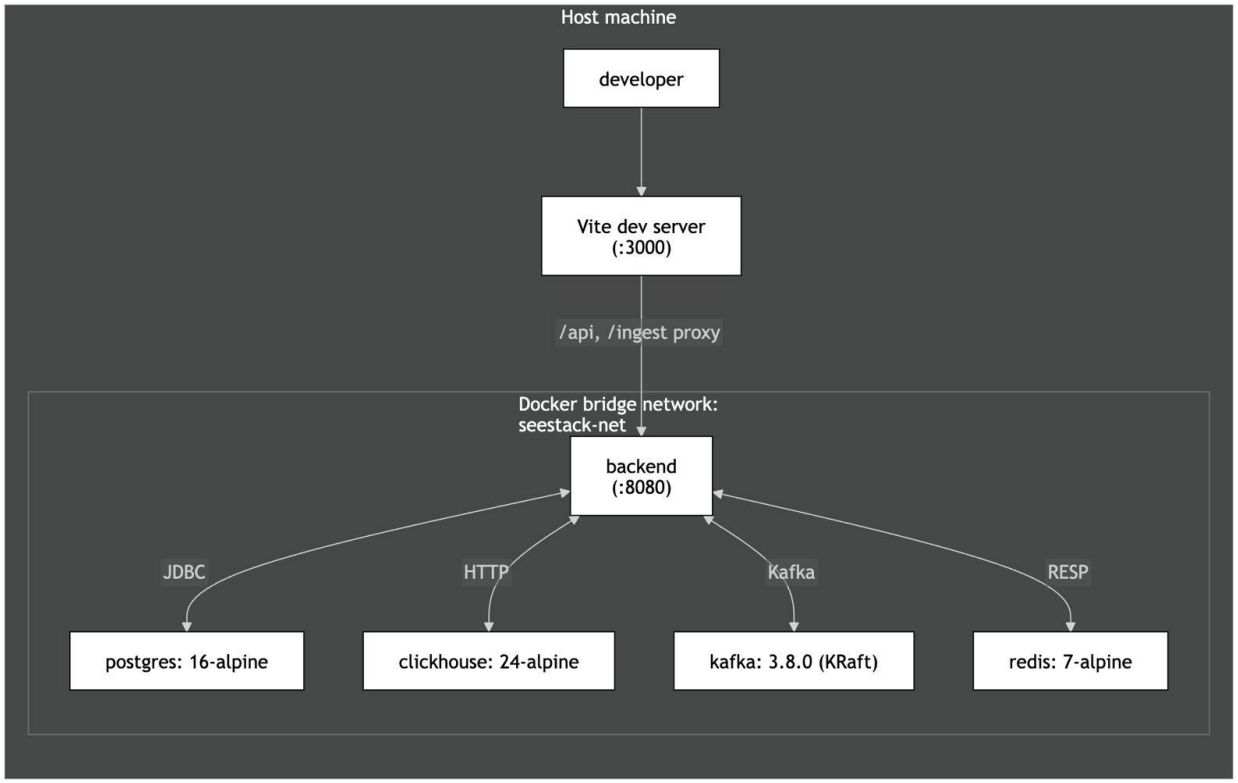


Figure 6.19. Deployment topology of the containerised SeeStack runtime.

Table 6.8. Infrastructure services and operational probes

Service	Image	Host port(s)	Health-check mechanism
PostgreSQL	`postgres:16-alpine`	5432	`pg_isready`
ClickHouse	`clickhouse/clickhouse-server:24-alpine`	8123, 9000	HTTP `GET /ping`
Kafka (KRaft)	`apache/kafka:3.8.0`	9092	`kafka-broker-api-versions.sh`
Redis	`redis:7-alpine`	6379	`redis-cli ping`
Backend	Built from `backend/Dockerfile`	8080	HTTP `GET /actuator/health`

Kafka is operated in KRaft mode, which eliminates the need for a separate ZooKeeper container and therefore reduces the footprint of the messaging infrastructure.

6.7.2 Backend container image

The backend image is produced through a two-stage Docker build. The first stage compiles the application using the JDK, and the second stage copies only the generated executable JAR into a slim JRE image. This separation excludes compilers and build tooling from production, thereby reducing image size and attack surface.

6.7.3 Configuration

The backend reads twelve environment variables at startup, with development defaults supplied in `application.properties`. In non-development deployment contexts, two variables are mandatory: `POSTGRES_PASSWORD` and `SEESTACK_JWT_SECRET`, with the latter requiring a minimum length of thirty-two characters. A third variable, `OPENAI_API_KEY`, is required to enable the AI-assisted analysis feature; its absence does not block startup but causes the AI status endpoint to report the feature as unavailable.

Table 6.9. Backend runtime configuration variables

Variable	Purpose	Requirement status
POSTGRES_HOST, POSTGRES_PORT	PostgreSQL connection location	Defaulted
POSTGRES_DB, POSTGRES_USER	PostgreSQL database and username	Defaulted
POSTGRES_PASSWORD	PostgreSQL password	Required
REDIS_HOST, REDIS_PORT, REDIS_PASSWORD	Redis connection settings	Defaulted
KAFKA_BOOTSTRAP_SERVERS	Kafka broker address	Defaulted
CLICKHOUSE_HOST, CLICKHOUSE_PORT	ClickHouse HTTP endpoint location	Defaulted
CLICKHOUSE_DB, CLICKHOUSE_USER	ClickHouse database and username	Defaulted
SEESTACK_JWT_SECRET	HS256 signing secret	Required
OPENAI_API_KEY	OpenAI API credential gating the AI-assisted error analysis feature	Required to enable the AI feature; absence does not block startup
OPENAI_MODEL	OpenAI model identifier used by the AI module	Defaulted to gpt-4o-mini

6.7.4 Local execution sequence

Local startup of the platform consists of four sequential commands.

```
docker compose -f infra/docker/docker-compose.yml up -d
cd backend && ./gradlew bootJar -x test
java -jar build/libs/seestack-backend-0.0.1-SNAPSHOT.jar
cd ../packages/web && pnpm install && pnpm dev
```

After the frontend is started, a user can navigate to `http://localhost:3000/register`, create an account, and observe the automatically provisioned default project together with its single-use ingest key. This sequence completes the path from infrastructure startup to an observable end-user workflow.

CHAPTER 7: TESTING & EVALUATION

The delivered SeeStack implementation was evaluated against the functional requirements and the realised project scope through a structured testing regime designed to verify correctness, integration reliability, deployment readiness, and user-facing behaviour. The evaluation covers the five delivered capabilities of the platform: AI-assisted error analysis, runtime error monitoring, website availability monitoring, automated load testing, and educational security scanning. Evidence was collected through six complementary layers: automated unit testing, end-to-end pipeline validation, browser-based integration verification, deployment-readiness probing, documentation-site validation, and functional acceptance testing.

7.1 Testing Strategy

A layered testing strategy was adopted to ensure that the implemented system was evaluated from the level of isolated logic to complete user-visible workflows. This structure follows the testing-pyramid principle by placing fast and deterministic tests at the base, narrower integration probes in the middle, and manual acceptance testing at the top. The approach is particularly appropriate for SeeStack because the delivered system is not a single deployable artifact; the backend application, infrastructure stack, dashboard, and documentation site each expose distinct behaviours that require separate verification.

Lower testing layers establish confidence in algorithms and service readiness before higher-level evidence is interpreted. A failure in unit logic, infrastructure health, or ingestion flow would invalidate conclusions drawn from dashboard-level or acceptance-level observations, so the evaluation proceeds from foundational checks upward. This ordering ensures that later observations are supported by earlier technical evidence rather than treated as isolated demonstrations.

The testing techniques applied across the evaluation, the system layer each technique exercises, the principal artefact associated with it, and the justification for its inclusion are consolidated in Table 7.1.

Technique	Layer exercised	Primary artefact	Justification
Automated unit testing with JUnit 5	Core logic for fingerprint generation, monitor classification, and API-key generation	`backend/src/test/java/com/seestack/modules/**/*Test.java`	Deterministic assertions isolate the correctness of the three core algorithms whose regression would silently corrupt error grouping, monitor status evaluation, or credential security.
End-to-end runnable probe	Full ingest path from SDK to dashboard	`sdks/examples/example-app.js`	A single runnable example verifies that the asynchronous contract across SDK, REST endpoint, Kafka, ClickHouse, PostgreSQL, and dashboard

			remains intact under realistic execution.
Manual browser integration probe	Frontend-to-backend interaction on the delivered dashboard surface	`TEST_REPORT_INTEGRATION.md`	Browser execution reveals behaviours such as persisted authentication, client-side state hydration, and network request sequencing that are not faithfully reproduced by unit-level harnesses.
Container health-check probe	Infrastructure readiness	`infra/docker/docker-compose.yml`	Health checks gate dependent service startup and therefore provide the minimum readiness evidence required before any higher-layer evaluation is meaningful.
Spring Boot Actuator health probe	Backend service readiness and liveness	`backend/src/main/resources/application.properties`	A single health endpoint consolidates the status of the database, Kafka broker, and Redis cache, making it the canonical readiness signal for backend operation.
Documentation-site validation	Public developer-experience surface	`docs/package.json`, `docs/scripts/generate-openapi.mts`, `docs/app/llms.txt/route.ts`	The documentation site is the entry point for integrators, so stale API references, broken builds, or missing AI-oriented export surfaces would degrade adoption even if the backend remained correct.
Manual functional acceptance testing	Delivered functional requirements	Functional test-case tables in this chapter	The project brief requires structured functional test cases, and the resulting cases trace delivered workflows directly to the use-case and implementation scope.

7.2 Automated Unit Tests

Seven JUnit 5 test suites were maintained alongside the production backend code to evaluate the correctness of the platform's most critical deterministic components. The suites were intentionally written without requiring a full Spring application context so that execution remained fast, repeatable, and independent of infrastructure state. Collectively, they contain forty-five test cases spanning fingerprint generation, monitor status classification, API-key generation, dashboard insights aggregation, PII sanitisation prior to AI dispatch, additive security risk scoring, and load-runner hard-cap enforcement.

The fingerprinting suite verifies that logically identical exceptions produce identical 64-character hexadecimal fingerprints regardless of runtime source, while distinct exception classes or distinct frame sequences generate different identifiers. It also verifies the handling of null or empty frame arrays, filtering of internal framework frames, and the cap applied to the top application frames that participate in the digest. These assertions protect the integrity of grouped error reporting by preventing incorrect fragmentation or accidental coalescence of issues.

The monitor-classification suite verifies the exact behaviour of the `MonitorScheduler.isUp(int, int)` predicate across the full decision boundary of interest. It confirms that 2xx and 3xx statuses are treated as operational, that 4xx and 5xx responses are treated as down, that `status == 0` is interpreted as connection failure, and that the 30,000 ms timeout boundary is applied precisely. These cases are significant because an incorrect verdict at any boundary would silently distort uptime metrics and dashboard badges.

The API-key suite verifies the security properties required by the one-time key reveal design. It confirms correct environment-specific prefixes, adequate key length, uniqueness across generated keys, deterministic hashing, correct prefix extraction, and proper SHA-256 formatting of persisted hashes. These tests establish that credential storage preserves verification capability without storing raw secrets in the database.

The dashboard insights suite verifies the projection logic that powers occurrence-trend reporting on the overview page. Operating against a fixed clock, it confirms that aggregations over rolling time windows return consistent counts and ordering and that empty windows do not produce spurious entries. The fixed clock removes time-dependent flakiness and isolates the aggregation from infrastructure variability.

The PII sanitiser suite verifies the redaction guarantees applied immediately before any payload leaves the AI module for the OpenAI endpoint. It confirms that bearer tokens, OpenAI-style keys, JWT-shaped strings, and long hexadecimal strings are removed from free-form values, that values keyed under a known sensitive name are replaced regardless of content, and that the sanitiser recurses into nested map structures so that no buried credential reaches the language model.

The security risk-scorer suite verifies the additive scoring rule that translates port and header findings into a numeric risk score and the LOW/MEDIUM/HIGH classification used by the dashboard. It confirms the +30 weight applied to each exposed database port, the +15 penalty for HTTP exposure without a corresponding HTTPS port, the missing-headers contribution capped at +20, and the level transitions at 30 and 60. These boundaries determine which scans escalate to a HIGH classification and therefore carry the strongest signal value to the user.

The load-runner suite verifies the public hard-cap constants exposed by `LoadTestRunner`. The constants are referenced in the controller, the dashboard configuration form, and the developer documentation; alignment of the test, runtime, and user-facing values prevents the runner from being silently relaxed beyond its safe upper bound.

The unit-test inventory and the invariants covered by each suite are consolidated in Table 7.2.

Test suite	File	Cases	Key invariants covered
ErrorFingerprintServiceTest	backend/src/test/java/com/seestack/modules/errors/service/ErrorFingerprintServiceTest.java	9	Determinism, 64-character hexadecimal output, internal-frame filtering, top-frame limit, null and empty fallback handling, and distinction between class-based and frame-based inputs.
MonitorSchedulerTest	backend/src/test/java/com/seestack/modules/monitors/service/MonitorSchedulerTest.java	10	Up/down classification across HTTP status classes, `status == 0` handling, timeout threshold at 30,000 ms, and 399/400 boundary behaviour.
ApiKeyGeneratorServiceTest	backend/src/test/java/com/seestack/modules/teams/service/ApiKeyGeneratorServiceTest.java	7	Prefix correctness, minimum key length, uniqueness, deterministic hashing, SHA-256 digest format, and prefix extraction for safe dashboard rendering.
ErrorInsightsServiceTest	backend/src/test/java/com/seestack/modules/errors/service/ErrorInsightsServiceTest.java	8	Dashboard insights aggregation under fixed clock, time-window filtering, projection consistency, and empty-window handling.
ErrorDataSanitizerTest	backend/src/test/java/com/seestack/modules/ai/service/ErrorDataSanitizerTest.java	5	Bearer-token, `sk-`-key, JWT, and long-hex regex redaction; key-name redaction; nested-map recursion before LLM dispatch.
SecurityAnalyzerTest	backend/src/test/java/com/seestack/modules/security/service/SecurityAnalyzerTest.java	5	Service detection from port map; database-port weighting; HTTP-without-HTTPS penalty; risk-level threshold transitions; score capping at 100.
LoadTestRunnerLimitsTest	backend/src/test/java/com/seestack/modules/loadtest/service/LoadTestRunnerLimitsTest.java	1	Hard-cap public constants alignment with controller, UI, and documentation.
Total	—	45	Coverage is concentrated on the five deterministic algorithms documented in section 6.6 plus the API-key generator and dashboard insights aggregator.

Representative cases from the seven suites, together with the rationale behind each assertion, appear in Table 7.3.

Suite	Representative case	Brief rationale
Fingerprint generation	Same inputs produce the same fingerprint	Ensures that repeated occurrences of the same logical exception collapse into the same grouped issue rather than producing duplicate records.
Fingerprint generation	Different frame sequences produce different fingerprints	Prevents unrelated faults from being merged into a single issue group.
Fingerprint generation	Internal framework frames are ignored	Preserves cross-environment consistency by excluding framework noise from the digest.
Monitor classification	HTTP 200 with fast latency is UP	Confirms the expected positive path for normal availability monitoring.
Monitor classification	HTTP 200 with latency greater than or equal to 30,000 ms is DOWN	Protects uptime reporting from treating timeouts as successful checks.
Monitor classification	HTTP 400 is DOWN while HTTP 399 is UP	Verifies the exact boundary between acceptable and unacceptable response classes.
API-key generation	Production keys begin with `ask_live_`	Encodes environment provenance in the credential itself for safe operational recognition.
API-key generation	Two generated keys are unique	Demonstrates that the generator draws from a sufficiently high-entropy source.
API-key generation	Hash output is deterministic 64-character lowercase hexadecimal	Ensures secure storage and reliable later verification without retaining the raw key.
PII sanitisation	Bearer tokens are redacted before LLM dispatch	Confirms that credential-shaped strings are removed prior to any third-party language-model call, which is the privacy-by-design property foregrounded by the AI module.
PII sanitisation	Sensitive metadata keys are replaced with `[REDACTED]` regardless of value	Establishes that key-name redaction operates independently of value content and recurses into nested maps.
Security risk scoring	An exposed PostgreSQL port produces a HIGH classification under the additive model	Verifies the dominant +30 contribution that anchors the risk model and surfaces the most consequential exposure findings.
Security risk scoring	All four security headers missing without HTTPS support produces HIGH	Confirms the cumulative effect of the HTTP-without-HTTPS penalty and the missing-headers cap at the upper end of the score range.
Load-runner limits	`MAX_REQUESTS = 100` and `MAX_CONCURRENCY = 10` are public constants	Aligns the runner, controller, dashboard, and documentation around a single safe upper bound and prevents silent relaxation of the cap.

7.3 End-to-End and Integration Validation

Unit tests establish algorithmic correctness, but they do not verify the agreement of the collaborating services that form the ingest pipeline. To address this gap, the delivered system includes a runnable example at `sdks/examples/example-app.js` that dispatches three real exceptions through the live platform. The script emits two identical `TypeError` events and one distinct `SyntaxError`, thereby providing a controlled scenario in which both successful ingestion and fingerprint-based coalescence can be observed.

The intended end-to-end behaviour is that each error is accepted by the ingest endpoint, written to Kafka, recorded in ClickHouse as raw events, and summarised in PostgreSQL as grouped issues. Because the first two exceptions share the same fingerprint, they should increment the same `error_groups` row to an occurrence count of two, while the third should create a second grouped row with a count of one. This probe provides the strongest operational evidence that the full asynchronous contract remains intact across the delivered architecture.

The end-to-end acceptance checklist used to validate the ingest pipeline is given in Table 7.4.

#	Observable	Verification method	Expected result
1	SDK constructor initialises successfully	Terminal output or runtime observation	A `SeeStack` instance is created against the supplied endpoint without throwing an exception.
2	First `POST /ingest/v1/errors` request	SDK return value	The request returns `{ ok: true, status: 202, eventId: <uuid> }`.
3	Kafka topic receives first two repeated events	Kafka consumer inspection	Two messages appear with the same record key corresponding to the shared fingerprint.
4	ClickHouse raw event storage	`clickhouse-client` query	Two raw rows are present for the repeated exception with identical fingerprints.
5	PostgreSQL grouped-error state after repeated exception	`psql` query	A single grouped-error row exists with `occurrences = 2`.
6	Distinct third exception submission	SDK return value	The request again returns `{ ok: true, status: 202, eventId: <uuid> }`.
7	PostgreSQL grouped-error state after all three events	`psql` query	Two grouped rows exist in total, with counts of `2` and `1`.
8	Dashboard rendering at `/errors`	Browser observation	The grouped issues list displays the two error groups with the expected exception classes and occurrence counts.

Browser-level integration was also examined through a manual probe documented. This probe was necessary because frontend behaviours such as JWT persistence, client-side store hydration, paginated API response handling, and one-time key reveal cannot be evaluated reliably from

Node-based unit or backend-only tests. The most relevant observations for the delivered scope concern the `/projects` page, which sits directly on the path required to obtain and use an ingest key.

The principal findings of the `/projects` integration probe are recorded in Table 7.5.

#	Observation	Verified outcome
1	`GET /api/v1/organizations` and `GET /api/v1/projects?orgId=…` execute after hard reload	Both requests return HTTP 200 and the projects page renders correctly.
2	The paginated envelope `{items, pagination}` is unwrapped before rendering	Project cards are displayed rather than an empty state.
3	The raw ingest key is shown only immediately after project creation	Subsequent renders retain only the safe prefix and an obfuscated suffix.

The integration probe identified a defect in authentication-store hydration, caused by the `useAuth()` hook not being mounted in `AppShell`. This defect prevented organisation-dependent queries from executing correctly on hard reload because the query `enabled` conditions remained false. Its detection demonstrates the necessity of real-browser verification alongside lower-level tests, since neither isolated unit tests nor the runnable ingest example exercised this browser-specific path.

7.4 Deployment and Documentation-Site Validation

Infrastructure readiness was evaluated through container-level health checks and a backend liveness endpoint so that all higher-level tests rested on a verified operational baseline. The project declares readiness only when PostgreSQL, ClickHouse, Kafka, and Redis each report healthy status through Docker Compose probes and the backend reports `status: UP` through the Spring Boot Actuator health endpoint. These probes function as a gate because a failure in any dependent service would invalidate subsequent evidence gathered from the application layer.

The deployment-readiness probes used in the evaluation appear in Table 7.6.

Service	Probe	Success criterion	Cadence
PostgreSQL	`pg_isready -U ${POSTGRES_USER}`	Exit code `0`	Every 10 seconds
ClickHouse	HTTP `GET /ping` on port 8123	HTTP 200 with body `Ok.`	Every 10 seconds
Kafka	`kafka-broker-api-versions.sh --bootstrap-server localhost:9092`	Exit code `0`	Every 15 seconds
Redis	`redis-cli ping`	`PONG` response	Every 10 seconds

Backend	HTTP `GET /actuator/health` on port 8080	`{"status":"UP"}` with nested `db`, `redis`, and `kafka` components also `UP`	On demand

The documentation site was validated separately because it is a distinct Next.js deployable with its own build pipeline and user-facing responsibilities. Its correctness matters directly to the practical usability of the system because integrators rely on it to understand the SDKs, endpoints, and developer workflow. Validation therefore covered build integrity, OpenAPI-based reference regeneration, and developer-experience features exposed by the site.

The documentation-site build probes are recorded in Table 7.7.

#	Command	Success criterion	Supporting artefact
1	`bun install --frozen-lockfile`	Exit code `0` and no lock-file modification	`docs/bun.lock`, `docs/nixpacks.toml`
2	`bun run types:check`	Exit code `0` from MDX generation, Next type generation, and TypeScript checking	`docs/package.json`
3	`bun run build`	Exit code `0` and successful production build of the standalone site	`docs/next.config.mjs`

OpenAPI regeneration was evaluated as a reproducibility check rather than a conventional feature test. The generated API reference under `docs/content/docs/api/` must remain consistent with the source specification in `docs/openapi-sdk.json`, ensuring that published documentation and machine-readable API definition remain aligned. This is especially important because stale or hand-maintained reference pages would undermine trust in the developer-facing surface even if the backend implementation itself remained correct.

The developer-experience probes used to validate the documentation site in a running state appear in Table 7.8.

#	Feature	Probe	Expected result
1	Client-side full-text search	Invoke search and query `error tracking`	Results include the relevant documentation page.
2	Multi-language code tabs with persistence	Switch a code example to `Python` and navigate to another page	The selected language tab remains persisted.
3	Copy-to-clipboard controls	Copy a code block and paste it externally	The pasted content matches the rendered block.
4	Dark-mode toggle	Toggle theme and reload	Theme changes visually and persists across reload.

5	`/llms.txt` endpoint	`curl <http://localhost:3000/llms.txt>`	A plain-text page index is returned.
6	`/llms-full.txt` endpoint	`curl <http://localhost:3000/llms-full.txt>`	A concatenated plain-text corpus of site content is returned.
7	Per-page Markdown export	Request a page-specific `llms.mdx` route	`Content-Type: text/markdown` is returned together with the page body.
8	Per-page Open Graph image	Request the `og` route for a page	A PNG response suitable for sharing metadata is returned.

7.5 Functional Test Cases

Functional acceptance testing was documented using the template required by the project brief, with the fields Identifier, Priority, Related requirement(s), Short description, Pre-condition(s), Input data, Detailed steps, Expected result(s), and Post-condition(s). Nineteen functional test cases were defined to cover the delivered user flows and the prerequisites on which those flows depend. The cases trace to UC-1 (error monitoring), UC-2 (load testing), UC-4 (AI-assisted analysis), and UC-5 (security scanning), and capture supporting paths such as registration, authentication, project creation, ingest-key handling, project deletion, AI cache and regeneration, PII redaction, load-test cooldown enforcement, and security-scan risk classification.

The functional test-case set is summarised in Table 7.9 before the detailed case specifications that follow.

Identifier	Priority	Related requirement	Short description
TC-1	High	UC-1 prerequisite	User account registration through `POST /api/auth/register`.
TC-2	High	UC-1 prerequisite	Authenticated login and JWT issuance through `POST /api/auth/login`.
TC-3	Medium	UC-1 prerequisite	Login rejection with an invalid password.
TC-4	High	UC-1 prerequisite	Project creation with one-time ingest-key reveal.
TC-5	High	UC-1	Runtime error ingestion through the JavaScript SDK.
TC-6	High	UC-1	Recurrent-error coalescence through fingerprint-based upsert.
TC-7	High	UC-1	Ingest rejection with an invalid API key.
TC-8	High	UC-1	Grouped-error listing on the dashboard.
TC-9	High	UC-1 monitor arm	Website monitor registration.
TC-10	High	UC-1 monitor arm	Up/down classification of reachable and unreachable URLs.

TC-11	Medium	UC-1 prerequisite	Cascade deletion of a project and related entities.
TC-12	Medium	UC-1	Fingerprint equality across JavaScript, Java, and Python SDKs.
TC-13	High	UC-4	AI-assisted error analysis request returning structured remediation.
TC-14	Medium	UC-4	AI analysis caching and explicit regeneration.
TC-15	High	UC-4	PII redaction prior to LLM dispatch.
TC-16	High	UC-2	Load test execution and metric capture.
TC-17	High	UC-2	Load test cooldown enforcement returning HTTP 429.
TC-18	High	UC-5	Security scan with additive risk classification.
TC-19	Medium	UC-5	Security scan detection of database-port exposure.

7.5.1 TC-1 User account registration

Field	Specification
Identifier	TC-1
Priority	High
Related requirement(s)	UC-1 prerequisite; a developer must possess an account before obtaining an ingest key.
Short description	Creation of a user account through `POST /api/auth/register`, including automatic provisioning of a default project and a one-time ingest key.
Pre-condition(s)	The backend is running at `http://localhost:8080`, the dashboard is available at `http://localhost:3000`, and no account exists for the submitted email address.
Input data	Full name, username or display name, department where applicable, email address, phone number where applicable, and password; the original implementation example uses `name`, `email`, and `password`.
Detailed steps	1. Navigate to `http://localhost:3000/register`. 2. Complete the registration form with valid data. 3. Submit the form.
Expected result(s)	The system returns HTTP 200 with `{ token, user, project, apiKey }`, redirects the user to `/overview`, persists the JWT in `localStorage`, and displays the initial project with the raw `ask_live_…` key revealed once.
Post-condition(s)	Rows are created in `users`, `projects`, and `api_keys`, with only the API-key hash and safe prefix stored persistently.

7.5.2 TC-2 Authenticated login

Field	Specification
Identifier	TC-2
Priority	High
Related requirement(s)	UC-1 prerequisite; authenticated access is required for dashboard use.
Short description	A registered user logs in successfully and receives a signed JWT.
Pre-condition(s)	TC-1 has been completed successfully and the authentication state can be rehydrated after reload.

Input data	Valid registered email and password.
Detailed steps	1. Navigate to `http://localhost:3000/login`. 2. Enter the valid credentials. 3. Submit the form.
Expected result(s)	The system returns HTTP 200 with a valid JWT, redirects the user to `/overview`, and stores the token for subsequent authenticated requests.
Post-condition(s)	The authentication store contains the logged-in user identity and the browser retains the persisted JWT.

7.5.3 TC-3 Login rejection with invalid password

Field	Specification
Identifier	TC-3
Priority	Medium
Related requirement(s)	UC-1 prerequisite; authentication must reject invalid credentials securely.
Short description	A registered user attempts to log in with an incorrect password.
Pre-condition(s)	TC-1 has been completed successfully.
Input data	Valid registered email and an invalid password.
Detailed steps	1. Navigate to `http://localhost:3000/login`. 2. Enter the email and incorrect password. 3. Submit the form.
Expected result(s)	The request returns HTTP 401, no JWT is issued, and the interface displays a generic invalid-credentials message without disclosing whether the email exists.
Post-condition(s)	No authentication state is created and no additional persistent records are written.

7.5.4 TC-4 Project creation with single-use ingest-key reveal

Field	Specification
Identifier	TC-4
Priority	High
Related requirement(s)	UC-1 prerequisite; all ingested events must be associated with a project.
Short description	An authenticated user creates a project and observes the raw ingest key exactly once.
Pre-condition(s)	TC-2 has been completed and the user is on the projects page.
Input data	Project name and platform, such as `{ "name": "Demo", "platform": "javascript" }`.
Detailed steps	1. Open the new-project form. 2. Enter the project details. 3. Submit the form. 4. Observe the initial project card. 5. Navigate away and return to the page.
Expected result(s)	The API returns project metadata and a raw `ask_live_…` key, the full key is shown once together with copy controls and warning text, and later renders show only a safe prefix with obfuscation.

Post-condition(s)	A project row and an API-key row exist, while only the hash and prefix of the key remain stored.

7.5.5 TC-5 Runtime error ingestion through the JavaScript SDK

Field	Specification
Identifier	TC-5
Priority	High
Related requirement(s)	UC-1 primary path.
Short description	The JavaScript SDK captures runtime exceptions and dispatches them successfully to the ingest endpoint.
Pre-condition(s)	TC-4 has been completed and the project's valid ingest key is available.
Input data	Valid `SEESTACK_API_KEY`, `SEESTACK_ENDPOINT=http://localhost:8080`, and the script `sdks/examples/example-app.js`.
Detailed steps	1. Export the required environment variables. 2. Execute `node sdks/examples/example-app.js`. 3. Wait for the three `sent` lines to appear.
Expected result(s)	Each call to `captureException` returns success with HTTP 202, Kafka receives three messages, and ClickHouse stores three raw events with distinct `event_id` values.
Post-condition(s)	PostgreSQL contains two grouped issues after coalescence and ClickHouse contains three raw rows.

7.5.6 TC-6 Recurrent-error coalescence through fingerprint upsert

Field	Specification
Identifier	TC-6
Priority	High
Related requirement(s)	UC-1 alternate path; existing fingerprints increment occurrence counters rather than creating duplicate issue groups.
Short description	Two exceptions with the same fingerprint are merged into one grouped issue with an incremented occurrence count.
Pre-condition(s)	TC-5 has been completed.
Input data	Two repeated `TypeError` events and one distinct `SyntaxError` event from the runnable example.
Detailed steps	Query the PostgreSQL `error_groups` table after execution of TC-5.
Expected result(s)	Exactly two grouped rows are present: one for `TypeError` with `occurrences = 2` and one for `SyntaxError` with `occurrences = 1`.
Post-condition(s)	The repeated error shows `last_seen > first_seen`, while the single-occurrence error shows `last_seen = first_seen`.

7.5.7 TC-7 Ingest rejection with invalid API key

Field	Specification
Identifier	TC-7
Priority	High
Related requirement(s)	UC-1 alternate path; invalid ingest credentials must be rejected with HTTP 401.
Short description	The ingest endpoint rejects missing, malformed, or unrecognised API keys.
Pre-condition(s)	The backend is running.
Input data	Three requests: one with no `X-SeeStack-Key`, one with `wrong`, and one with a well-formed but non-existent `ask_live_…` key.
Detailed steps	Execute the three invalid ingestion requests against `POST /ingest/v1/errors`.
Expected result(s)	All three requests return HTTP 401, no Kafka message is published, and no database row is created in either persistence layer.
Post-condition(s)	System state remains unchanged.

7.5.8 TC-8 Grouped-error listing on the dashboard

Field	Specification
Identifier	TC-8
Priority	High
Related requirement(s)	UC-1 grouped error query path; AI suggestions are deferred, but grouping behaviour is delivered and evaluated.
Short description	The `/errors` page displays the coalesced issue groups with the correct occurrence counts.
Pre-condition(s)	TC-5 and TC-6 have been completed successfully.
Input data	A valid JWT from authenticated login.
Detailed steps	1. Navigate to `http://localhost:3000/errors`. 2. Observe the grouped issues list. 3. Select one row to open the detail page.
Expected result(s)	Exactly two grouped rows are displayed, labelled with the `TypeError` and `SyntaxError` classes and occurrence counts of `2` and `1`; selecting a row navigates to the issue detail view and shows the full stack trace.
Post-condition(s)	The session remains authenticated and no data-mutation request is triggered by viewing the page.

7.5.9 TC-9 Website-monitor registration

Field	Specification
Identifier	TC-9
Priority	High
Related requirement(s)	UC-1 monitor arm in the realised scope.

Short description	An authenticated user registers a reachable URL for periodic availability checks.
Pre-condition(s)	TC-2 has been completed successfully.
Input data	Monitor data such as `{ "name": "Example", "url": "<https://example.com>", "interval_seconds": 60 }`.
Detailed steps	1. Navigate to `/monitors`. 2. Open the new-monitor form. 3. Enter the monitor data. 4. Submit the form.
Expected result(s)	The API accepts the request, the monitor is stored in `monitor_configs`, and a monitor-check row appears in ClickHouse indicating an operational result.
Post-condition(s)	The monitor appears on the dashboard with an operational status badge.

7.5.10 TC-10 Up/down classification

Field	Specification
Identifier	TC-10
Priority	High
Related requirement(s)	UC-1 monitor arm and the monitor-classification algorithm.
Short description	A reachable URL is classified as operational and an unreachable URL is classified as down.
Pre-condition(s)	TC-2 has been completed and at least one monitor can be registered.
Input data	A second monitor such as `{ "name": "Unreachable", "url": "<http://127.0.0.1:1/nope>", "interval_seconds": 60 }`.
Detailed steps	1. Register the unreachable monitor. 2. Wait for a scheduled check to run. 3. Query recent monitor-check records. 4. Observe the dashboard status badges.
Expected result(s)	The reachable URL records `status = 1` and appears operational, while the unreachable URL records `status = 0` with `status_code = 0` and appears down.
Post-condition(s)	Both monitors remain visible and their dashboard summaries reflect the recorded check outcomes.

7.5.11 TC-11 Project cascade deletion

Field	Specification
Identifier	TC-11
Priority	Medium
Related requirement(s)	UC-1 prerequisite; project lifecycle must preserve referential integrity.
Short description	Deleting a project removes related API keys, monitor configurations, and grouped-error summaries.
Pre-condition(s)	TC-4, TC-5, and TC-9 have been completed for the target project.
Input data	The identifier of the project to be deleted.

Detailed steps	1. Open the project actions menu. 2. Select delete and confirm. 3. Verify that related PostgreSQL tables no longer contain rows for the deleted project.
Expected result(s)	The system returns HTTP 204, the project card disappears from the interface, and counts in `api_keys`, `monitor_configs`, and `error_groups` for that project become zero.
Post-condition(s)	The project no longer exists in PostgreSQL, while raw ClickHouse records remain subject to the configured retention policy.

7.5.12 TC-12 Fingerprint equality across SDK languages

Field	Specification
Identifier	TC-12
Priority	Medium
Related requirement(s)	UC-1 and cross-language SDK interoperability.
Short description	JavaScript, Java, and Python SDKs produce the same fingerprint for the same canonical exception input.
Pre-condition(s)	TC-4 has been completed and a valid ingest key exists.
Input data	A shared synthetic exception class and two identical non-internal stack frames dispatched by all three SDKs.
Detailed steps	1. Send the same canonical exception payload through each SDK. 2. Query grouped errors by fingerprint for the project.
Expected result(s)	A single fingerprint is produced across all three SDKs and the grouped occurrence count reflects all submitted events.
Post-condition(s)	The shared fingerprint equals the SHA-256 digest of the canonicalised input string built from the exception class and application frames.

7.5.13 TC-13 AI-assisted error analysis request

Field	Specification
Identifier	TC-13
Priority	High
Related requirement(s)	UC-4; AI-assisted analysis is the project's primary novel capability.
Short description	An authenticated user requests AI-assisted analysis for a grouped issue and receives a structured remediation report.
Pre-condition(s)	TC-5 and TC-6 have been completed, the `OPENAI_API_KEY` environment variable is configured, and a valid JWT is held in the browser session.
Input data	The fingerprint of an existing grouped issue and the project's JWT.
Detailed steps	1. Open `http://localhost:3000/errors/<fingerprint>`. 2. Activate the **Explain & Suggest Fix** control. 3. Wait for the response.
Expected result(s)	The backend returns HTTP 200 with a JSON body containing `explanation`, `rootCause`, `fixSteps`, `prevention`, `severity`, and

	`confidence`; the panel renders all four output cards together with severity, confidence, and model metadata pills.
Post-condition(s)	A row is inserted in `ai_error_analyses` keyed by `(project_id, fingerprint)` containing the serialised response and the model identifier.

7.5.14 TC-14 AI analysis caching and regeneration

Field	Specification
Identifier	TC-14
Priority	Medium
Related requirement(s)	UC-4; cache-first retrieval prevents unnecessary external dispatch.
Short description	A repeat AI analysis request returns the cached response, while an explicit regeneration overwrites it with a freshly generated analysis.
Pre-condition(s)	TC-13 has been completed for the target grouped issue.
Input data	The fingerprint of the same grouped issue used in TC-13.
Detailed steps	1. Issue `GET /api/v1/errors/{fingerprint}/ai-analysis` and confirm the cached payload is returned. 2. Issue `POST /api/v1/errors/{fingerprint}/ai-analysis?force=true` to regenerate. 3. Re-query the cached endpoint.
Expected result(s)	The first GET returns the previously stored payload with `cached = true` and the original `created_at`; the regenerate call returns a fresh payload with an updated `created_at` and `occurrences_at_generation`; the subsequent GET returns the new payload.
Post-condition(s)	The single row in `ai_error_analyses` for the issue reflects the most recent regeneration.

7.5.15 TC-15 PII redaction prior to LLM dispatch

Field	Specification
Identifier	TC-15
Priority	High
Related requirement(s)	UC-4 supporting; the privacy-by-design property of the AI module.
Short description	Sensitive metadata, bearer tokens, OpenAI-style keys, and JWT-shaped strings are redacted before the AI module dispatches the payload.
Pre-condition(s)	TC-4 has been completed and a project ingest key is available.
Input data	An error event whose metadata map contains `password`, `token`, and `authorization` fields, whose message contains a `Bearer` token, and whose stack trace contains an `sk-` API key.
Detailed steps	1. Submit the crafted event through the JavaScript SDK. 2. Trigger AI analysis on the resulting grouped issue. 3. Inspect the outbound payload through `ErrorDataSanitizerTest` for unit-level evidence and through temporary instrumentation for end-to-end verification.

Expected result(s)	The metadata fields appear as `[REDACTED]`; the bearer token, `sk-` key, and JWT pattern in free-form values are replaced with `[REDACTED]`; no sensitive value reaches the OpenAI request.
Post-condition(s)	The cached AI response in `ai_error_analyses` carries no sensitive value originating from the redacted inputs.

7.5.16 TC-16 Load test execution and metric capture

Field	Specification
Identifier	TC-16
Priority	High
Related requirement(s)	UC-2; bounded HTTP load testing within the realised scope.
Short description	An authenticated user runs a bounded load test against a registered monitor and observes the captured metrics.
Pre-condition(s)	TC-9 has been completed for a reachable monitor target and no load test against that target has run within the previous 60 seconds.
Input data	`{ "monitorId": "<uuid>", "requestedCount": 20, "concurrency": 5 }`.
Detailed steps	1. Submit `POST /api/v1/load-tests`. 2. Wait for the synchronous response. 3. Refresh the load-test history.
Expected result(s)	The response body reports total, successful, and failed counts together with average, minimum, maximum, and p95 latency, and a status-code distribution map; the corresponding `load_tests` row records identical values; the history table renders the new entry.
Post-condition(s)	The 60-second cooldown for the project–target pair is active.

7.5.17 TC-17 Load test cooldown enforcement

Field	Specification
Identifier	TC-17
Priority	High
Related requirement(s)	UC-2 alternate path; abuse prevention through bounded execution.
Short description	A second load test against the same target within 60 seconds is rejected with HTTP 429 and the remaining cooldown duration.
Pre-condition(s)	TC-16 has just been completed.
Input data	A second `POST /api/v1/load-tests` for the same monitor identifier.
Detailed steps	1. Submit the second request immediately after TC-16. 2. Inspect the response status and body. 3. Confirm that no new row is created in `load_tests`.
Expected result(s)	The system returns HTTP 429 with a body indicating the remaining cooldown seconds; no new `load_tests` row is created; the user-facing dashboard surfaces the rejection without disrupting prior history.

Post-condition(s)	The cooldown timer continues to count down toward zero without modification.

7.5.18 TC-18 Security scan with additive risk classification

Field	Specification
Identifier	TC-18
Priority	High
Related requirement(s)	UC-5; educational security scanning within the realised scope.
Short description	An authenticated user submits a security scan and receives a risk classification consistent with the additive scoring algorithm.
Pre-condition(s)	TC-2 has been completed.
Input data	`{ "target": "<hostname-or-ip>" }` for a target with at least one open port from the fixed scan set.
Detailed steps	1. Submit `POST /api/v1/security-scans`. 2. Wait for the synchronous response. 3. Open the scan detail page on the dashboard.
Expected result(s)	The response carries the resolved host, the open and closed port lists, the detected services, the HTTP probe summary, the security-headers presence map, the numeric risk score, the LOW/MEDIUM/HIGH risk level, and the generated summary text; the dashboard renders all of these and the matching `security_scans` row carries identical values.
Post-condition(s)	The historical scan list contains the new entry with its risk badge.

7.5.19 TC-19 Security scan detection of database-port exposure

Field	Specification
Identifier	TC-19
Priority	Medium
Related requirement(s)	UC-5; database-port exposure dominates the additive risk model.
Short description	A security scan against a target whose PostgreSQL port is reachable produces a HIGH classification and an explicit warning.
Pre-condition(s)	A controlled test target exposes port 5432 within the scanner's reach.
Input data	`{ "target": "<hostname-or-ip>" }` for the controlled target.
Detailed steps	1. Submit the scan request. 2. Inspect the response and dashboard rendering. 3. Cross-reference the warnings array against the `SecurityAnalyzerTest` evidence.
Expected result(s)	The risk score is at least 30 due to the +30 database-port contribution; the classification escalates to HIGH when combined with at least one further finding; the warnings array contains "Database port 5432 (PostgreSQL) is publicly reachable"; the persisted `security_scans` row carries identical values.

Post-condition(s)	The dashboard's latest-scan card on the overview page reflects the HIGH classification.

7.6 Traceability and Evaluation

The functional test cases are traceable to both requirements and implementation artefacts, which ensures that the evaluation is not merely descriptive but structurally linked to the realised scope. This traceability also shows where evidence is provided by a single testing layer and where it is reinforced by multiple layers, such as manual testing plus automated unit assertions. The resulting matrix clarifies the degree of defence-in-depth achieved across the delivered behaviours.

Table 7.10 presents the traceability matrix linking each functional test case to its requirement, implementation artefact, and automation level.

Identifier	Requirement	Implementation artefact	Automation level
TC-1	UC-1 prerequisite (authentication)	`AuthService.register`	Manual
TC-2	UC-1 prerequisite	`AuthService.login` and `JwtService.issue`	Manual
TC-3	UC-1 prerequisite (negative path)	`AuthService.login`	Manual
TC-4	UC-1 prerequisite (project and ingest key)	`ProjectService.create` and `ApiKeyGeneratorService`	Manual, supported by `ApiKeyGeneratorServiceTest`
TC-5	UC-1 primary path	`ErrorIngestController` and `sdks/examples/example-app.js`	End-to-end runnable probe
TC-6	UC-1 alternate path (coalescence)	`ErrorGroupConsumer`, database uniqueness constraint, and `ErrorFingerprintService`	End-to-end runnable probe plus unit support
TC-7	UC-1 alternate path (unauthorised key rejection)	`ApiKeyAuthFilter`	Manual
TC-8	UC-1 query path	`ErrorQueryController` and dashboard errors page	Manual
TC-9	UC-1 monitor arm	`MonitorController` and `MonitorScheduler`	Manual

TC-10	UC-1 monitor arm and classifier	`MonitorScheduler.isUp`	Manual plus unit support
TC-11	Referential integrity in project lifecycle	`ON DELETE CASCADE` constraints	Manual
TC-12	UC-1 and SDK interoperability	JavaScript, Java, and Python SDKs with `ErrorFingerprintService`	Manual plus unit support
TC-13	UC-4 (AI-assisted analysis)	`ErrorAiAnalysisController`, `ErrorAiAnalysisService`, and `OpenAiClient`	Manual
TC-14	UC-4 (cache and regeneration)	`ErrorAiAnalysisService` and `AiErrorAnalysisRepository`	Manual
TC-15	UC-4 supporting (privacy)	`ErrorDataSanitizer`	Manual plus unit support through `ErrorDataSanitizerTest`
TC-16	UC-2 (load testing)	`LoadTestController`, `LoadTestService`, and `LoadTestRunner`	Manual plus unit support through `LoadTestRunnerLimitsTest`
TC-17	UC-2 (cooldown enforcement)	`LoadTestService` cooldown logic	Manual
TC-18	UC-5 (security scanning)	`SecurityScanController`, `SecurityScanService`, `PortScanner`, and `SecurityAnalyzer`	Manual plus unit support through `SecurityAnalyzerTest`
TC-19	UC-5 (database-port exposure)	`SecurityAnalyzer` additive scoring	Manual plus unit support through `SecurityAnalyzerTest`

The evaluation demonstrates that every delivered path within the realised SeeStack scope is covered by at least one explicit test case and traceable to a concrete implementation artefact. Defence-in-depth is strongest for the five algorithmic components documented in section 6.6—fingerprint generation, monitor classification, PII sanitisation, percentile latency calculation, and additive risk scoring—together with API-key generation and dashboard insights aggregation, where manual or end-to-end evidence is reinforced by deterministic unit assertions. This distribution is appropriate because these components represent the most critical correctness, privacy, and security boundaries of the delivered system.

The evidence base further shows that the system operates correctly at multiple levels of abstraction. Unit tests verify the invariants of the five algorithmic components together with API-key generation and dashboard insights aggregation, the runnable example validates the complete ingestion contract, browser-based inspection confirms correct frontend integration, infrastructure probes demonstrate deployability, and documentation-site checks verify the public developer-facing surface. Together, these layers provide a coherent and technically justified evaluation of the implemented system within the realised project scope.

7.7 User Interface Evidence

The delivered interface is illustrated through screenshots showing the principal operational states of the application. These figures provide visual evidence of the implemented frontend and demonstrate how the platform exposes its core capabilities, including error monitoring, AI-assisted analysis, uptime monitoring, load testing, and educational security scanning.

The screenshots are presented to support the implementation discussion by showing representative user-facing views of the realised system. Together, they help confirm that the delivered release includes a coherent and usable interface aligned with the functional scope described in the preceding sections.

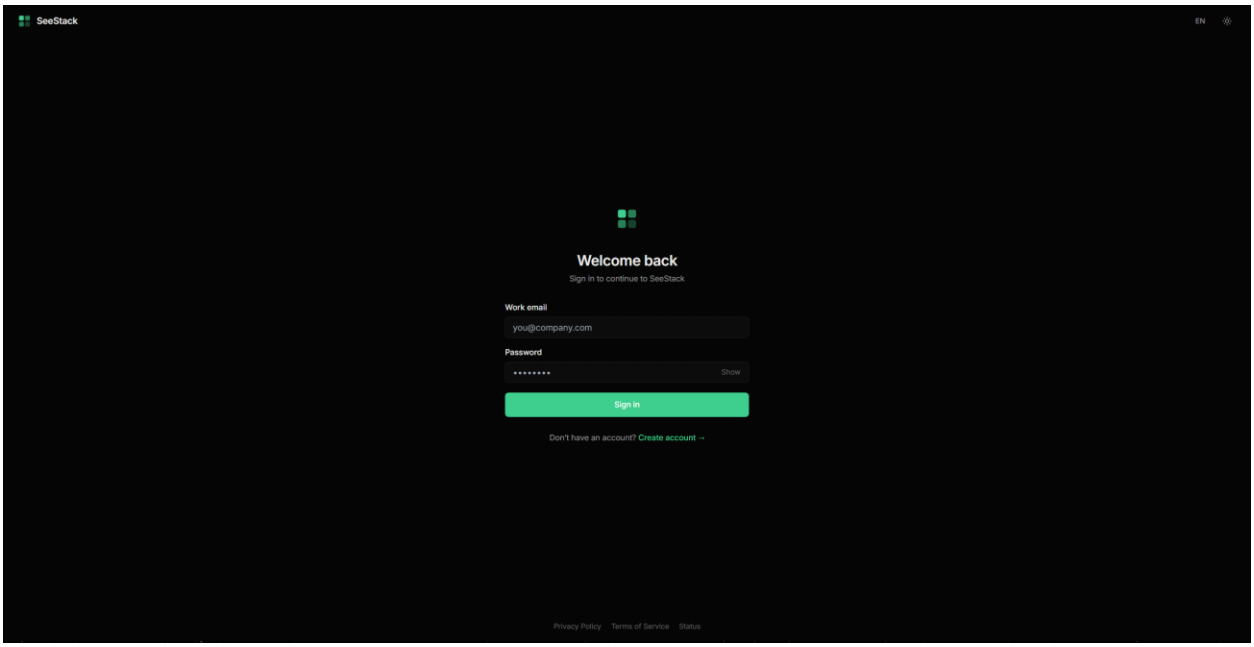


Figure 7.1. Login interface presenting the email and password credential entry fields used by the internal authentication flow.

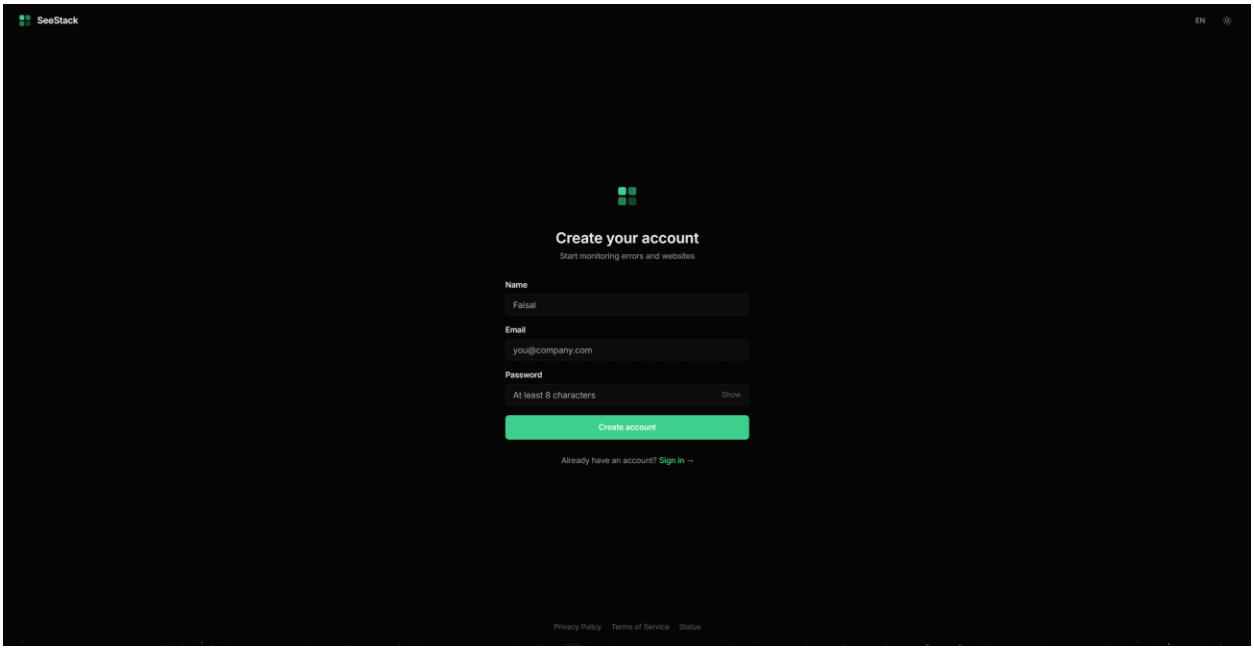


Figure 7.2. Registration interface used to create a new user account, which simultaneously provisions a default project and an associated ingest API key.

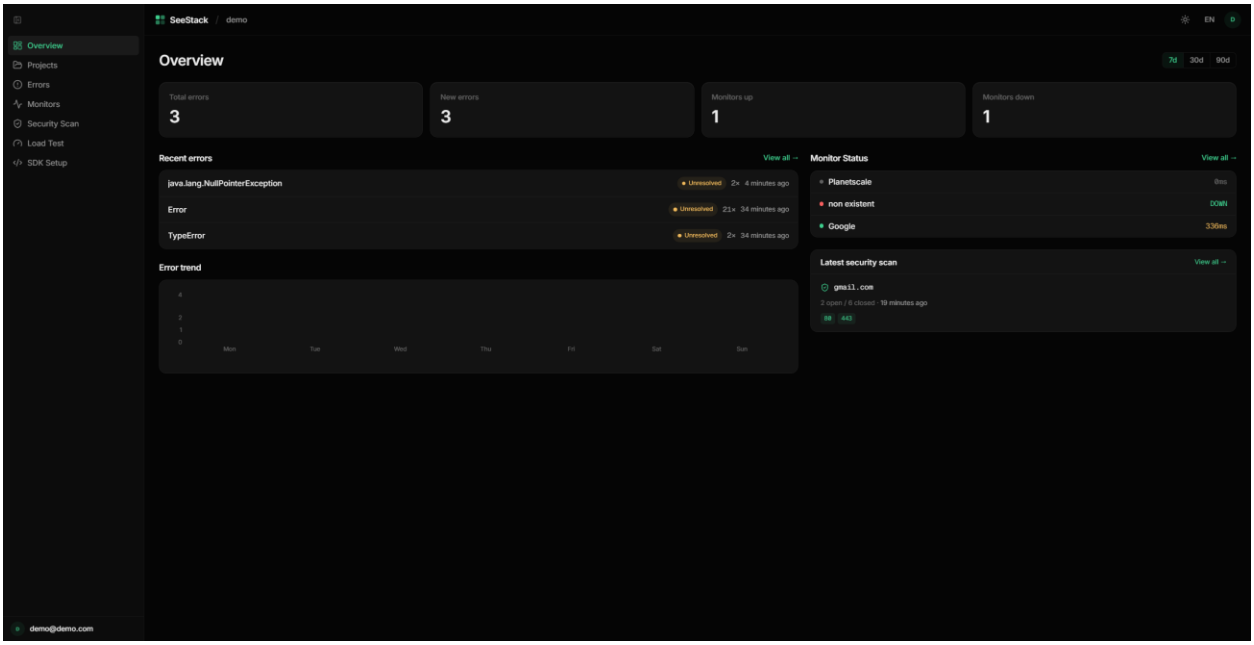


Figure 7.3. Overview dashboard presenting aggregate project statistics, recent error events, and the current status of registered monitors.

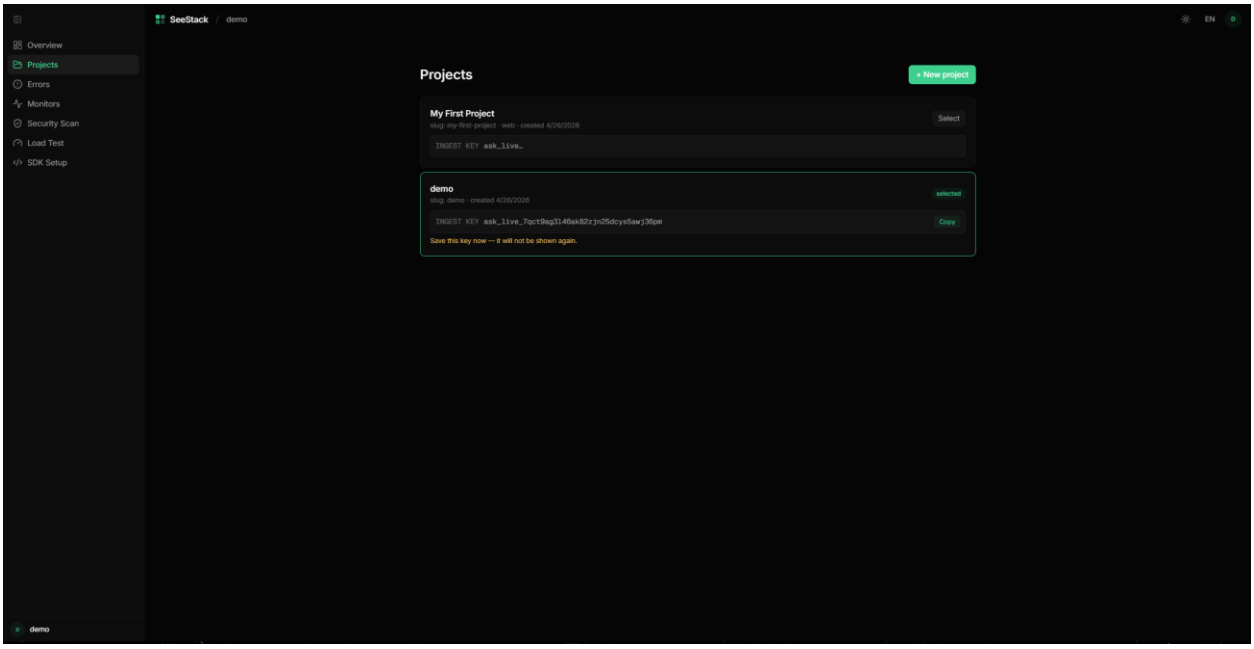


Figure 7.4. Projects page demonstrating the one-time ingest-key reveal pattern immediately following the creation of a new project.

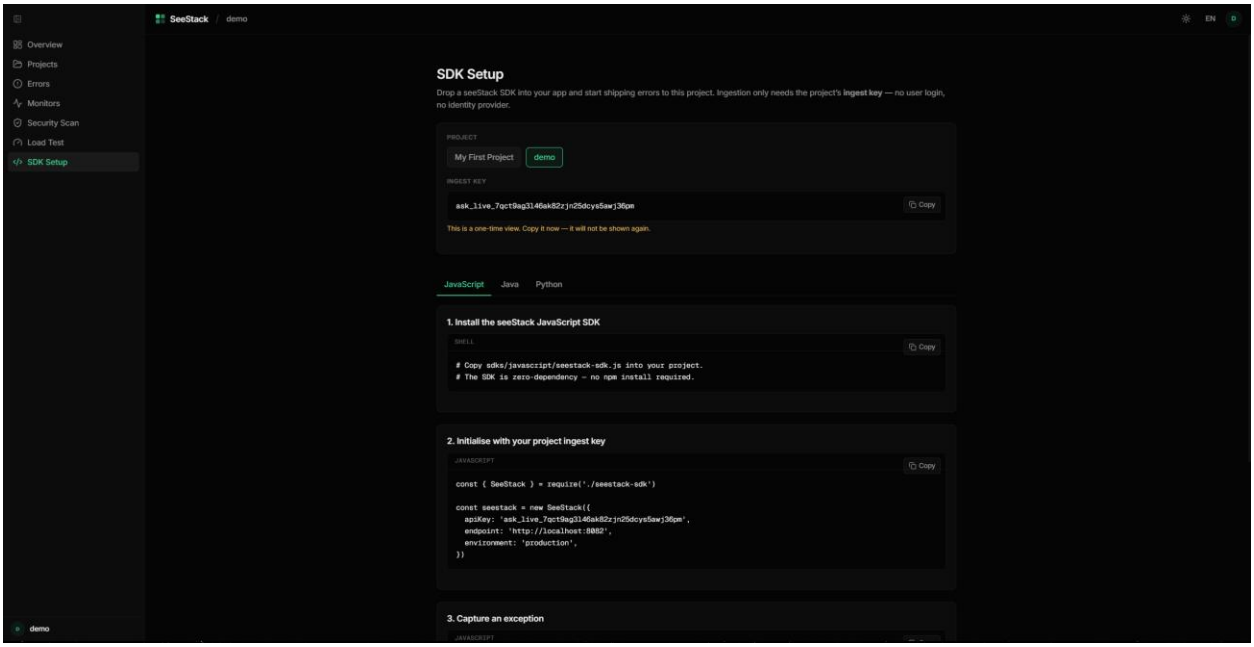


Figure 7.5. SDK Setup page providing copy-ready installation, initialisation, and event-capture snippets for the three first-party SDK languages.

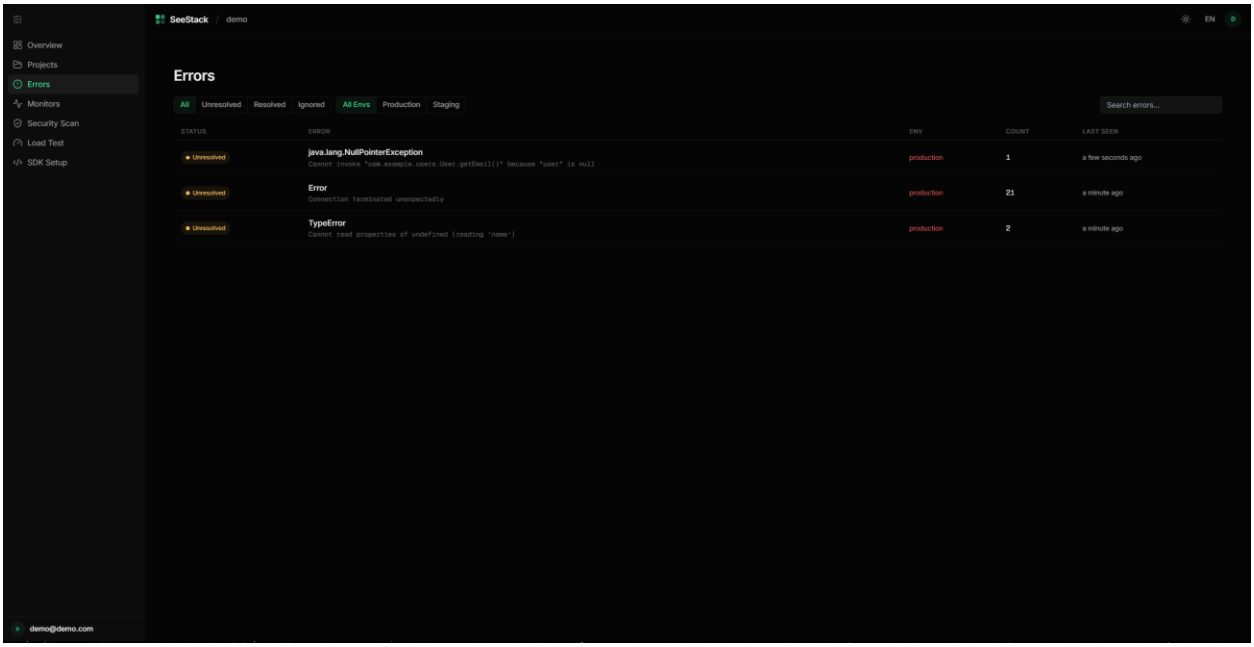


Figure 7.6. Errors page rendering the grouped issues table with filter controls for status, severity level, and environment.

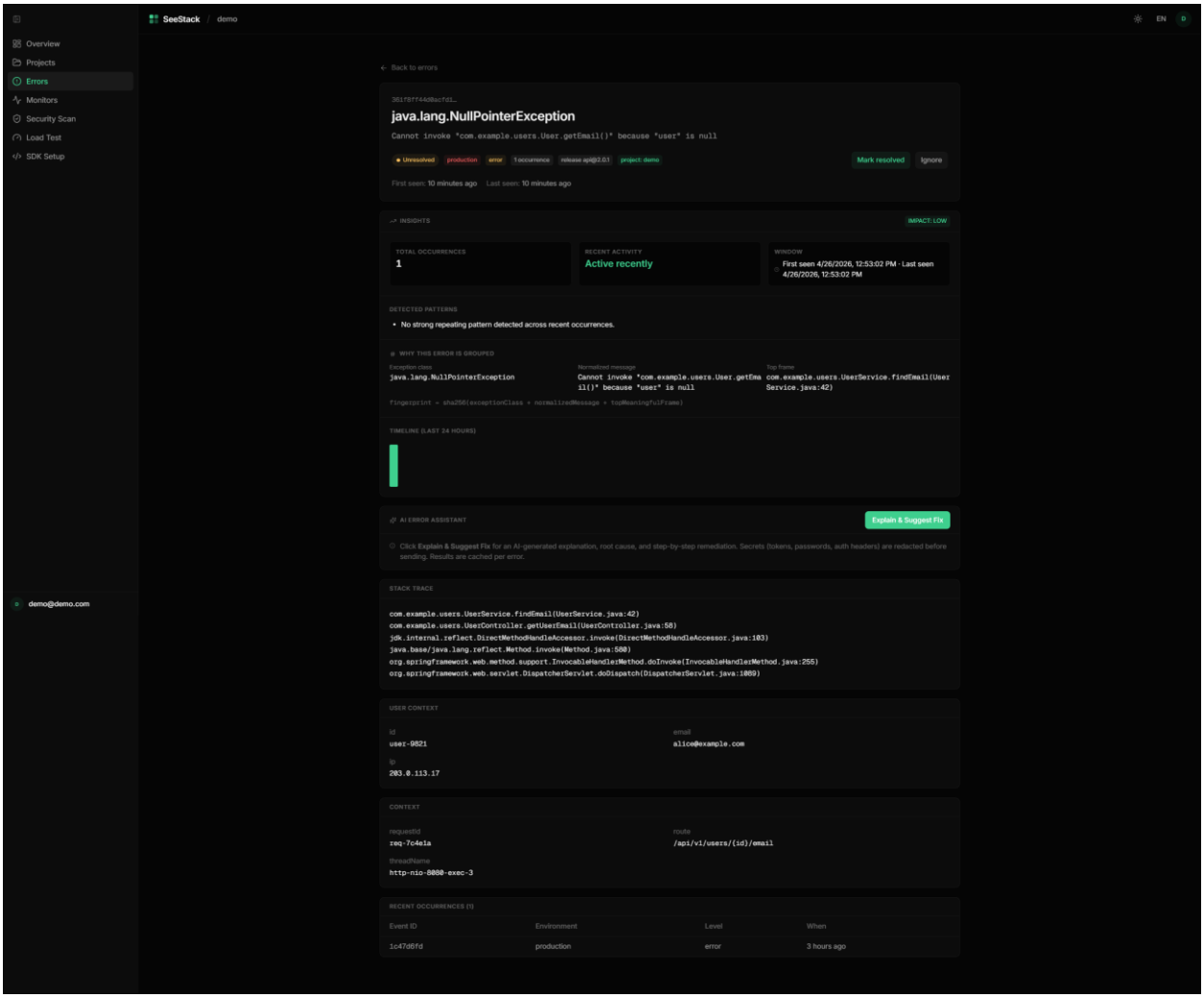


Figure 7.7. Error Detail page presenting the full stack trace, exception metadata, occurrence statistics, and resolution actions for a single grouped issue.

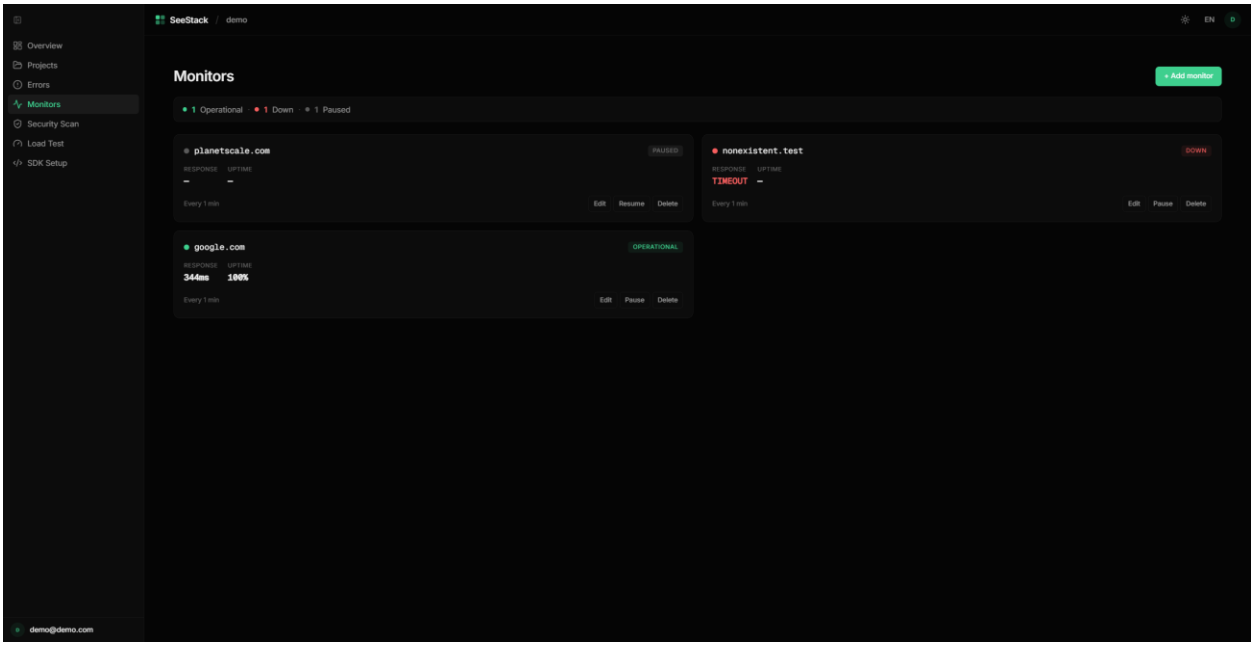


Figure 7.8. Monitors page presenting card-based status visualisation for registered URLs and distinguishing operational from unreachable endpoints.

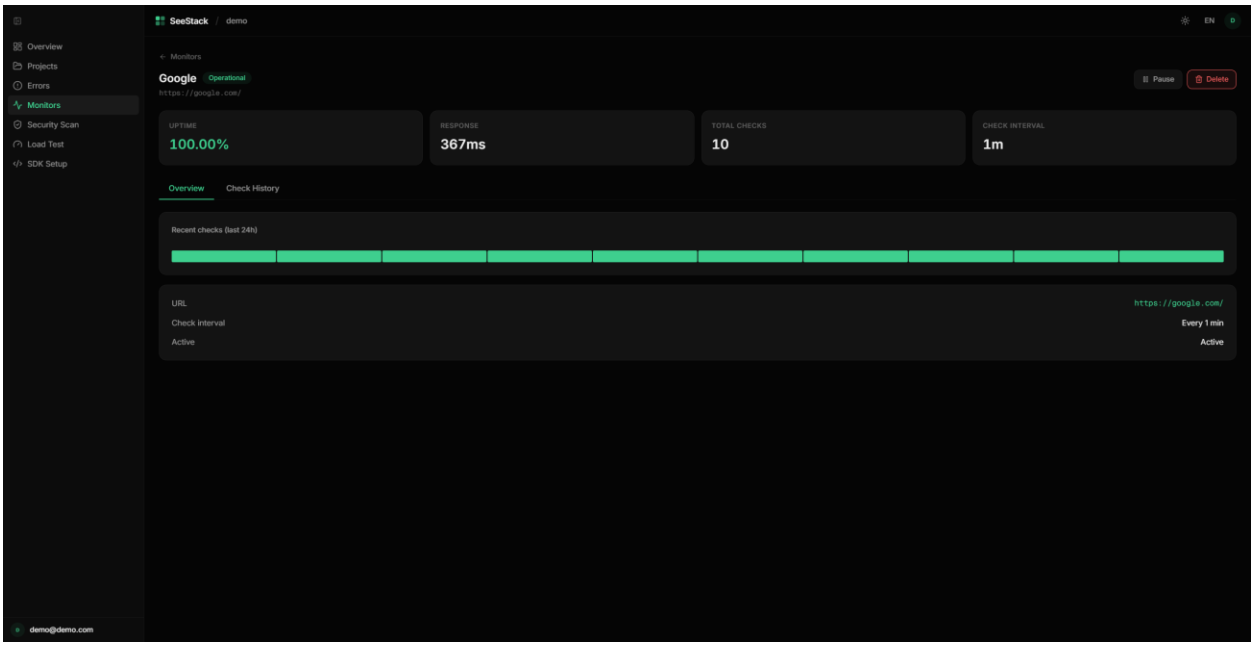


Figure 7.9. Monitor Detail page showing historical uptime, response-time trends, and the most recent check outcomes for a registered URL.

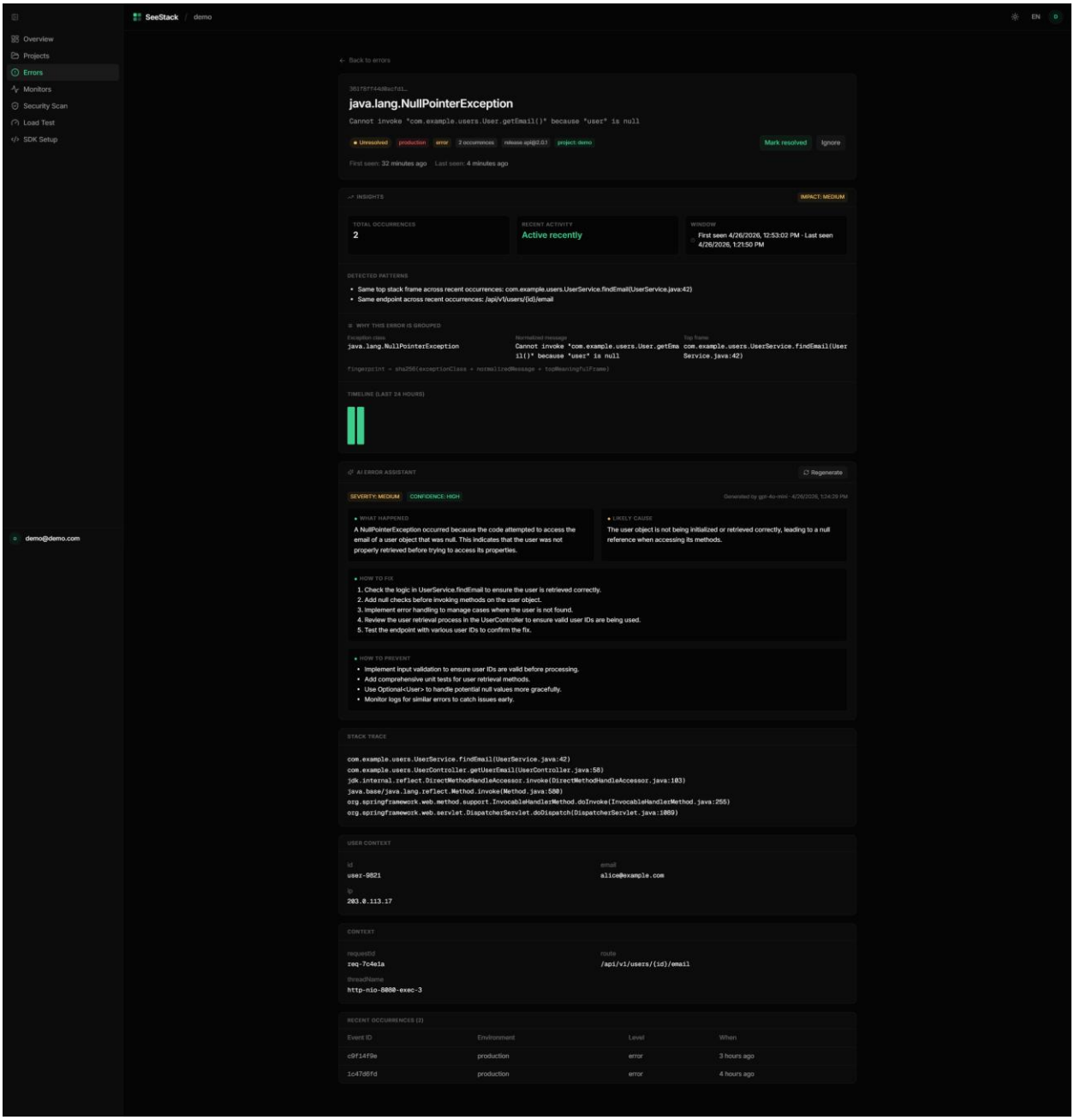


Figure 7.10. AI-Assisted Error Analysis panel presenting a structured GPT-4o-mini explanation with severity, confidence, and remediation steps for a grouped runtime issue.

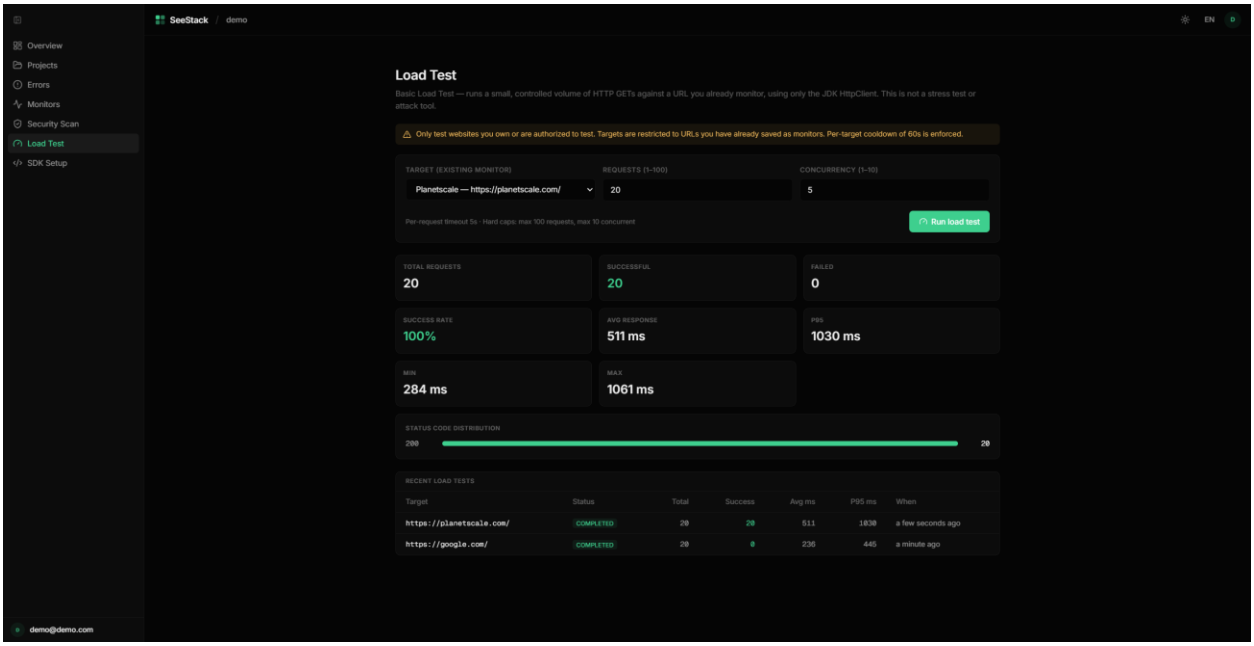


Figure 7.11. Load Test page showing bounded request execution against a registered monitor with success rate, percentile latency, and status-code distribution.

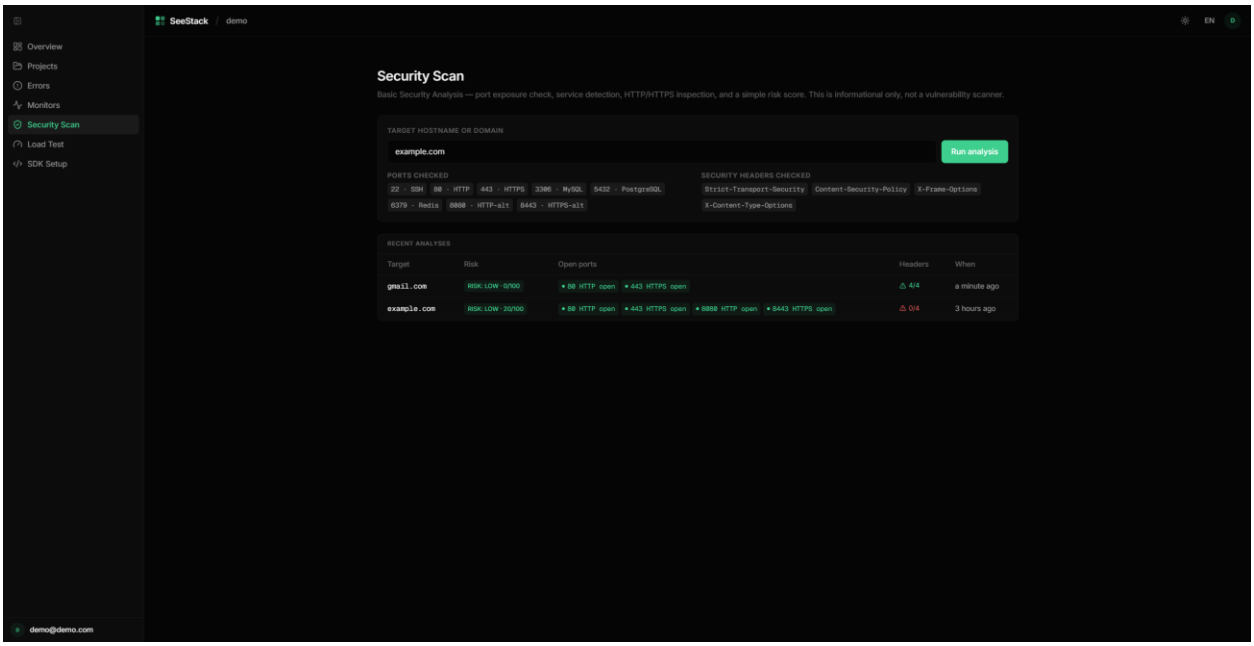


Figure 7.12. Security Scan page presenting port detection, HTTP header analysis, and additive risk scoring for an inspected target.

CHAPTER 8: RESULTS AND ANALYSIS

8.1 Analytical Framing and Techniques

SeeStack is an engineering-delivery senior project whose principal outcome is a working platform rather than a research publication. Accordingly, the analysis focuses on the delivered artefact itself through measurable system characteristics, verification outcomes, dependency footprint, deployment profile, architectural-decision effects, and comparative positioning against the tools examined in the literature review. All empirical statements are restricted to evidence supported by repository artefacts or by verification evidence established elsewhere in the report.

Three complementary analytical techniques were selected because they address different properties of the delivered system while drawing on different evidence sources. Descriptive quantitative analysis provides reproducible measurements of system size, testing outcomes, dependency distribution, and deployment characteristics. Architectural-decision analysis evaluates the observable consequences and trade-offs of the principal engineering decisions, while comparative analysis situates the delivered capabilities within the competitive landscape without extending claims beyond the implemented scope.

Table 8.1. Analytical techniques applied to the delivered system

Technique	Analytical focus	Evidence source	Rationale
Descriptive quantitative analysis	System size, testing outcomes, dependency footprint, deployment profile	Repository counts obtained through `wc -l` and `find`, Gradle test output, and verification evidence reported in the testing material	Produces reproducible and independently verifiable measurements with minimal interpretive ambiguity.
Architectural-decision analysis	Ten implementation refinements identified earlier in the report	Repository artefacts together with implementation and testing evidence	Evaluates each decision through its observable consequence and associated cost rather than merely restating that the decision was made.
Comparative analysis against prior art	Five delivered capabilities: AI-assisted error analysis, runtime error monitoring, website availability monitoring,	Literature-review evidence and public documentation of Sentry, Prometheus/Grafana, Datadog, and the ELK stack	Positions the delivered system within the established landscape while restricting

	automated load testing, and educational security scanning		comparison to capabilities that are demonstrably implemented.

The three techniques are complementary rather than redundant. Quantitative characterisation establishes what has been built, architectural-decision analysis explains what the principal design choices achieved, and comparative analysis clarifies how the delivered system stands relative to prior tools operating in the same scope. The progression therefore moves from artefact-level evidence to design-level interpretation and then to market-level positioning, before addressing scope realisation and the limitations that qualify the findings.

8.2 Quantitative Characterisation of the Delivered System

The source-code footprint of the delivered artefact is summarised in Table 8.2 through reproducible counts derived from the repository. Application code, documentation-site infrastructure, test code, and authored documentation content are reported separately because they represent different forms of work and are best measured on different bases. Backend, frontend, and SDK sources are expressed in lines of code, whereas MDX documentation is more meaningfully represented through word count.

Table 8.2. Source-code size by component

Component	Repository path	Files	Size measure
Backend, production	`backend/src/main/java/com/seestack/`	95	4,998 lines
Backend, tests	`backend/src/test/java/com/seestack/`	7	504 lines
Frontend	`packages/web/src/`	110	6,243 lines
JavaScript SDK	`sdks/javascript/seestack-sdk.js`	1	68 lines
Java SDK	`sdks/java/SeeStack.java`	1	111 lines
Python SDK	`sdks/python/seestack_sdk.py`	1	80 lines
Documentation site, code	`docs/app/`, `docs/components/`, `docs/lib/`, `docs/scripts/`	21	543 lines
Documentation site, MDX content	`docs/content/docs/`	23	approximately 10,429 words
Total deliverable, code only	—	236	12,547 lines

Several analytical observations follow from these measurements. The frontend is approximately 1.25 times the size of the backend, a tighter ratio than in earlier iterations because the backend has grown to accommodate the new AI, load-testing, and security-scanning modules without a corresponding expansion of dashboard surface area. The three SDKs together occupy only 259 lines, reflecting the deliberate zero-dependency design that limits them to serialising a JSON envelope and transmitting it through runtime-native primitives. The backend production-to-test ratio is approximately 9.9:1, with unit tests concentrated on the five algorithmic components documented in section 6.6 alongside the API-key generator and dashboard insights aggregator, where correctness cannot be inferred solely from end-to-end behaviour. The documentation site contributes relatively little infrastructure code but substantial authored prose, which aligns with the content-first design philosophy adopted for the developer documentation.

Verification evidence from the testing material is consolidated in Table 8.3. Each verification layer reports the expected outcome, and the only defect identified during browser probing was corrected during the same validation process. Taken together, the six layers provide the evidential basis for the analytical claims developed in the remaining sections.

Table 8.3. Verification outcomes by layer

Verification layer	Artefact or method	Scope exercised	Result
Unit testing	7 JUnit 5 suites	45 test cases covering error fingerprinting, monitor classification, API-key generation, dashboard insights, PII sanitisation, security risk scoring, and load-runner hard-cap enforcement	45/45 pass
End-to-end pipeline validation	`sdks/examples/example-app.js`	3 dispatched exceptions, 2 expected groups, 8 verification points	All 8 points passed
Deployment readiness	Docker health checks and Spring Boot Actuator	5 probes: `pg_isready`, ClickHouse `/ping`, Kafka broker, `redis-cli ping`, `/actuator/health`	5/5 green
Documentation-site validation	Build, type-checking, OpenAPI regeneration, developer-experience checklist	3 build probes, 1 regeneration probe, 8 developer-experience probes	12/12 pass
Functional acceptance	19 acceptance cases	UC-1, UC-2, UC-4, and UC-5 primary and alternate paths together with prerequisite primitives	19/19 pass

The system's dependency footprint falls into three clear tiers, shown in Table 8.4. Backend and frontend dependency counts are moderate for frameworks of their class, whereas the documentation site introduces a smaller documentation-focused package set. The most analytically significant result is that all three SDKs maintain zero external dependencies, reducing the supply-chain surface at the integration point to zero.

Table 8.4. Declared direct dependencies by component

Component	Declared direct dependencies	Evidence
Backend	Approximately 17 dependencies	`backend/build.gradle`
Frontend	Approximately 19 production packages	`packages/web/package.json`
Documentation site	11 production packages	`docs/package.json`
JavaScript SDK	0 external dependencies	Uses native `fetch` from Node.js 18
Java SDK	0 external dependencies	Uses `java.net.http.HttpClient` from JDK 11
Python SDK	0 external dependencies	Uses `urllib.request` from the standard library

The deployment profile of the runtime is presented in Table 8.5. The operational surface of a non-development deployment is intentionally small, with only two environment variables requiring explicit operator input and all other settings defaulted for convenience. A further deployment-strengthening decision is the consolidation of approximately thirty exploratory migrations into a single canonical Flyway migration, allowing a fresh database to reach the target schema in one reproducible step.

Table 8.5. Deployment profile of the delivered runtime

Property	Value	Evidence
Runtime containers	5 containers: `postgres`, `clickhouse`, `kafka`, `redis`, `backend`	`infra/docker/docker-compose.yml`
Cold-start time to dashboard-ready state	Approximately 30 seconds	Implementation evidence cited in the source chapter
Flyway migrations applied at startup	4 sequenced migrations (V1–V4)	`backend/src/main/resources/db/migration/V1__schema.sql` together with `V2__security_scans.sql`, `V3__load_tests.sql`, and `V4__security_scan_analysis.sql`

ClickHouse initialisation scripts	1 script	`backend/src/main/resources/clickhouse/init.sql`
Earlier exploratory migration history	Approximately 30 incremental migrations consolidated into `V1__schema.sql` prior to the addition of V2–V4	Database implementation evidence cited in the source chapter
Mandatory environment variables	2: `POSTGRES_PASSWORD`, `SEESTACK_JWT_SECRET`; `OPENAI_API_KEY` is conditionally required to enable AI-assisted analysis	Backend environment-variable inventory
Optional environment variables with defaults	9 (including `OPENAI_MODEL` defaulted to `gpt-4o-mini`)	Backend environment-variable inventory

The documentation site itself contains three analytically distinct layers, summarised in Table 8.6. Rendering and routing logic provide the infrastructural layer, narrative MDX pages represent the primary authored documentation effort, and REST reference material is generated automatically from the shared OpenAPI specification. This arrangement makes the documentation surface predominantly narrative while also preventing structural drift between the published API reference and the implemented contract.

Table 8.6. Composition of the documentation site

Layer	Path	Files	Size or extent	Origin
Rendering and routing infrastructure	`docs/app/`, `docs/components/`, `docs/lib/`, `docs/scripts/`	21	543 lines	Hand-authored TypeScript integration code built on Fumadocs
Narrative MDX documentation	`docs/content/docs/(sdk)/`	16	approximately 10,429 words across narrative pages	Hand-authored project documentation
Auto-generate	`docs/content/docs/api/`	7	regenerated from `docs/openapi-sdk.json`	Generated by `docs/scripts/generate-`

d REST reference				`openapi.mts` duri ng build

8.3 Architectural-Decision Analysis and Comparative Positioning

The thirteen engineering refinements catalogued in Table 6.1 are evaluated in Table 8.7 through two analytical dimensions: observable consequence and trade-off. This approach is more informative than merely listing design choices because it ties each decision to a concrete property of the delivered system and to the cost incurred in achieving that property. Collectively, these decisions define the practical engineering contribution of the project.

Table 8.7. Analytical evaluation of the principal implementation decisions

No.	Design decision	Observable consequence in the delivered system	Trade-off or cost
1	Dual-database persistence using PostgreSQL and ClickHouse	Time-series error events and metric samples are stored in ClickHouse, while transactional metadata remains in PostgreSQL with `ON DELETE CASCADE`; this separation is supported by the functional evidence cited for group upsert and cascade deletion.	Two database engines must be administered instead of one.
2	Zero-dependency SDK design	Integrators consume a single source file with no transitive package tree, and cross-language fingerprint equality is supported by the cited functional evidence.	SDK implementation is restricted to runtime-native primitives, excluding convenience libraries such as `axios` or `requests`.
3	Deterministic SHA-256 error fingerprinting	Recurrent exceptions collapse into one `error_groups` record with an incremented occurrence counter, as supported by the cited unit and functional evidence.	Fingerprint definitions must remain stable across releases because rotation on SDK upgrade is not supported.
4	Apache Kafka 3.8 in KRaft mode	The messaging layer operates with one container rather than the two-container ZooKeeper-based arrangement used in older Kafka deployments.	Older tooling that assumes ZooKeeper compatibility cannot be used directly.
5	One-time ingest-key reveal with SHA-256 hash storage	Database compromise would expose only hashes rather than raw ingest keys, and the one-time reveal property is supported by the cited verification evidence.	A lost key cannot be recovered and must be regenerated.

6	Consolidated single-migration schema	A fresh database reaches the intended schema in one canonical Flyway step rather than replaying approximately thirty incremental migrations.	Historical migration lineage is not preserved for earlier exploratory deployments.
7	Hand-written HS256 JWT issuer	The signing path remains within a small inspectable Java codebase without dependence on a third-party JWT library, and its correctness is supported by the cited test evidence.	Advanced features such as JWKS support and key rotation by `kid` are absent.
8	Multi-stage Docker build for the backend	The production image excludes JDK tooling, reducing the operational attack surface relative to a single-stage image.	Build-time debugging utilities such as `jshell` and `jstack` are unavailable in the production image.
9	Internal REST-plus-Kafka contract	The ingest endpoint returns HTTP 202 after enqueueing, which decouples SDK latency from persistence throughput; this behaviour is supported by the cited end-to-end evidence.	Kafka failure is not surfaced directly to the SDK caller and instead depends on downstream broker monitoring.
10	Forward-looking documentation scaffold using Fumadocs and OpenAPI-driven reference generation	The site combines sixteen hand-authored narrative pages with five auto-generated REST reference pages and exposes additional LLM-oriented documentation endpoints.	Some forward-looking documentation refers to later-phase modules that are not yet backed by implemented endpoints.
11	Zero-dependency OpenAI integration with pre-dispatch PII sanitisation	GPT-4o-mini analyses runtime errors without introducing an OpenAI SDK dependency or forwarding raw secrets, and analyses are cached in `ai_error_analyses` for repeat retrieval.	Streaming responses, function calling, and richer model APIs are unavailable through the hand-rolled client.
12	In-process load runner with hard caps and per-target cooldown	Developers can run bounded load tests against their own monitors without external tooling, while the 100-request cap, 10-concurrency cap, and 60-second cooldown make abuse difficult.	The runner is not distributed and cannot reproduce realistic production load profiles.
13	Custom port and security-header analyser with	Integrators receive immediate visibility of exposed services and missing security headers without	There is no CVE matching, no payload-based vulnerability discovery, and no authenticated-scan capability; the module is

	educational scope	depending on Trivy, Snyk, or nuclei.	positioned as educational rather than as a production penetration-testing or CVE-matching tool.

Comparative positioning against the tools examined in the literature review covers the delivered scope only. The five capability areas now fully implemented and evidenced are AI-assisted error analysis, runtime error monitoring with fingerprint-based grouping, real-time website availability monitoring, automated load testing, and educational security scanning. Table 8.8 compares each capability across the four reference systems together with the cross-cutting properties of SDK dependency footprint and self-hosting posture.

Table 8.8. Comparative positioning within the delivered scope

Capability within delivered scope	Sentry	Prometheus + Grafana	Datadog	ELK stack	SeeStack
Error detection with fingerprint-based grouping	First-class support	Outside core scope	First-class support	Supported with Elastic APM add-on	First-class support through SHA-256 fingerprinting
Application-layer SDK for runtime exception capture	Available across multiple languages	Exporter-oriented rather than application-SDK-oriented	Available	Available with agents or add-ons	Available in three languages
External dependencies on the SDK surface	Multiple dependencies	Not applicable	Multiple dependencies	Multiple dependencies	Zero external dependencies
Real-time website availability monitoring	Not part of core product, available separately through Uptime	Supported with `blackbox_exporter`	Available through Synthetics	Supported with Heartbeat	Available through the scheduled monitor flow

Self-hostable with a single-command runtime	Self-hostable but operationally complex	Self-hostable	SaaS-only	Self-hostable	Self-hostable through `docker compose up`
AI-assisted error analysis with structured remediation	Available through Sentry Seer	Outside core scope	Available through Bits AI	Limited add-on summarisation	First-class through GPT-4o-mini integration with PII pre-sanitisation as a designed property
Automated load testing within the platform surface	Not part of the core product	External through `blackbox_exporter` and adjacent tooling	Available through Synthetics	Not applicable	Available through an in-process bounded runner with cooldown enforcement
Vulnerability and exposure scanning	Not applicable	Not applicable	Available through Cloud Security Management	Available through ELK security integrations	Available through an educational port and security-header analyser, not a production penetration-testing or CVE-matching tool
Cost and distribution profile at this scope level	Commercial SaaS with free tier	Open source and self-administered	Commercial SaaS	Open source or Elastic-licensed	Self-hostable senior-project artefact, not publicly released

Three analytical conclusions emerge from the comparison. First, SeeStack now operates in the same general solution space as the incumbent tools across all five delivered capabilities rather than within a

narrower observability subset. Second, the clearest differentiators are the zero-dependency SDK surface, the single-command self-hosting model, and the privacy-aware AI integration, a combination that none of the reference systems jointly offer at the same operational scale. Third, the educational caveat on the security module remains an honest limitation that distinguishes the contribution from the production-grade scanners in Datadog and the ELK stack.

8.4 Scope Realisation and Threats to Validity

The requirements defined a four-module product vision consisting of error tracking, performance monitoring, automated stress testing, and security scanning, supported by an AI-analysis layer spanning these modules. Implementation evidence shows that the present iteration realises four flows at demonstrable depth—runtime error monitoring, website availability monitoring, AI-assisted error analysis, and automated load testing—together with a fifth flow at educational depth in security scanning. Table 8.9 distinguishes clearly between implemented and deferred modules so that the analysis remains aligned with the evidence base.

Table 8.9. Scope realisation against the original product vision

Envisioned module	Delivery status	Evidence in the codebase	Analytical interpretation
Runtime error tracking, including UC-1	Delivered	Backend `errors/` module, three SDKs, dashboard `/errors` page	Supported by the cited unit and functional evidence for fingerprinting, ingestion, grouping, and cross-language consistency.
Website availability monitoring	Delivered	Backend `monitors/` module, dashboard `/monitors` page	Supported by the cited unit and functional evidence for monitor classification and arming.
Performance monitoring with continuous metrics	Deferred	No implemented module	Supporting infrastructure such as Kafka and ClickHouse is present, but the feature itself is not delivered.
Automated stress testing, including UC-2	Delivered	Backend `loadtest/` module, dashboard `/load-test` page, `LoadTestRunnerLimitsTest`	Bounded in-process HTTP load runner with cooldown enforcement; distributed load generation remains future work.

Security scanning and vulnerability reporting, including UC-5	Delivered with caveats	Backend `security/` module, dashboard `/security-scan` page, `SecurityAnalyzerTest`	Educational port and security-header analyser, not a production penetration-testing or CVE-matching tool; production-grade scanning remains future work.
AI-assisted analysis, including UC-3 alerts and UC-4 root-cause support	Delivered	Backend `ai/` module, dashboard `AiAnalysisPanel` on the Error Detail page, `ErrorDataSanitizerTest`	Now the project's primary novel capability; production-grade evaluation against a labelled regression dataset remains future work.

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

SeeStack delivers a self-hostable observability platform that implements five end-user capabilities: AI-assisted error analysis, runtime error monitoring, website availability monitoring, automated load testing, and security scanning. AI-assisted analysis is the project's primary novel contribution and is foregrounded throughout the platform, while the remaining four capabilities form the supporting observability and assurance surface. Continuous performance monitoring with host-side metric collection is the only originally envisioned module that remains outside the realised scope. The depth-across-breadth strategy that produced this artefact is the central contribution of the project because it demonstrates that a multi-capability distributed platform can be delivered credibly within senior-project constraints when scope control, architectural discipline, and verification are treated as primary design principles.

The implemented platform consists of a Spring Boot 3.4.4 backend organised into seven feature modules, a React 19 dashboard, three first-party SDKs for JavaScript, Java, and Python, a PostgreSQL and ClickHouse persistence layer governed by four Flyway migrations, and a Fumadocs-based documentation site generated in part from a shared OpenAPI specification. The complete stack is containerised and reaches dashboard readiness in about thirty seconds through a single `docker compose up` command. The delivered codebase contains approximately 12,547 lines of source code, 10,429 words of authored documentation across twenty-three pages, and forty-five unit tests across seven JUnit 5 suites covering five algorithmic components together with the API-key generator and the dashboard insights aggregator. These implementation outcomes show that the project is a working software system supported by deployment evidence, documentation, and a reproducible testing and analysis pipeline.

The project's novelty lies in four reinforcing properties. First, the SDKs use only runtime-provided HTTP primitives, which creates a zero-dependency integration surface and reduces the supply-chain exposure commonly introduced through external package ecosystems. Second, the platform supports single-command self-hosting, making it practical for small teams and academic environments that require direct control over infrastructure and trusted code. Third, the documentation workflow is structurally aligned with implementation because the REST reference is regenerated from the same OpenAPI specification used by the system, which prevents drift between published documentation and actual endpoint behaviour. Fourth, AI-assisted error analysis is delivered as a privacy-aware capability: a dedicated sanitiser scrubs bearer tokens, OpenAI-style keys, JSON Web Tokens, long hexadecimal strings, and known sensitive metadata keys before any payload leaves the backend for the OpenAI endpoint, making privacy a designed property rather than an afterthought.

These characteristics make the platform applicable in two settings. The first is self-hosted deployment for small teams or academic users who value transparency, low operational friction, and the ability to inspect the full trusted codebase, including the compact JWT implementation, the four-migration Flyway schema, the privacy-aware AI integration, and the bounded load runner. The second is educational use, where the project provides a complete example of how an

observability platform—including AI-assisted analysis—can be designed, implemented, documented, tested, and analytically evaluated within a limited academic timeframe. The project is therefore valuable both as a usable software artefact and as an evidence-backed engineering case study.

9.2 Future Work

Future work follows directly from the delivered architecture, the acknowledged testing limitations, and the operational requirements of a broader production deployment. The first priority is production-grade hardening of the four newly delivered capabilities—AI-assisted analysis, automated load testing, security scanning, and the supporting infrastructure that surrounds them—together with completion of the only originally envisioned module that remains outside scope, continuous performance monitoring. These extensions do not require architectural redesign because the project already provides reusable foundations through its Kafka topic contract, ClickHouse time-to-live compaction, four-migration Flyway schema, OpenAPI-driven documentation pipeline, and `X-SeeStack-Key` authentication flow.

Table 9.1. Future-work capabilities and implementation prerequisites

No.	Future-work capability	Related requirement	Principal prerequisite for implementation
1	Production-grade AI evaluation harness with prompt-drift monitoring	UC-4 hardening	A curated dataset of analysed error groups with expert-rated remediation steps, plus an evaluation pipeline that measures fidelity, severity calibration, and confidence calibration across model versions before release.
2	Production vulnerability scanning beyond educational port analysis	UC-5 hardening	Integration with Trivy or Snyk for dependency CVE matching and nuclei for runtime probing, together with an OWASP Top 10 or CIS report mapping.
3	Continuous performance monitoring	Product-features scope	A host-side agent that emits OpenTelemetry metrics into the existing ClickHouse-oriented telemetry pipeline.

A second line of future work concerns stronger evaluation evidence. The present report acknowledges several limitations, including reliance on manual browser-level verification, the absence of formal benchmarking, limited end-to-end sample size, AI evaluation that is not yet anchored to a labelled regression dataset, and competitor comparison drawn from secondary sources rather than matched empirical deployments. Addressing these limitations through controlled benchmarking, distributed load validation, AI-evaluation rigour, and security-scan validation against authoritative CVE feeds would strengthen the empirical basis of the project without changing its feature scope, and is therefore particularly important for any future academic extension or publication-oriented version of the work.

REFERENCES

1) **Amazon Web Services.**
(n.d.). ***Amazon S3 and Glacier storage documentation*****.** **https://aws.amazon.com**

2) **Apache Kafka Project. (n.d.).** ***Apache Kafka: A distributed streaming platform*****.** **https://kafka.apache.org**

3) **ClickHouse.**
(n.d.). ***ClickHouse: Open-source column-oriented database management system*****.** **https://clickhouse.com**

4) **Cortex Project. (n.d.).** ***Cortex: Horizontally scalable Prometheus-as-a-service*****.** **https://cortexmetrics.io**

5) **Datadog, Inc.**
(n.d.). ***Datadog cloud monitoring and security platform*****.** **https://www.datadoghq.com**

6) **Docker, Inc. (n.d.).** ***Docker container platform documentation*****.** **https://docs.docker.com**

7) **Elasticsearch Project.**
(n.d.). ***Elasticsearch: Distributed search and analytics engine*****.** **https://www.elastic.co/elasticsearch**

8) **GitLab Inc. (n.d.).** ***GitLab CI/CD documentation*****.** **https://docs.gitlab.com/ee/ci**

9) **Google Cloud.**
(n.d.). ***Google Cloud Platform (GCP) documentation*****.** **https://cloud.google.com**

10) **Grafana Labs.**
(n.d.). ***Grafana observability and data visualization platform*****.** **https://grafana.com**

11) **HashiCorp. (n.d.).** ***Terraform: Infrastructure as code tool*****.** **https://www.terraform.io**

12) **IBM. (n.d.).** ***IBM Cloud observability and monitoring*****.** **https://www.ibm.com/cloud**

13) **Jaeger Project. (n.d.).** ***Jaeger: Distributed tracing platform*****.** **https://www.jaegertracing.io**

14) **Jenkins Project. (n.d.).** ***Jenkins: Open-source automation server*****.** **https://www.jenkins.io**

15) **JMeter Project.**
(n.d.). ***Apache JMeter: Load testing and performance measurement tool*****.** **https://jmeter.apache.org**

16) **LoadRunner (OpenText).**
(n.d.). ***LoadRunner professional documentation*****.** **https://www.opentext.com/products/loadrunner-professional**

17) **Locust. (n.d.).** ***Locust load testing tool documentation*****.** **https://locust.io**

18) **Loki Project.**
(n.d.). ***Loki: Log aggregation system for Grafana*****.** **https://grafana.com/oss/loki**

19) **Microsoft. (n.d.).** ***Azure cloud services documentation*****.** **https://learn.microsoft.com/azure**

20) **National Institute of Standards and Technology.**
(n.d.). ***NIST Cybersecurity Framework (CSF)*****.** https://www.nist.gov/cyberframework

21) **OpenAI.**
(n.d.). ***GPT-4 technical overview and API documentation*****.** https://platform.openai.com/docs

22) **OpenTelemetry Project.**
(n.d.). ***OpenTelemetry specification and SDKs*****.** https://opentelemetry.io

23) **OWASP Foundation. (n.d.).** ***OWASP Top 10 – Web application security risks*****.** https://owasp.org

24) **PagerDuty.**
(n.d.). ***PagerDuty digital operations management platform*****.** https://www.pagerduty.com

25) **Payment Card Industry Security Standards Council.**
(n.d.). ***PCI data security standard (PCI-DSS)*****.** https://www.pcisecuritystandards.org

26) **Prometheus Authors.**
(n.d.). ***Prometheus monitoring system and time series database*****.** https://prometheus.io

27) **Qualys.**
(n.d.). ***Qualys cloud platform – Vulnerability management*****.** https://www.qualys.com

28) **Rapid7. (n.d.).** ***InsightVM vulnerability management*****.** https://www.rapid7.com

29) **React Team. (n.d.).** ***React: A JavaScript library for building user interfaces*****.** https://react.dev

30) **Redis Project. (n.d.).** ***Redis in-memory data store documentation*****.** https://redis.io

31) **Sentry. (n.d.).** ***Sentry application monitoring and error tracking*****.** https://sentry.io

32) **SonarSource.**
(n.d.). ***SonarQube: Continuous code quality inspection*****.** https://www.sonarsource.com/products/sonarqube

33) **Spring Team. (n.d.).** ***Spring Boot documentation*****.** https://spring.io/projects/spring-boot

34) **The Kubernetes Authors.**
(n.d.). ***Kubernetes: Production-grade container orchestration*****.** https://kubernetes.io

35) **Thanos Project. (n.d.).** ***Thanos: Highly available Prometheus setup with long-term storage*****.** https://thanos.io

APPENDIX: Glossary

ADR (Architecture Decision Record): Documentation of major technical decisions made during system design, including the rationale, trade-offs considered, and consequences of each decision. ADRs help future developers understand why specific architectural approaches were chosen for SeeStack.

API (Application Programming Interface): A set of protocols, tools, and definitions for building application software that specifies how different software components should interact. SeeStack provides RESTful APIs for SDK integration, alerting, and third-party integrations.

API Key: A unique identifier issued to projects in SeeStack for programmatic access and authentication. API keys are automatically revoked if credentials are compromised or suspicious activity is detected.

APM (Application Performance Monitoring): A category of software tools designed to monitor and analyze the performance and availability of applications. SeeStack integrates APM capabilities through its performance monitoring module.

Audit Log: Comprehensive record of all user actions including login, data access, configuration changes, and API key generation/revocation with timestamps, user identities, and IP addresses enabling forensic analysis.

Backend Services: Server-side applications built with Spring Boot, Kafka, and Redis that handle event processing, data storage, and API requests for the SeeStack platform.

ClickHouse: Fast open-source column-oriented database management system used by SeeStack for efficient storage and querying of time-series data and events. Supports distributed deployments and data compression.

CloudFormation: AWS infrastructure-as-code service enabling provisioning and management of AWS resources through version-controlled configuration files used for SeeStack deployments.

CPU Utilization: Percentage of CPU processing capacity being used. SeeStack maintains CPU utilization below 70% during normal operations and below 85% during peak load.

CVE (Common Vulnerabilities and Exposures): Standardized identifiers for publicly disclosed cybersecurity vulnerabilities. SeeStack dependencies are regularly scanned for CVEs, with critical patches applied within 48 hours.

Email OTP (One-Time Password): Passwordless authentication method where SeeStack sends a time-limited code to user's registered email address, valid for 15 minutes enabling secure login without passwords.

Encryption at Rest: Cryptographic protection of data stored in databases, caches, and file storage using AES-256 encryption managed through separate key management services.

Encryption in Transit: Cryptographic protection of data transmitted between clients and SeeStack servers using HTTPS/TLS 1.2 or later with strong cipher suites.

EPS (Events Per Second): Throughput metric measuring the number of events (errors, metrics, traces) processed by SeeStack per second. Target: 50,000 EPS during peak traffic.

Event Ingestion Pipeline: Backend system built on Apache Kafka and Spring Boot that receives, validates, and processes telemetry events from SDKs and monitoring agents.

Exponential Backoff: Retry strategy where wait time between attempts increases exponentially (5 seconds, 10 seconds, 30 seconds) to prevent overwhelming systems during transient failures.

Failover: Automatic switching to secondary servers or data centers when primary infrastructure fails, ensuring continued service availability.

Fingerprinting: Process of generating unique identifiers for error events using normalized stack traces, enabling SeeStack to group similar errors together.

HMAC-SHA256 (Hash-based Message Authentication Code): Cryptographic algorithm used by SeeStack to verify webhook signatures ensuring callbacks originate from SeeStack and have not been tampered with.

HTTPS: Secure variant of HTTP using TLS encryption, mandatory for all SeeStack connections to protect data in transit.

Identity Provider: External services (OAuth 2.0, SAML 2.0) managing user authentication and authorization for SeeStack single sign-on integrations.

Incident Response: Process of detecting, investigating, and resolving security incidents or system failures, enhanced through SeeStack's unified monitoring and alerting.

ISO 27001: International standard for information security management systems. SeeStack pursues ISO 27001 certification demonstrating security maturity.

ISO 27002: Standard providing best practices and guidelines for information security management, informing SeeStack security architecture.

JWT (JSON Web Token): Standard for representing claims securely between parties. SeeStack generates JWT access tokens with 10-hour expiration and JWT refresh tokens also expiring after 10 hours for authentication.

Kafka (Apache Kafka): Distributed event streaming platform used by SeeStack for high-throughput, real-time data feeds and asynchronous processing of error logs and metrics with automatic partitioning for scalability.

KMS (Key Management Service): Secure key management service storing and rotating encryption keys separately from data storage systems, preventing attackers from decrypting data.

Kubernetes: Container orchestration platform enabling automated deployment, scaling, and management of containerized SeeStack services with health checks and automatic restarts.

Latency: Delay before data transfer begins following an instruction, measured in milliseconds in SeeStack (target: 2-second dashboard load, 500ms API response at P95).

Load Balancer: Network component distributing incoming requests across multiple backend servers, enabling SeeStack horizontal scaling and high availability.

Load Testing: Process of simulating user load on systems to measure performance under stress, implemented through SeeStack stress testing module using Locust.

Locust: Open-source load testing tool used by SeeStack to simulate concurrent users and generate stress test traffic in a distributed manner.

Machine Learning: Algorithms enabling systems to learn patterns from data without explicit programming, used in SeeStack for anomaly detection, error classification, and recommendations.

Magic Link: Passwordless authentication method where SeeStack sends email-based link that when clicked authenticates the user without requiring password entry.

Materialized Views: Pre-computed query results stored in database for fast access, used by SeeStack to accelerate dashboard queries on large datasets.

Mean Time to Detect (MTTD): Average time from incident occurrence to detection, targeted for reduction through SeeStack's automated error detection.

Mean Time to Resolution (MTTR): Average time from incident detection to full resolution, significantly reduced through SeeStack's AI-powered recommendations and unified monitoring.

Metrics: Quantitative measurements of system behavior (CPU, memory, response time) collected and visualized by SeeStack for performance monitoring.

Microsoft Teams: Enterprise communication platform supported as webhook notification channel for SeeStack alerts.

Microservices: Architectural style structuring applications as loosely coupled, independently deployable services, the primary deployment pattern SeeStack monitors.

Multi-Factor Authentication (MFA): Security method requiring multiple verification factors (e.g., password + OTP) for account access, supported in SeeStack for administrators.

NullPointerException: Common Java exception when code attempts to use objects that are null, used as example in SeeStack error tracking use cases.

OAuth 2.0: Standard authorization protocol used for SSO integrations with SeeStack, enabling secure delegation of user authentication to external identity providers.

OpenAI: Provider of GPT-4 language models integrated into SeeStack for analyzing error logs and generating remediation suggestions.

OpenTelemetry: Open-source project providing libraries and standards for distributed tracing and metrics collection, used by SeeStack for observability.

P50 / P95 / P99: Percentile latency metrics showing response times for 50%, 95%, and 99% of requests. SeeStack targets P95 latency < 500ms for API requests.

RBAC (Role-Based Access Control): Authorization model assigning permissions based on user roles, enabling SeeStack project-level access control with granular permissions.

Recovery Point Objective (RPO): Maximum acceptable amount of data loss measured from backup, target RPO for SeeStack: 1 hour of data loss maximum.

Recovery Time Objective (RTO): Maximum acceptable time to restore full service after failure, target RTO for SeeStack: 4 hours maximum.

Redis: In-memory data structure store used by SeeStack as database, cache, and message broker for high-performance data operations and session management.

RESTful API: Architectural style for HTTP-based APIs using standard methods (GET, POST, PUT, DELETE), used by SeeStack for integration endpoints.

RPS (Requests Per Second): Throughput metric measuring request processing rate, monitored in SeeStack stress testing and production monitoring.

Scalability: System ability to handle increasing load through adding resources without code changes, a primary design goal for SeeStack.

SDK (Software Development Kit): Collection of tools and libraries enabling developers to integrate SeeStack monitoring into their applications, provided for multiple programming languages.

SeeStack: AI-Powered Error Detection and Monitoring System for Developers, the subject of this project report.

Session Timeout: Duration after which inactive user sessions automatically expire, set to 30 minutes for web users and 24 hours for API tokens in SeeStack.

Spring Boot: Java framework for building microservices and applications, used for implementing SeeStack backend services and API endpoints.

SSH (Secure Shell): Encrypted protocol for secure remote access, used for administrative access to SeeStack infrastructure.

SSL/TLS: Cryptographic protocols enabling encrypted communication, with TLS 1.2 or later mandatory for all SeeStack connections.

Stack Trace: Detailed record of function calls showing execution path when error occurred, captured and analyzed by SeeStack error detection.

Staging Environment: Pre-production environment for testing configurations and deployments before production release.

Stress Testing: Load testing approach applying pressure beyond normal capacity to identify performance limits and failure points, automated in SeeStack.

Structured Logging: Logging with consistent JSON format including correlation IDs enabling request tracing through system layers.

Telemetry: Automatic recording and transmission of data from remote applications to centralized system for monitoring, collected by SeeStack SDKs and agents.

Terraform: Infrastructure-as-code tool using HCL for provisioning cloud resources, enabling reproducible SeeStack infrastructure deployment.

TimeoutException: Common application exception when operations exceed time limits, detected and analyzed by SeeStack error detection.

Time-Series Data: Sequences of observations ordered by time, efficiently stored in ClickHouse used by SeeStack for metrics and events.

TOTP (Time-Based One-Time Password): MFA method generating time-limited codes from shared secrets, supported by SeeStack for account security.

Traceability: Ability to track changes and correlate events across system components, enabled through SeeStack audit logs and distributed tracing.

Uptime: Duration system operates continuously without failure, measured as percentage (target: 99.9% for SeeStack).

Virtual Users: Simulated user requests in stress testing, generated by SeeStack up to 100,000 concurrent virtual users for load testing.

Webhook: HTTP callbacks allowing external systems to subscribe to events and receive notifications when specific actions occur, supported by SeeStack for integrations.

WebSocket: Protocol enabling persistent bidirectional communication between client and server, used by SeeStack for real-time dashboard updates with 100ms latency target.

WSS (WebSocket Secure): Encrypted variant of WebSocket protocol using TLS, mandatory for SeeStack real-time communications.

Zero-Downtime Deployment: Deploying updates to SeeStack without service interruption through load balancer routing and graceful service transitions.